Tema 01 — Subrutinas

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 7: Subrutinas

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl, Nicolás Romo Aubele (nroma@uc.cl)

Pregunta 1: Preguntas conceptuales

a) En el computador básico del curso, la memoria de *stack* se utiliza para soportar la ejecución de subrutinas. Describa cómo está organizada esta memoria y el rol del registro `SP` (*Stack Pointer*). Luego, explique por qué es fundamental inicializar correctamente el `SP` y analice las consecuencias de una inicialización incorrecta. Finalmente, discuta cómo estos problemas se ven acentuados en presencia de llamadas recursivas y explique en qué consiste un *stack overflow* en este contexto.

b) Considera el siguiente código en *assembly* del computador básico. Explique el propósito, y cómo funciona, la secuencia de instrucciones `PUSH B`, `CALL func` y `POP B`; y cuál es el efecto de que no haya una instrucción `PUSH A` justo antes, ni una instrucción `POP A` justo después de la llamada a la función `func`.

```
DATA:
    r 3
CODE :
    MOV  A, (r)
    MOV  B, 2
    PUSH  B
    CALL  func
    POP  B
    JMP  end
    func:
        MOV A, B
        ADD A, 3
        MOV B, A
        MOV (r), B
        RET
    end:
```

Pregunta 2: Análisis de código

2.1.a) (I2 2025-2) Para el siguiente fragmento de código indique los valores de los registros `A` y `B` al finalizar la ejecución del código.

2.1.b) Para el mismo fragmento de código, indique los valores de los registros `A` y `B` al finalizar la ejecución del código, pero asumiendo que la línea con la instrucción `RET` no es parte del programa.

```
DATA:
    var1 12
    var2 4
CODE:
    MOV A,5
    MOV B,3
    RET
    case_1:
        ADD A,B // Dir. memoria 0x04
        PUSH A  // Dir. memoria 0x05
        MOV A,B // Dir. memoria 0x06
        SHL A,A // Dir. memoria 0x07
        MOV B,A // Dir. memoria 0x08
        POP A   // Dir. memoria 0x09-0x0A
        JMP end // Dir. memoria 0x0B
    case_2:
        SUB A,B // Dir. memoria 0x0C
        PUSH A  // Dir. memoria 0x0D
        MOV A,B // Dir. memoria 0x0E
        SHR A,A // Dir. memoria 0x0F
        MOV B,A // Dir. memoria 0x10
        POP A   // Dir. memoria 0x11-0x12
    end:
```

2.2) Se describirán nuevas instrucciones, considere que ya están implementadas en un computador básico con subrutinas. Considere que cada instrucción se ejecuta en un ciclo.

- `CALL B,Dir`: Almacena en el registro B la dirección de retorno PC+1 y salta a la dirección de memoria equivalente al literal Dir.
- `RET B`: Salta a la dirección de retorno almacenada en el registro B.

A continuación, se adjunta el siguiente fragmento de código escrito en el *Assembly* del computador básico, con la diferencia de que incorpora las nuevas instrucciones implementadas. Para cada fragmento, indique los valores de los registros A y B al finalizar la ejecución del código.

```
DATA:
    n 2
    res 0
CODE:
    MOV A,(n)           // Dir. mem. 0x00
    CALL B,fun          // Dir. mem. 0x01
    JMP end             // Dir. mem. 0x02
        fun:
            CMP A,0     // Dir. mem. 0x03
            JLE fun_end // Dir. mem. 0x04
            PUSH B      // Dir. mem. 0x05
            MOV B,A     // Dir. mem. 0x06
            ADD A,(res) // Dir. mem. 0x07
            MOV (res),A // Dir. mem. 0x08
            MOV A,B     // Dir. mem. 0x09
            SUB A,1     // Dir. mem. 0x0A
            CALL B,fun  // Dir. mem. 0x0B
            POP B       // Dir. mem. 0x0C-0x0D
        fun_end:
            RET B       // Dir. mem. 0x0E

    end:
        MOV A,(res)     // Dir. mem. 0x0F
```

Pregunta 3: Modificaciones al computador básico

En varias arquitecturas se hace uso del registro *base pointer* (`BP`). Este funciona como punto de referencia para obtener los parámetros y dirección de retorno de cada llamada de una subrutina, lo que facilita el manejo de llamados anidados. Deberá modificar la microarquitectura del computador básico para implementar el registro `BP` de 8 *bits* y su funcionamiento. Se le listarán las instrucciones que deben ser implementadas. Para cada una de ellas, debe incluir la combinación completa de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el nombre de la entrada escogida ("-" si no afecta). Puede realizar todas las modificaciones en un solo diagrama.

- `MOV BP, SP`. Almacena en el registro `BP` el valor del registro `SP`.
- `PUSH BP`. Almacena en `Mem[SP]` el valor de `BP` y decrementa `SP` en una unidad.
- `POP BP`. Incrementa `SP` en una unidad y almacena en `BP` el valor `Mem[SP]`.
- `MOV Reg,(BP + Lit)`. Almacena en `Reg (A o B)` el valor `Mem[BP+Lit]`.

Instrucción	Operandos	*Opcode*	Condición	L_{PC}	L_A	L_B	S_A0,1	S_B0,1	S_{Op}0,1,2	S_{Add}0,1	S_{Din}	S_{PC}	W	Inc_{SP}	Dec_{SP}
MOV	A,B	0000000		0	1	0	ZERO	B	ADD	-	-	-	0	0	0
	B,A	0000001		0	0	1	A	ZERO	ADD	-	-	-	0	0	0
	A,Lit	0000010		0	1	0	ZERO	LIT	ADD	-	-	-	0	0	0
	B,Lit	0000011		0	0	1	ZERO	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0000100		0	1	0	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	B,(Dir)	0000101		0	0	1	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	(Dir),A	0000110		0	0	0	A	ZERO	ADD	LIT	ALU	-	1	0	0
	(Dir),B	0000111		0	0	0	ZERO	B	ADD	LIT	ALU	-	1	0	0
	A,(B)	0001000		0	1	0	ZERO	DOUT	ADD	B	-	-	0	0	0
	B,(B)	0001001		0	0	1	ZERO	DOUT	ADD	B	-	-	0	0	0
	(B),A	0001010		0	1	0	A	ZERO	ADD	B	ALU	-	1	0	0
ADD	A,B	0001011		0	1	0	A	B	ADD	-	-	-	0	0	0
	B,A	0001100		0	0	1	A	B	ADD	-	-	-	0	0	0
	A,Lit	0001101		0	1	0	A	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0001110		0	1	0	A	DOUT	ADD	LIT	-	-	0	0	0
	A,(B)	0001111		0	1	0	A	DOUT	ADD	B	-	-	0	0	0
	(Dir)	0010000		0	0	0	A	B	ADD	LIT	ALU	-	1	0	0
SUB	A,B	0010001		0	1	0	A	B	SUB	-	-	-	0	0	0
	B,A	0010010		0	0	1	A	B	SUB	-	-	-	0	0	0
	A,Lit	0010011		0	1	0	A	LIT	SUB	-	-	-	0	0	0
	A,(Dir)	0010100		0	1	0	A	DOUT	SUB	LIT	-	-	0	0	0
	A,(B)	0010101		0	1	0	A	DOUT	SUB	B	-	-	0	0	0
	(Dir)	0010110		0	0	0	A	B	SUB	LIT	ALU	-	1	0	0
AND	A,B	0010111		0	1	0	A	B	AND	-	-	-	0	0	0
	B,A	0011000		0	0	1	A	B	AND	-	-	-	0	0	0
	A,Lit	0011001		0	1	0	A	LIT	AND	-	-	-	0	0	0
	A,(Dir)	0011010		0	1	0	A	DOUT	AND	LIT	-	-	0	0	0
	A,(B)	0011011		0	1	0	A	DOUT	AND	B	-	-	0	0	0
	(Dir)	0011100		0	0	0	A	B	AND	LIT	ALU	-	1	0	0
OR	A,B	0011101		0	1	0	A	B	OR	-	-	-	0	0	0
	B,A	0011110		0	0	1	A	B	OR	-	-	-	0	0	0
	A,Lit	0011111		0	1	0	A	LIT	OR	-	-	-	0	0	0
	A,(Dir)	0100000		0	1	0	A	DOUT	OR	LIT	-	-	0	0	0
	A,(B)	0100001		0	1	0	A	DOUT	OR	B	-	-	0	0	0
	(Dir)	0100010		0	0	0	A	B	OR	LIT	ALU	-	1	0	0
NOT	A,A	0100011		0	1	0	A	-	NOT	-	-	-	0	0	0
	B,A	0100100		0	0	1	A	-	NOT	-	-	-	0	0	0
	(Dir)	0100101		0	0	0	A	B	NOT	LIT	ALU	-	1	0	0
XOR	A,B	0100110		0	1	0	A	B	XOR	-	-	-	0	0	0
	B,A	0100111		0	0	1	A	B	XOR	-	-	-	0	0	0
	A,Lit	0101000		0	1	0	A	LIT	XOR	-	-	-	0	0	0
	A,(Dir)	0101001		0	1	0	A	DOUT	XOR	LIT	-	-	0	0	0
	A,(B)	0101010		0	1	0	A	DOUT	XOR	B	-	-	0	0	0
	(Dir)	0101011		0	0	0	A	B	XOR	LIT	ALU	-	1	0	0
SHL	A,A	0101100		0	1	0	A	-	SHL	-	-	-	0	0	0
	B,A	0101101		0	0	1	A	-	SHL	-	-	-	0	0	0
	(Dir)	0101110		0	0	0	A	B	SHL	LIT	ALU	-	1	0	0
SHR	A,A	0101111		0	1	0	A	-	SHR	-	-	-	0	0	0
	B,A	0110000		0	0	1	A	-	SHR	-	-	-	0	0	0
	(Dir)	0110001		0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
INC	B	0110010		0	0	1	ONE	B	ADD	-	-	-	0	0	0
CMP	A,B	0110101		0	0	0	A	B	SUB	-	-	-	0	0	0
	A,Lit	0110110		0	0	0	A	LIT	SUB	-	-	-	0	0	0
JMP	Dir	0111001		1	0	0	-	-	-	-	-	LIT	0	0	0
JEQ	Dir	0111010	Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JNE	Dir	0111011	Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JGT	Dir	0111100	N=0 y Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLT	Dir	0111101	N=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JGE	Dir	0111110	N=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLE	Dir	0111111	N=1 o Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JCR	Dir	1000000	C=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JOV	Dir	1000001	V=1	1	0	0	-	-	-	-	-	LIT	0	0	0
CALL	Dir	1000010		1	0	0	-	-	-	SP	PC	LIT	1	0	1
RET		1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000100		1	0	0	-	-	-	SP	-	DOUT	0	0	0
PUSH	A	1000101		0	0	0	A	ZERO	ADD	SP	ALU	-	1	0	1
	B	1000110		0	0	0	ZERO	B	ADD	SP	ALU	-	1	0	1
POP	A	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000111		0	1	0	ZERO	DOUT	ADD	SP	-	-	0	0	0
POP	B	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1001000		0	0	1	ZERO	DOUT	ADD	SP	-	-	0	0	0
NOP		1001001		0	0	0	-	-	-	-	-	-	0	0	0

Tabla 1: ISA del computador básico.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 7: Subrutinas

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl, Nicolás Romo Aubele (nroma@uc.cl)

Pregunta 1: Preguntas conceptuales

a) En el computador básico del curso, la memoria de *stack* se utiliza para soportar la ejecución de subrutinas. Describa cómo está organizada esta memoria y el rol del registro `SP` (*Stack Pointer*). Luego, explique por qué es fundamental inicializar correctamente el `SP` y analice las consecuencias de una inicialización incorrecta. Finalmente, discuta cómo estos problemas se ven acentuados en presencia de llamadas recursivas y explique en qué consiste un *stack overflow* en este contexto.

b) Considera el siguiente código en *assembly* del computador básico. Explique el propósito, y cómo funciona, la secuencia de instrucciones `PUSH B`, `CALL func` y `POP B`; y cuál es el efecto de que no haya una instrucción `PUSH A` justo antes, ni una instrucción `POP A` justo después de la llamada a la función `func`.

```
DATA:
    r 3
CODE :
    MOV  A, (r)
    MOV  B, 2
    PUSH  B
    CALL  func
    POP  B
    JMP  end
    func:
        MOV A, B
        ADD A, 3
        MOV B, A
        MOV (r), B
        RET
    end:
```

Solución: a) En el computador básico, la memoria de *stack* corresponde a una región de la memoria de datos ubicada en el segmento inferior, utilizada para almacenar información temporal como direcciones de retorno y valores de registros. El acceso se realiza mediante el registro `SP` (*Stack Pointer*), que se inicializa en la última dirección de memoria (por ejemplo, 255 en una memoria de 8 *bits*). A medida que se ejecutan instrucciones como `PUSH` o `CALL`, el *stack* crece hacia direcciones menores, decrementando el valor de `SP`.

Inicializar correctamente el `SP` es fundamental para asegurar que el *stack* tenga suficiente espacio para crecer sin interferir con las variables del programa, manteniendo separadas ambas regiones de memoria.

Si el `SP` se inicializa incorrectamente, por ejemplo en la dirección 0, o en la 64, pueden producirse errores graves. Ya que el *stack* podría sobrescribir variables del programa, generando corrupción de datos, y las instrucciones `CALL` y `RET` pueden utilizar direcciones de retorno incorrectas, afectando el flujo de ejecución.

Este problema se vuelve aún más crítico en presencia de recursión, donde múltiples llamadas a una misma subrutina se acumulan en el *stack*. Cada llamada recursiva requiere almacenar su propia dirección de retorno y posibles datos adicionales, por lo que el uso del *stack* crece rápidamente. Si el espacio disponible es insuficiente —ya sea por una mala inicialización del `SP` o por una gran profundidad de recursión— puede ocurrir *stack overflow*, donde el *stack* colisiona con otras regiones de memoria. Esto provoca comportamiento incorrecto, pérdida de información o incluso la detención del programa.

b) El propósito es ejecutar la subrutina `func`, pero de modo que el valor del registro `B` al terminar la ejecución de `func` y volver al programa principal `(CODE)` sea el mismo que `B` tenía justo antes de hacer la llamada; es decir, las actualizaciones del registro `B` al interior de `func` no se ven reflejadas en `CODE`.

Esto ocurre porque el valor de `B` justo antes de la llamada es guardado en el *stack*, mediante `PUSH B`, y es restaurado justo después de la llamada mediante `POP B`. Por esto mismo, la ausencia de las instrucciones `PUSH A` y `POP A` tiene el efecto contrario en el caso del registro `A`: los cambios que el registro `A` sufre dentro de *func* se ven reflejados en `CODE` cuando `func` termina su ejecución.

Las instrucciones `PUSH` y `POP` acualizan el valor del *stack pointer* — justo después y justo antes, respectivamente — de tener acceso a la dirección de memoria apuntada por el *stack pointer*.

Pregunta 2: Análisis de código

2.1.a) (I2 2025-2) Para el siguiente fragmento de código indique los valores de los registros `A` y `B` al finalizar la ejecución del código.

2.1.b) Para el mismo fragmento de código, indique los valores de los registros `A` y `B` al finalizar la ejecución del código, pero asumiendo que la línea con la instrucción `RET` no es parte del programa.

```
DATA:
    var1 12
    var2 4
CODE:
    MOV A,5
    MOV B,3
    RET
    case_1:
        ADD A,B // Dir. memoria 0x04
        PUSH A  // Dir. memoria 0x05
        MOV A,B // Dir. memoria 0x06
        SHL A,A // Dir. memoria 0x07
        MOV B,A // Dir. memoria 0x08
        POP A   // Dir. memoria 0x09-0x0A
        JMP end // Dir. memoria 0x0B
    case_2:
        SUB A,B // Dir. memoria 0x0C
        PUSH A  // Dir. memoria 0x0D
        MOV A,B // Dir. memoria 0x0E
        SHR A,A // Dir. memoria 0x0F
        MOV B,A // Dir. memoria 0x10
        POP A   // Dir. memoria 0x11-0x12
    end:
```

2.2) Se describirán nuevas instrucciones, considere que ya están implementadas en un computador básico con subrutinas. Considere que cada instrucción se ejecuta en un ciclo.

- `CALL B,Dir`: Almacena en el registro B la dirección de retorno PC+1 y salta a la dirección de memoria equivalente al literal Dir.
- `RET B`: Salta a la dirección de retorno almacenada en el registro B.

A continuación, se adjunta el siguiente fragmento de código escrito en el *Assembly* del computador básico, con la diferencia de que incorpora las nuevas instrucciones implementadas. Para cada fragmento, indique los valores de los registros A y B al finalizar la ejecución del código.

```
DATA:
    n 2
    res 0
CODE:
    MOV A,(n)           // Dir. mem. 0x00
    CALL B,fun          // Dir. mem. 0x01
    JMP end             // Dir. mem. 0x02
        fun:
            CMP A,0     // Dir. mem. 0x03
            JLE fun_end // Dir. mem. 0x04
            PUSH B      // Dir. mem. 0x05
            MOV B,A     // Dir. mem. 0x06
            ADD A,(res) // Dir. mem. 0x07
            MOV (res),A // Dir. mem. 0x08
            MOV A,B     // Dir. mem. 0x09
            SUB A,1     // Dir. mem. 0x0A
            CALL B,fun  // Dir. mem. 0x0B
            POP B       // Dir. mem. 0x0C-0x0D
        fun_end:
            RET B       // Dir. mem. 0x0E

    end:
        MOV A,(res)     // Dir. mem. 0x0F
```

Solución:

2.1.a) El fragmento de código accede, por overflow del registro `SP`, a la dirección de memoria 0 (equivalente a `var1`) para cargar su valor en el registro PC, por lo que la ejecución de `RET` genera el salto al label `case_2`. A continuación, se muestra el resultado de la ejecución del fragmento:

```
DATA:
    var1 12 // MEM[0] =12
    var2 4  // MEM[1] =4
CODE:
    MOV A,5 // A= 5
    MOV B,3 // B= 3
    RET     // SP++ =>255 +1= 0,PC= MEM[0]= 12
    case_1:
        ADD A,B // NO SE EJECUTA
        PUSH A  // NO SE EJECUTA
        MOV A,B // NO SE EJECUTA
        SHL A,A // NO SE EJECUTA
        MOV B,A // NO SE EJECUTA
        POP A   // NO SE EJECUTA
        JMP end // NO SE EJECUTA
    case_2:
        SUB A,B // A= 2
        PUSH A  // MEM[SP]= A=>MEM[0]=2, SP--=>0-1
     =255
        MOV A,B // A= 3
        SHR A,A // A= 1
        MOV B,A // B= 1
        POP A   // SP++ =>255 +1= 0,A=MEM[0]= 2
    end:        // A=2, B=1
```

2.1.b) A diferencia del código anterior, dado que la instrucción `RET` ya no forma parte del programa, se entra directamente al `case_1`, y dado que existe un `JMP` antes de `case_2`, entonces esta vez no se ejecuta esta parte del código saltando directamente al final de este. A continuación, se muestra el resultado de la ejecución del fragmento:

```
DATA:
    var1 12 // MEM[0] =12
    var2 4  // MEM[1] =4
CODE:
    MOV A,5 // A= 5
    MOV B,3 // B= 3
    case_1:
        ADD A,B // A = 8
        PUSH A  // MEM[SP]= A=>MEM[0]=8, SP--=>0-1
     =255
        MOV A,B // A = 3
        SHL A,A // A = 6
        MOV B,A // B = 6
        POP A   // SP++ => 255 + 1 = 0,A=MEM[0] = 8
        JMP end // Va al al final
    case_2:
        SUB A,B // NO SE EJECUTA
        PUSH A  // NO SE EJECUTA
        MOV A,B // NO SE EJECUTA
        SHR A,A // NO SE EJECUTA
        MOV B,A // NO SE EJECUTA
        POP A   // NO SE EJECUTA
    end:        // A=8, B=6
```

2.2) El fragmento de código computa de forma recursiva la suma de la variable `n` con todos sus antecesores hasta llegar a cero, almacenando el resultado final en `res`. A continuación, se muestra el resultado de la ejecución del fragmento:

```
DATA:
    n 2                 // MEM[0]=2
    res 0               // MEM[1]=0
CODE:
    MOV A,(n)           // A = n = 2
    CALL B,fun          // B = 2, fun(n)
    JMP end
    fun:
        CMP A,0
        JLE fun_end     // Termina cuando A = 0
        PUSH B          // Respalda B en stack (retorno)
        MOV B,A         // B = A = n
        ADD A,(res)     // A += res
        MOV (res),A     // res = res + n
        MOV A,B         // A = n
        SUB A,1         // n--
        CALL B,fun      // fun(n-1)
        POP B           // Recupera B (retorno)
    fun_end:
        RET B           // Retorna de la forma esperada
    end:
    // A = res = fun(2) = 2+fun(1) = 2+1+fun(0) = 2+1+0 = 3
        MOV A,(res)     // B = Primer retorno computado = 2
```

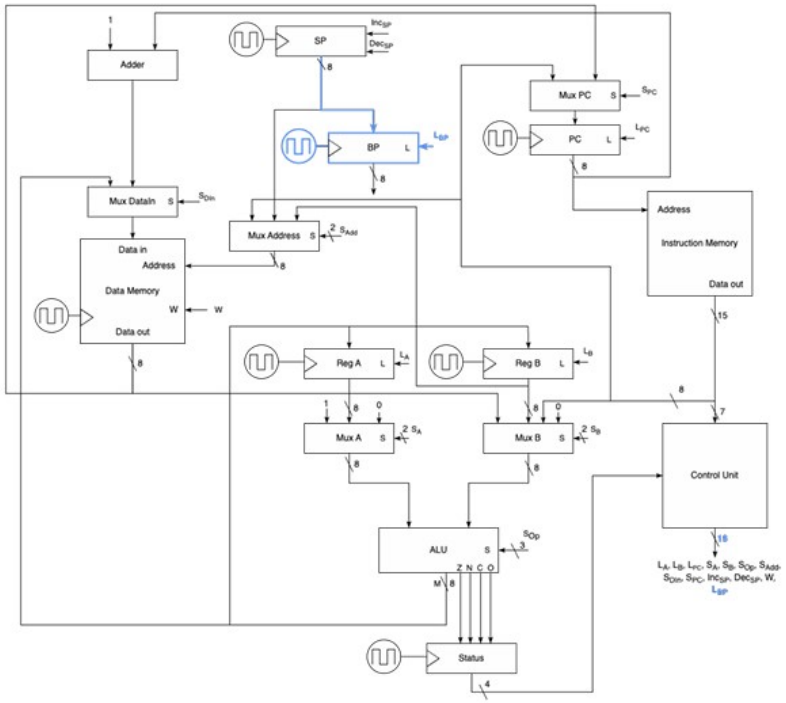
Pregunta 3: Modificaciones al computador básico

En varias arquitecturas se hace uso del registro *base pointer* (`BP`). Este funciona como punto de referencia para obtener los parámetros y dirección de retorno de cada llamada de una subrutina, lo que facilita el manejo de llamados anidados. Deberá modificar la microarquitectura del computador básico para implementar el registro `BP` de 8 *bits* y su funcionamiento. Se le listarán las instrucciones que deben ser implementadas. Para cada una de ellas, debe incluir la combinación completa de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el nombre de la entrada escogida ("-" si no afecta). Puede realizar todas las modificaciones en un solo diagrama.

- `MOV BP, SP`. Almacena en el registro `BP` el valor del registro `SP`.
- `PUSH BP`. Almacena en `Mem[SP]` el valor de `BP` y decrementa `SP` en una unidad.
- `POP BP`. Incrementa `SP` en una unidad y almacena en `BP` el valor `Mem[SP]`.
- `MOV Reg,(BP + Lit)`. Almacena en `Reg (A o B)` el valor `Mem[BP+Lit]`.

Solución:

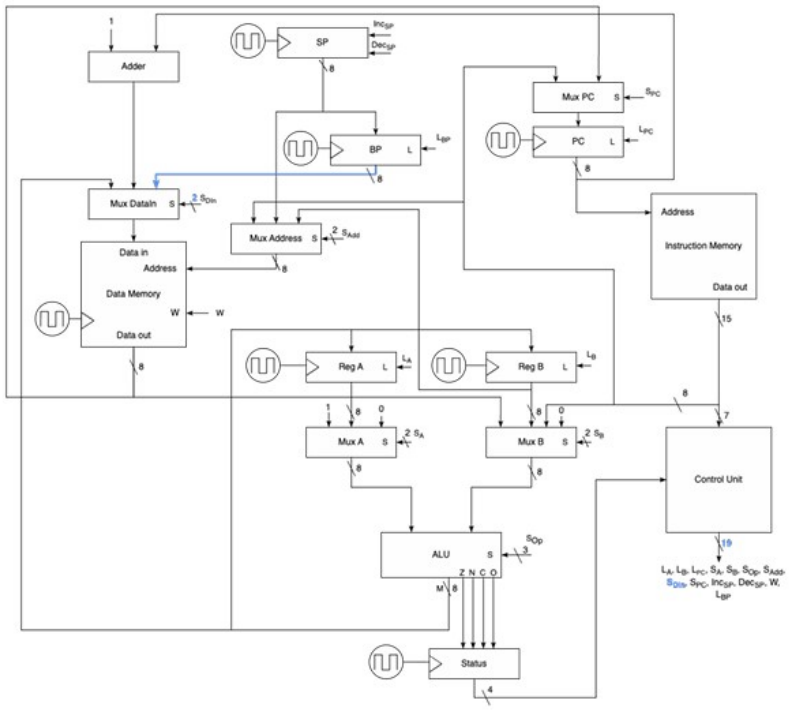
a) Para esta modificación, agregamos el registro `BP` a nuestra arquitectura con una nueva señal de carga L_{BP} y con valor de entrada igual a la salida de `SP`. De momento, no conectamos la salida de `BP` con ninguna compuerta (se asume que va a tierra).



La combinación de instrucciones para ejecutar la instrucción es:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}
MOV BP,SP	0	0	0	0	0	0	-	-	-	-	-	-	1

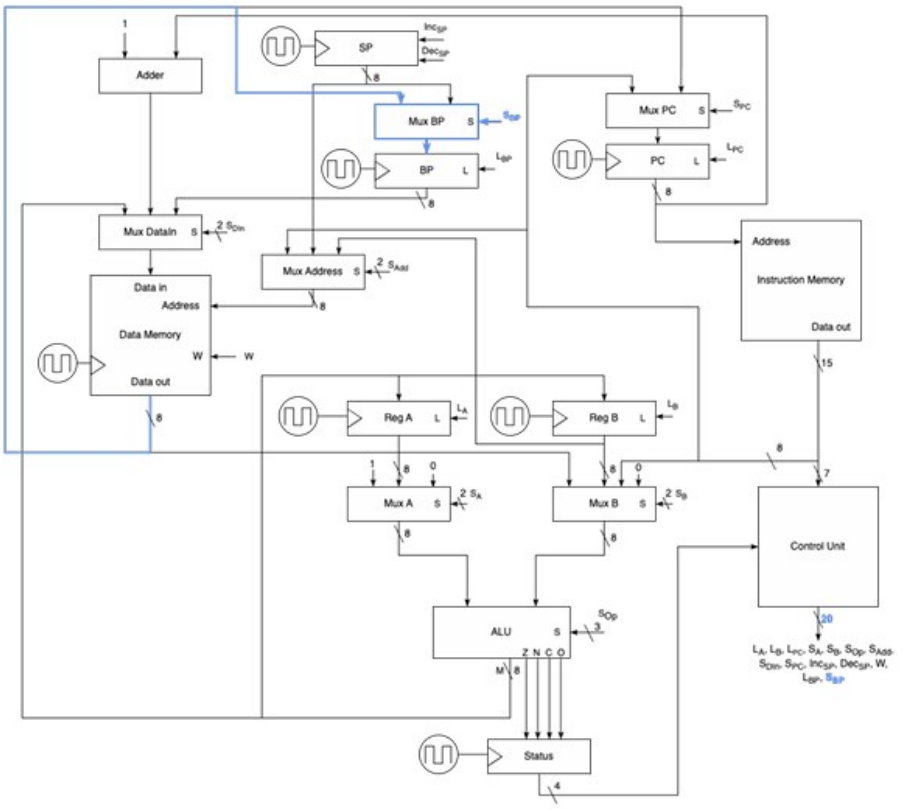
b) Para esta modificación, conectamos la salida del registro `BP` con una de las entradas del componente Mux DataIn. Para ello, es necesario incrementar la cantidad de bits de la señal S_{DIn}) a dos.



La combinación de instrucciones para ejecutar la instrucción es:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}
PUSH BP	0	0	0	1	0	1	-	-	-	SP	BP	-	0

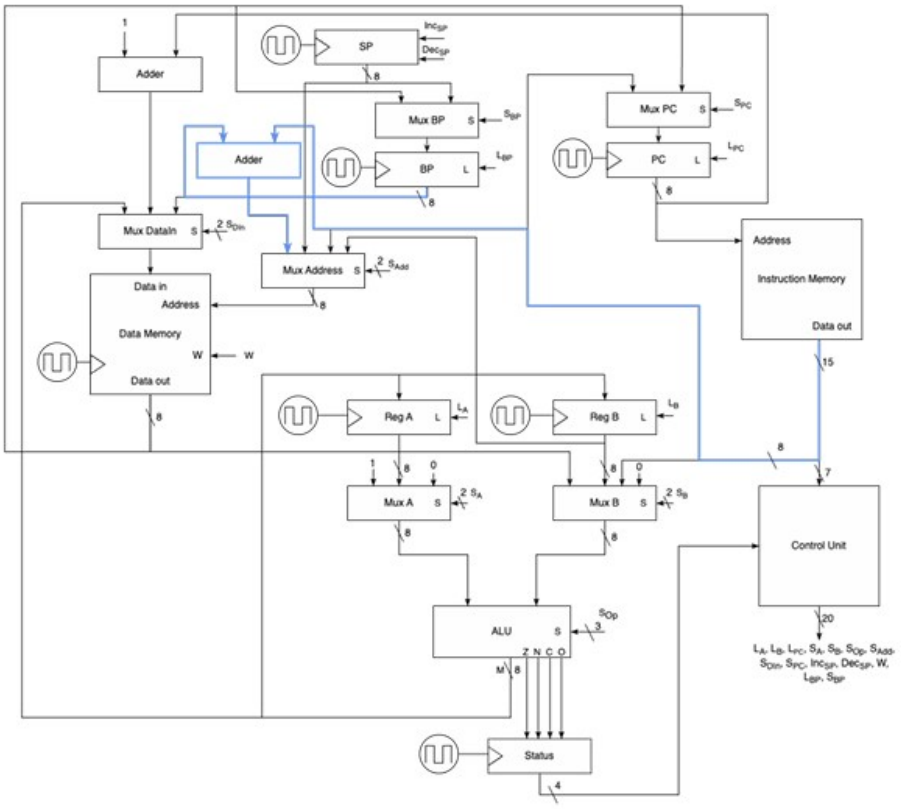
c) Para esta modificación, conectamos la salida de la memoria de datos con la entrada de BP. Dado que ya existe una entrada para SP, se agrega un nuevo componente Mux BP con señal de selección S_{BP} para escoger el valor de carga entre SP y DOUT.



La combinación de instrucciones para ejecutar la instrucción es:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
POP BP	0	0	0	0	1	0	-	-	-	-	-	-	0	-
	0	0	0	0	0	0	-	-	-	SP	-	-	1	DOUT

d) Para esta modificación, agregamos un `Adder` (sumador) que recibirá como entradas el valor del registro `BP` y el literal `LIT` de la memoria de instrucciones, para luego conectar su salida al componente `Mux Address`.



La combinación de instrucciones para ejecutar la instrucción es:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
`MOV A,(BP + Lit)`	1	0	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-
`MOV B,(BP + Lit)`	0	1	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-

Instrucción	Operandos	*Opcode*	Condición	L_{PC}	L_A	L_B	S_A0,1	S_B0,1	S_{Op}0,1,2	S_{Add}0,1	S_{Din}	S_{PC}	W	Inc_{SP}	Dec_{SP}
MOV	A,B	0000000		0	1	0	ZERO	B	ADD	-	-	-	0	0	0
	B,A	0000001		0	0	1	A	ZERO	ADD	-	-	-	0	0	0
	A,Lit	0000010		0	1	0	ZERO	LIT	ADD	-	-	-	0	0	0
	B,Lit	0000011		0	0	1	ZERO	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0000100		0	1	0	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	B,(Dir)	0000101		0	0	1	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	(Dir),A	0000110		0	0	0	A	ZERO	ADD	LIT	ALU	-	1	0	0
	(Dir),B	0000111		0	0	0	ZERO	B	ADD	LIT	ALU	-	1	0	0
	A,(B)	0001000		0	1	0	ZERO	DOUT	ADD	B	-	-	0	0	0
	B,(B)	0001001		0	0	1	ZERO	DOUT	ADD	B	-	-	0	0	0
	(B),A	0001010		0	1	0	A	ZERO	ADD	B	ALU	-	1	0	0
ADD	A,B	0001011		0	1	0	A	B	ADD	-	-	-	0	0	0
	B,A	0001100		0	0	1	A	B	ADD	-	-	-	0	0	0
	A,Lit	0001101		0	1	0	A	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0001110		0	1	0	A	DOUT	ADD	LIT	-	-	0	0	0
	A,(B)	0001111		0	1	0	A	DOUT	ADD	B	-	-	0	0	0
	(Dir)	0010000		0	0	0	A	B	ADD	LIT	ALU	-	1	0	0
SUB	A,B	0010001		0	1	0	A	B	SUB	-	-	-	0	0	0
	B,A	0010010		0	0	1	A	B	SUB	-	-	-	0	0	0
	A,Lit	0010011		0	1	0	A	LIT	SUB	-	-	-	0	0	0
	A,(Dir)	0010100		0	1	0	A	DOUT	SUB	LIT	-	-	0	0	0
	A,(B)	0010101		0	1	0	A	DOUT	SUB	B	-	-	0	0	0
	(Dir)	0010110		0	0	0	A	B	SUB	LIT	ALU	-	1	0	0
AND	A,B	0010111		0	1	0	A	B	AND	-	-	-	0	0	0
	B,A	0011000		0	0	1	A	B	AND	-	-	-	0	0	0
	A,Lit	0011001		0	1	0	A	LIT	AND	-	-	-	0	0	0
	A,(Dir)	0011010		0	1	0	A	DOUT	AND	LIT	-	-	0	0	0
	A,(B)	0011011		0	1	0	A	DOUT	AND	B	-	-	0	0	0
	(Dir)	0011100		0	0	0	A	B	AND	LIT	ALU	-	1	0	0
OR	A,B	0011101		0	1	0	A	B	OR	-	-	-	0	0	0
	B,A	0011110		0	0	1	A	B	OR	-	-	-	0	0	0
	A,Lit	0011111		0	1	0	A	LIT	OR	-	-	-	0	0	0
	A,(Dir)	0100000		0	1	0	A	DOUT	OR	LIT	-	-	0	0	0
	A,(B)	0100001		0	1	0	A	DOUT	OR	B	-	-	0	0	0
	(Dir)	0100010		0	0	0	A	B	OR	LIT	ALU	-	1	0	0
NOT	A,A	0100011		0	1	0	A	-	NOT	-	-	-	0	0	0
	B,A	0100100		0	0	1	A	-	NOT	-	-	-	0	0	0
	(Dir)	0100101		0	0	0	A	B	NOT	LIT	ALU	-	1	0	0
XOR	A,B	0100110		0	1	0	A	B	XOR	-	-	-	0	0	0
	B,A	0100111		0	0	1	A	B	XOR	-	-	-	0	0	0
	A,Lit	0101000		0	1	0	A	LIT	XOR	-	-	-	0	0	0
	A,(Dir)	0101001		0	1	0	A	DOUT	XOR	LIT	-	-	0	0	0
	A,(B)	0101010		0	1	0	A	DOUT	XOR	B	-	-	0	0	0
	(Dir)	0101011		0	0	0	A	B	XOR	LIT	ALU	-	1	0	0
SHL	A,A	0101100		0	1	0	A	-	SHL	-	-	-	0	0	0
	B,A	0101101		0	0	1	A	-	SHL	-	-	-	0	0	0
	(Dir)	0101110		0	0	0	A	B	SHL	LIT	ALU	-	1	0	0
SHR	A,A	0101111		0	1	0	A	-	SHR	-	-	-	0	0	0
	B,A	0110000		0	0	1	A	-	SHR	-	-	-	0	0	0
	(Dir)	0110001		0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
INC	B	0110010		0	0	1	ONE	B	ADD	-	-	-	0	0	0
CMP	A,B	0110101		0	0	0	A	B	SUB	-	-	-	0	0	0
	A,Lit	0110110		0	0	0	A	LIT	SUB	-	-	-	0	0	0
JMP	Dir	0111001		1	0	0	-	-	-	-	-	LIT	0	0	0
JEQ	Dir	0111010	Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JNE	Dir	0111011	Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JGT	Dir	0111100	N=0 y Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLT	Dir	0111101	N=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JGE	Dir	0111110	N=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLE	Dir	0111111	N=1 o Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JCR	Dir	1000000	C=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JOV	Dir	1000001	V=1	1	0	0	-	-	-	-	-	LIT	0	0	0
CALL	Dir	1000010		1	0	0	-	-	-	SP	PC	LIT	1	0	1
RET		1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000100		1	0	0	-	-	-	SP	-	DOUT	0	0	0
PUSH	A	1000101		0	0	0	A	ZERO	ADD	SP	ALU	-	1	0	1
	B	1000110		0	0	0	ZERO	B	ADD	SP	ALU	-	1	0	1
POP	A	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000111		0	1	0	ZERO	DOUT	ADD	SP	-	-	0	0	0
POP	B	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1001000		0	0	1	ZERO	DOUT	ADD	SP	-	-	0	0	0
NOP		1001001		0	0	0	-	-	-	-	-	-	0	0	0

Tabla 1: ISA del computador básico.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



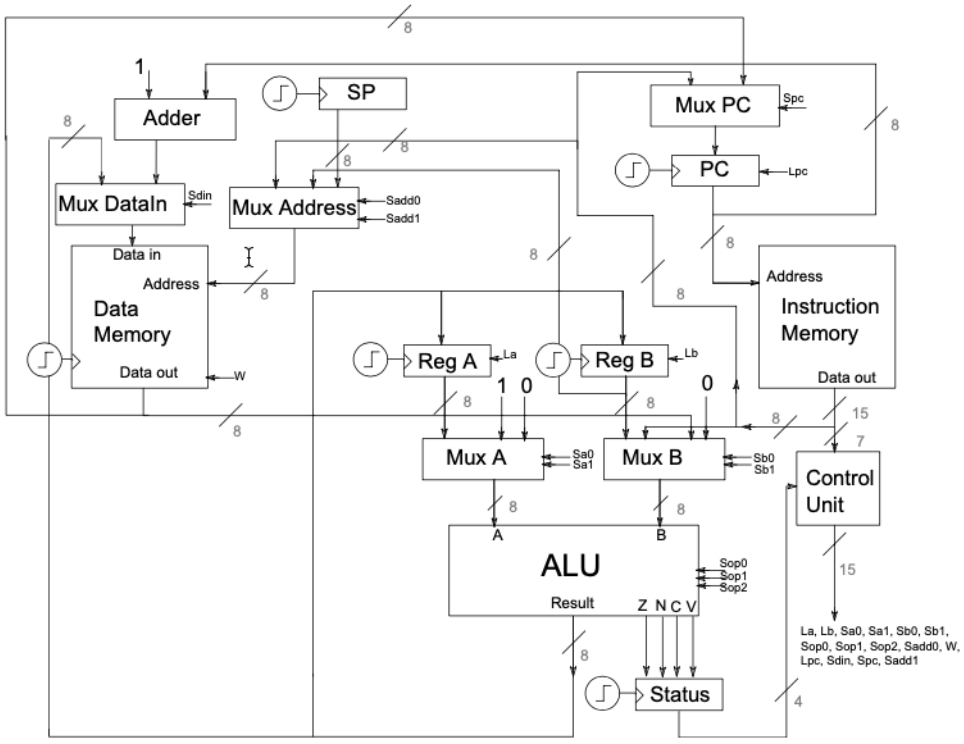
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2025)

Guía de ejercicios: Subrutinas

Ayudantes: Daniela Ríos (danielaarp@uc.cl), Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl)

Computador básico con subrutinas



Pregunta 1: Computador básico (P4-I1-2024-1)

Asuma que se encuentra trabajando como programador(a) en la oficina del famoso juego desarrollado completamente en Assembly: *Rollercoaster Tycoon*. Luego de unas horas, se da cuenta que la letra T del teclado de su computador ha dejado de funcionar, la que es de suma importancia al momento de escribir subrutinas por el uso de la instrucción `RET`. Adicionalmente, no cuenta con la opción de copiar y pegar texto, por lo que deberá desistir de su uso. Por el motivo anterior, deberá diseñar tres nuevas instrucciones que le permitan simular parte del comportamiento de la instrucción `RET`.

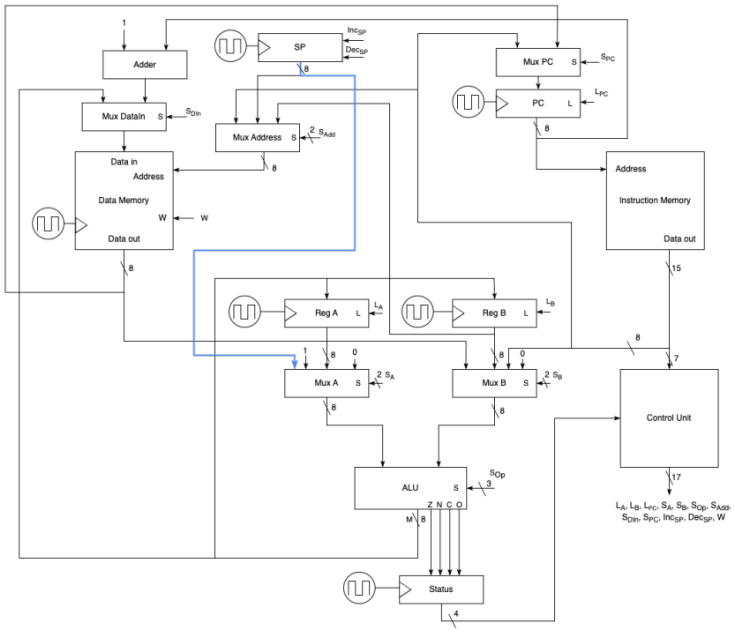
1. `MOV B, SP`: Almacena el valor del registro `SP` en el registro `B`.
2. `JMP B`: Salta a la instrucción con dirección igual al valor almacenado en el registro `B`.
3. `JMP (B)`: Salta a la instrucción con dirección igual al valor almacenado en la memoria de datos en la dirección `B`.

Deberá modificar la microarquitectura del computador básico para implementar las nuevas instrucciones y su funcionamiento. Para cada instrucción nueva, deberá incluir la combinación completa de señales que la ejecutan. Por cada señal de carga/escritura/incremento/decremento, deberá indicar si se activan (1) o no (0); en las señales de selección, deberá indicar el nombre de la entrada escogida ("-"si no afecta). Puede realizar todas las modificaciones en un solo diagrama.

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV B, SP`												
`JMP B`												
`JMP (B)`												

1. `MOV B, SP`. Guarda en `B` el valor de `SP`.

Solución: Esta instrucción no se puede habilitar sin modificaciones de hardware. Una opción consiste en conectar la salida del contador SP con la entrada restante del componente Mux A, como se muestra a continuación:



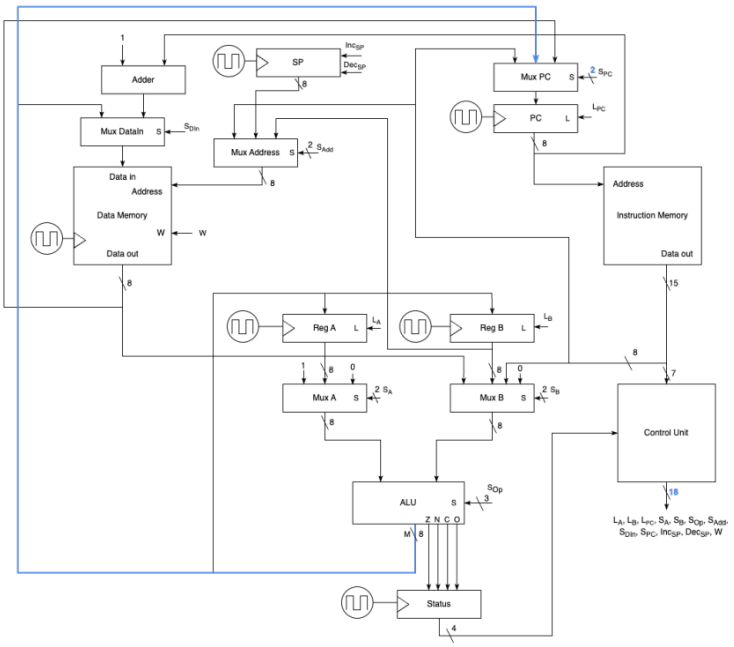
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV B, SP`	0	1	0	0	0	0	SP	ZERO	ADD	-	-	-

Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

2. `JMP B`. Salta a la instrucción con dirección igual al valor `B`.

Solución: Esta instrucción no se puede habilitar sin modificaciones de hardware. Una opción consiste en conectar la salida de la ALU con una de las entradas del componente Mux PC (lo que implica agregar un bit a su señal de selección), como se muestra a continuación:



Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
JMP B	0	0	1	0	0	0	ZERO	B	ADD	-	-	ALU

Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

3. `JMP (B)`. Salta a la instrucción con dirección igual al valor `Mem[B]`.

Solución: Esta instrucción se puede implementar sin modificaciones de hardware, como se muestra a continuación:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
JMP (B)	0	0	1	0	0	0	-	-	-	B	-	DOUT

Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

- Si no se modifica el diagrama: Se otorgan **1.5 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.
- Si se modifica el diagrama: Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

4. Asuma que se implementan las instrucciones anteriores y modifique el siguiente fragmento de código de forma que no utilice la instrucción `RET`, pero que se llegue al mismo resultado. Luego, comente sobre su resultado: ¿Cumple la misma función que el original? ¿Existe alguna consideración a tener en cuenta?

```
DATA:
    number_of_rides 0

CODE:
    JMP main

    // Incrementa la cantidad de veces a la que se sube a la montaña rusa
    increase_number_of_rides:
        MOV A, (number_of_rides)
        ADD A, 1
        MOV (number_of_rides), A
        RET

    main:
        PUSH A
        CALL increase_number_of_rides
        POP A
```

Solución: Para simular el comportamiento de `RET` con las instrucciones nuevas, existen dos alternativas:

a) Reemplazar `RET` por:

1) `POP B`
2) `JMP B`

En este caso, el código cumple la función original y no se requiere ningún arreglo adicional.

b) Reemplazar `RET` por:

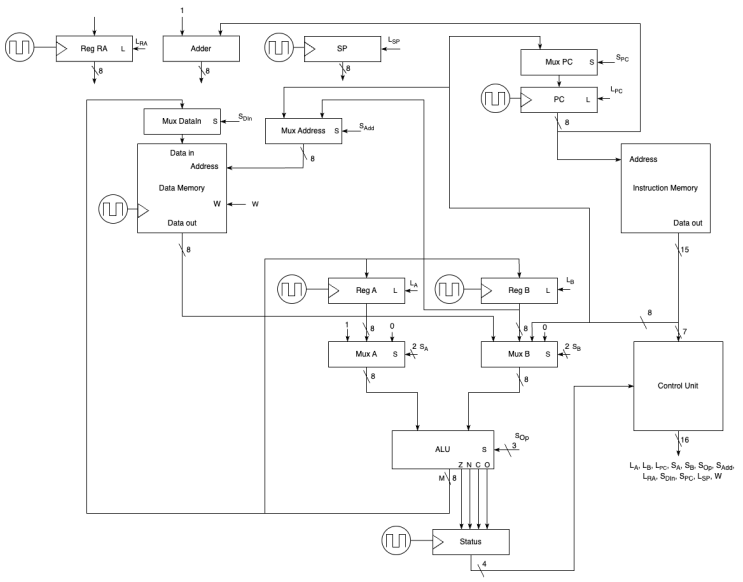
1) `MOV B, SP`
2) `INC B`
3) `JMP (B)`

En este caso, la ejecución no es exactamente igual a la original, dado que si bien se accede correctamente a la dirección de retorno, el valor de `SP` no es ajustado mediante un incremento. Esto genera que la ejecución de `POP A` almacene la dirección de retorno de la subrutina en `A` y no el valor respaldado al comienzo del programa.

Se otorgan **0.75 ptos.** por reemplazar correctamente el uso de `RET` con las instrucciones implementadas; y **0.75 ptos.** por señalar correctamente si su implementación cumple la misma función o no. También se aceptan como correctas alternativas donde se hagan modificaciones adicionales aparte del reemplazo de `RET` para almacenar correctamente el valor de `A`. En caso de existir como máximo un error de implementación en la implementación de código, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

Pregunta 2: Computador básico (P2-I2-2024-2)

En arquitecturas RISC, las direcciones de retorno de una subrutina se almacenan en un **registro de enlace** (`RA`), a diferencia del computador básico donde se guardan en el *stack*. Asimismo, estas arquitecturas no poseen `PUSH`/`POP`, sino que acceden al *stack* directamente con el *stack pointer* (`SP`). Deberá contestar las preguntas de esta sección a partir de este contexto y el siguiente diagrama:

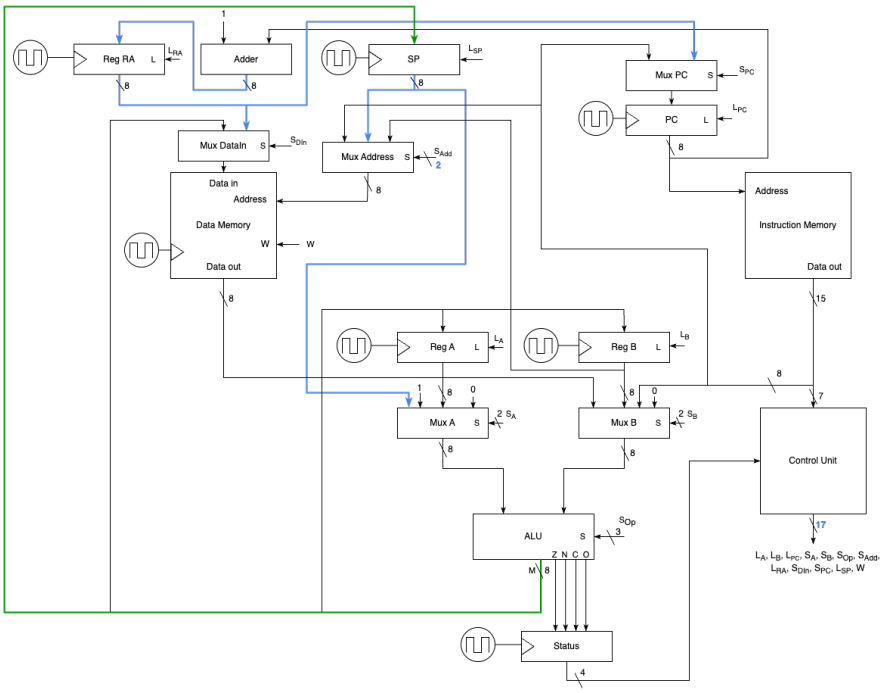


(a) **(5 ptos.)** A partir del diagrama de base adjunto, y asumiendo que se eliminan las instrucciones `CALL`, `RET`, `PUSH` y `POP`, realice las modificaciones de *hardware* necesarias e indique la combinación de señales completa para ejecutar las siguientes instrucciones en un ciclo:

- **(0.5 ptos.)** `MOV A,(SP)`. Guarda en `A` el valor `Mem[SP]`.
- **(0.5 ptos.)** `MOV (SP), A`. Guarda en `Mem[SP]` el valor `A`.
- **(1 pto.)** `ADD SP,Lit`. Guarda en `SP` el valor `SP + Lit`. `Lit` puede ser negativo.
- **(1 pto.)** `MOV (SP), RA`. Guarda en `Mem[SP]` el valor `RA`.
- **(1 pto.)** `JAL RA, label`. Guarda `PC+1` en `RA` y salta a la dirección asociada a `label`.
- **(1 pto.)** `JALR RA`. Salta a la dirección almacenada en `RA`.

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV A,(SP)`												
`MOV (SP), A`												
`ADD SP, Lit`												
`MOV (SP), RA`												
`JAL RA, label`												
`JALR RA`												

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- La conexión entre `SP` y `Mux Address` permite el uso de `SP` como dirección de memoria con la señal S_{Add}, la que ahora debe ser de 2 bits.
- La conexión entre `SP` y `Mux A` permite su uso como primer operando en la `ALU`.
- La conexión entre la salida de la `ALU` y `SP` permite cargar en este último los resultados de las operaciones con la señal L_{SP}.
- La conexión entre `RA` y `Mux DataIn` permite la escritura de `RA` en memoria.
- La conexión entre `RA` y `PC+1` permite escribir este valor en `RA` con la señal L_{RA}.
- La conexión entre `RA` y `Mux PC` permite realizar saltos hacia direcciones almacenadas en `RA`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA

Por cada instrucción, se otorga **la mitad** del puntaje por *hardware* y **la otra mitad** por la combinación de señales. En cada criterio, se descuenta la mitad del puntaje en caso de existir, como máximo, un error de implementación o valor en la combinación de señales. En otro caso, no se otorga puntaje.

(b) **(1 pto.)** Modifique el siguiente fragmento de código para que haga uso exclusivo de las instrucciones anteriores. Debe mantener su funcionamiento original y terminar sin errores.

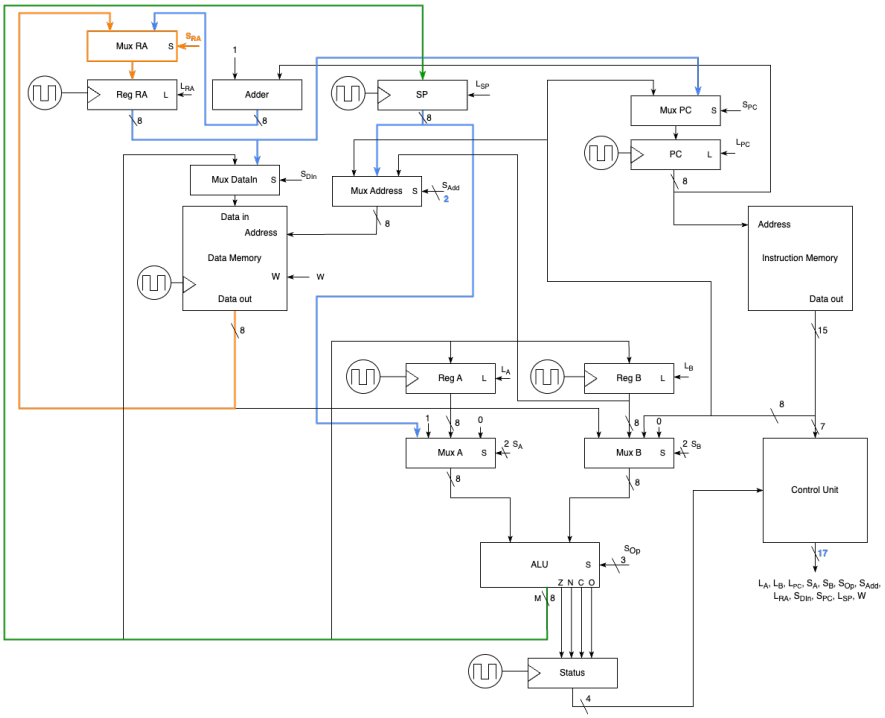
```
MOV A,1
PUSH A
CALL func
POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  CALL func
  end_func:
    RET
end:
```

Solución: Es importante señalar que, con las instrucciones entregadas, **no es posible** entregar un fragmento de código que mantenga el funcionamiento original **sin errores**. Esto se debe al hecho de que no existe una instrucción que permita recuperar una dirección de retorno respaldada en memoria (por ejemplo, `MOV RA, (SP)`. Por este motivo, se consideran correctas respuestas que solo caigan en ese caso. A continuación, una solución posible.

```
MOV A,1
MOV (SP),A        ; Reemplazo por PUSH A
ADD SP,-1         ; Reemplazo por PUSH A
JAL RA,func       ; Reemplazo por CALL func
ADD SP,1          ; Reemplazo por POP A
MOV A,(SP)        ; Reemplazo por POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  MOV (SP),RA     ; Respaldo de RA para evitar error en llamado recursivo
  ADD SP,-1       ; Respaldo de RA para evitar error en llamado recursivo
  JAL RA,func     ; Reemplazo por CALL func
  ADD SP,1        ; De todas formas, se restituye el valor de SP
  MOV RA,(SP)     ; Si existiera, podríamos recuperar el valor respaldado de RA
  end_func:
    JALR RA       ; Reemplazo por RET
end:
```

Se otorgan **0.25 ptos.** por reemplazar correctamente la instrucción `PUSH A`; **0.25 ptos.** por reemplazar correctamente la instrucción `CALL label`; **0.25 ptos.** por reemplazar correctamente la instrucción `POP A`; y **0.25 ptos.** por reemplazar correctamente la instrucción `RET`. Adicionalmente, si la respuesta alude a la falta de una instrucción para recuperar la dirección de retorno; o bien se usa de forma hipotética una instrucción que permita resolver el problema, **se otorgarán 0.5 ptos. de bonus al total**.

A modo de completitud, se incluye tanto el diagrama como la tabla de señales de la arquitectura que habilita la ejecución de la instrucción `MOV RA,(SP)`.



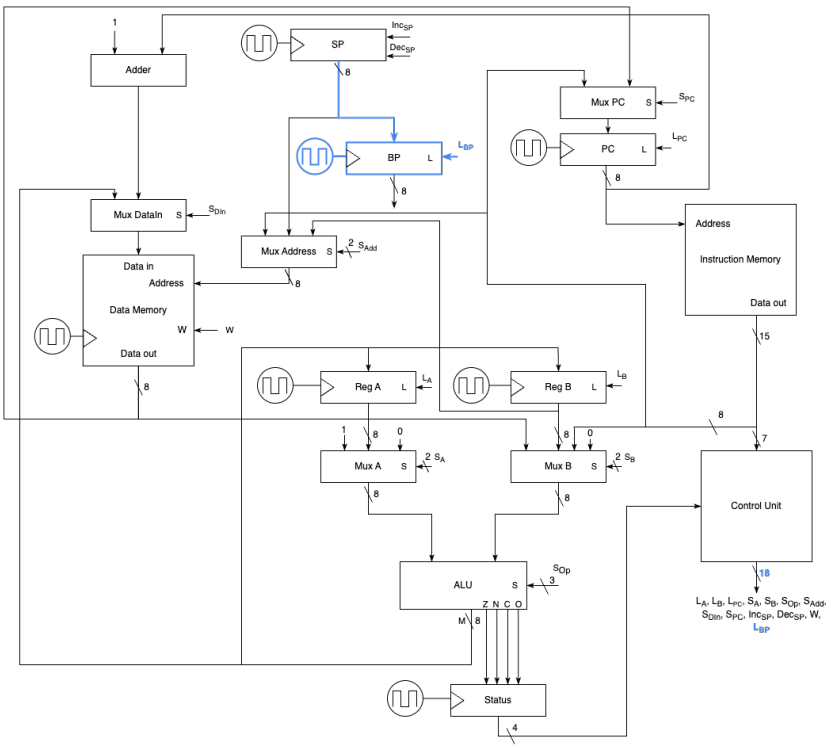
Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	S_{RA}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-	-
`MOV RA, (SP)`	0	0	0	0	1	0	-	-	-	SP	-	-	DOUT
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT	PC+1
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA	-

Pregunta 3: Computador básico (P4-I1-2023-2)

En varias arquitecturas se hace uso del registro *base pointer* (`BP`). Este funciona como punto de referencia para obtener los parámetros y dirección de retorno de cada llamada de una subrutina, lo que facilita el manejo de llamados anidados. Deberá modificar la microarquitectura del computador básico para implementar el registro `BP` de 8 bits y su funcionamiento. Se le listarán las instrucciones que deben ser implementadas. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida ("-" si no afecta). Puede realizar todas las modificaciones en un solo diagrama.

(a) (1 pto.) `MOV BP,SP`. Almacena en el registro `BP` el valor del registro `SP`.

Solución: Para esta modificación, agregamos el registro `BP` a nuestra arquitectura con una nueva señal de carga L_{BP} y con valor de entrada igual a la salida de `SP`. De momento, no conectamos la salida de `BP` con ninguna compuerta (se asume que va a tierra).



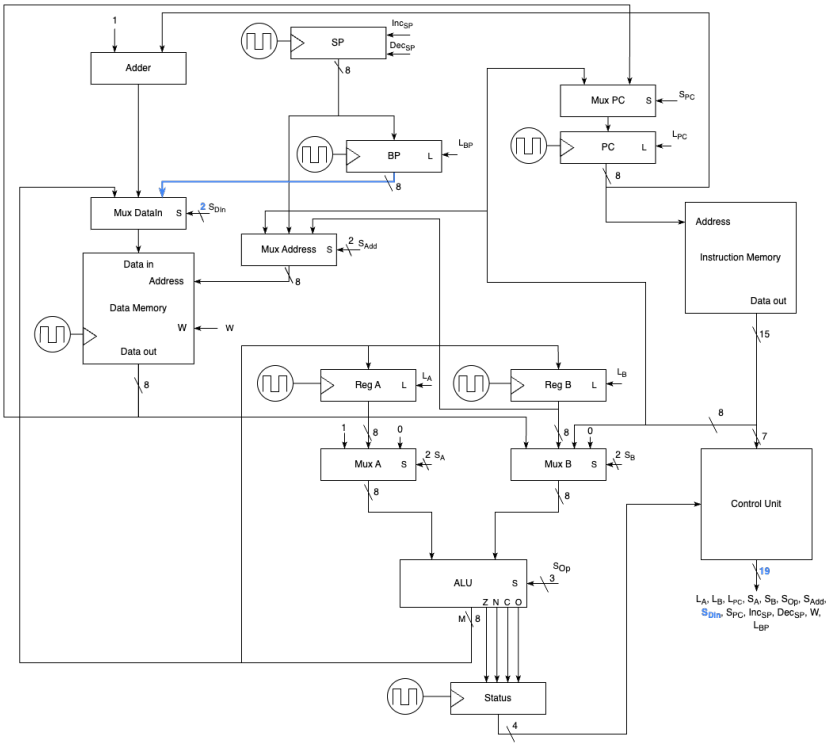
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}
`MOV BP,SP`	0	0	0	0	0	0	-	-	-	-	-	-	1

Se otorgan **0.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para la instrucción.

(b) (1 pto.) `PUSH BP`. Almacena en `Mem[SP]` el valor de `BP` y decrementa `SP` en una unidad.

Solución: Para esta modificación, conectamos la salida del registro `BP` con una de las entradas del componente `Mux DataIn`. Para ello, es necesario incrementar la cantidad de bits de la señal S_{DIn} a dos.



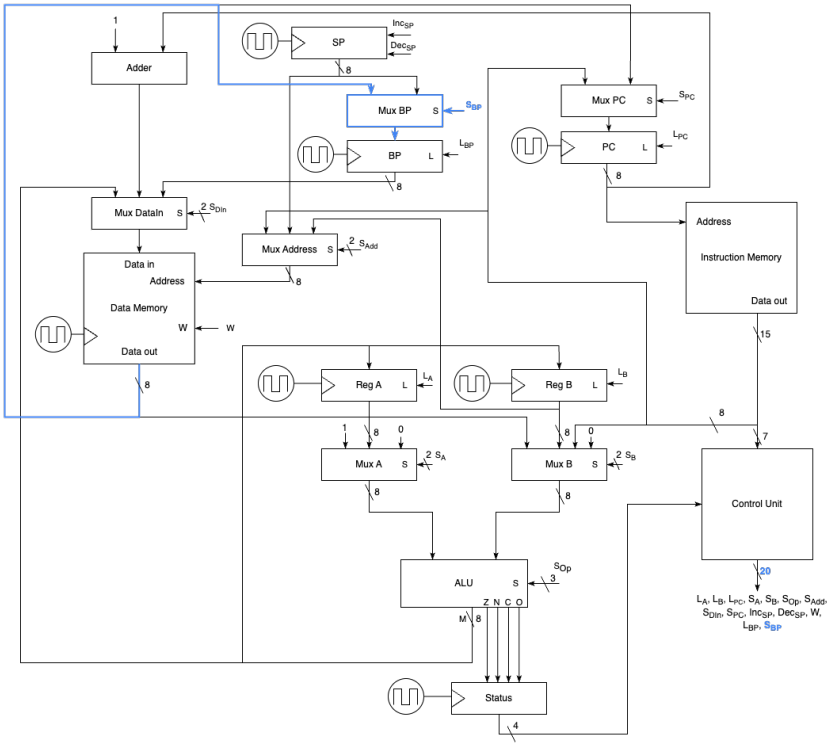
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}
PUSH BP	0	0	0	1	0	1	-	-	-	SP	BP	-	0

Se otorgan **0.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para la instrucción.

(c) (1 pto.) `POP BP`. Incrementa `SP` en una unidad y almacena en `BP` el valor `Mem[SP]`.

Solución: Para esta modificación, conectamos la salida de la memoria de datos con la entrada de `BP`. Dado que ya existe una entrada para `SP`, se agrega un nuevo componente `Mux BP` con señal de selección S_{BP} para escoger el valor de carga entre `SP` y `DOUT`.



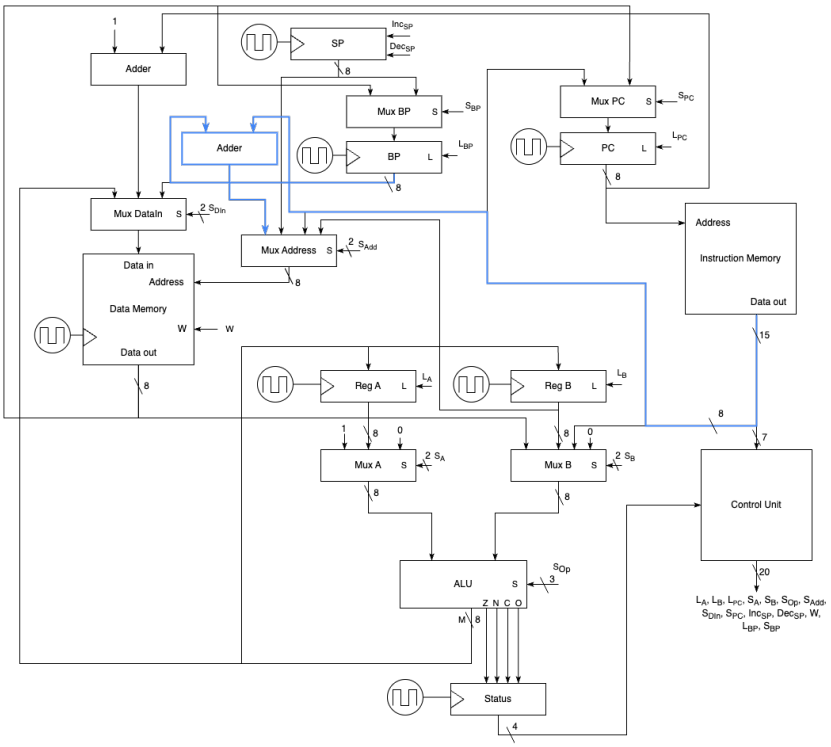
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
POP BP	0	0	0	0	1	0	-	-	-	-	-	-	0	-
	0	0	0	0	0	0	-	-	-	SP	-	-	1	DOUT

Se otorgan **0.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para la instrucción.

(d) (2 ptos.) `MOV Reg,(BP + Lit)`. Almacena en `Reg` (`A` o `B`) el valor `Mem[BP + Lit]`.

Solución: Para esta modificación, agregamos un `Adder` que recibirá como entradas el valor del registro `BP` y el literal `LIT` de la memoria de instrucciones, para luego conectar su salida al componente `Mux Address`.

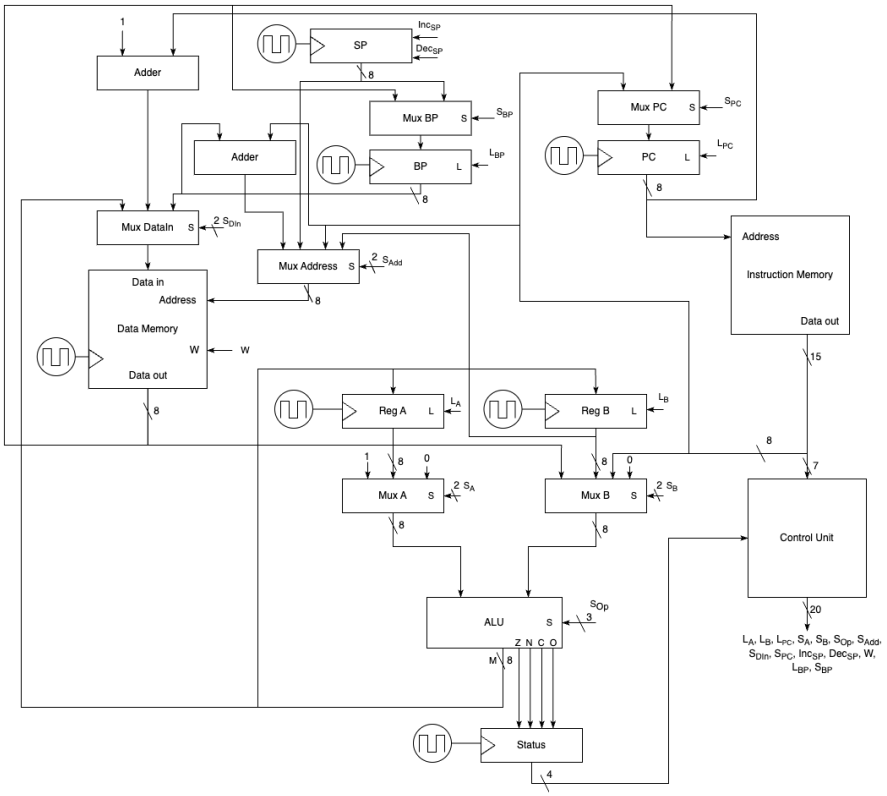


Para ejecutar las instrucciones, se utilizan la siguientes combinaciones de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
`MOV A,(BP + Lit)`	1	0	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-
`MOV B,(BP + Lit)`	0	1	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-

Se otorgan **1.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para una o ambas instrucciones. Se otorgan **0.75 ptos.** por la modificación si esta presenta como máximo un error de implementación.

En la siguiente página se muestra el diagrama completo de la arquitectura esperada y su tabla de instrucciones, asumiendo que se implementan todas las modificaciones juntas.



Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
MOV BP,SP	0	0	0	0	0	0	-	-	-	-	-	-	1	SP
PUSH BP	0	0	0	1	0	1	-	-	-	SP	BP	-	0	-
POP BP	0	0	0	0	1	0	-	-	-	-	-	-	0	-
	0	0	0	0	0	0	-	-	-	SP	-	-	1	DOUT
MOV A,(BP + Lit)	1	0	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-
MOV B,(BP + Lit)	0	1	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
MOV BP,SP														
PUSH BP														
POP BP														
MOV A,(BP+Lit)														
MOV B,(BP+Lit)														

(e) (3 ptos.) Asuma que se agregan las instrucciones anteriores a la ISA del computador básico y que se ejecuta el programa adjunto. Explique, a grandes rasgos, lo que realiza la subrutina `func` y señale el valor final de la variable `result`. Puede asumir que `BP` parte en 255.

```
DATA:
  n      3
  result 0
CODE:
  MOV A,(n)          // Dirección Mem. Instr.: 0x00
  PUSH A             // Dirección Mem. Instr.: 0x01
  CALL func          // Dirección Mem. Instr.: 0x02
  POP A              // Dirección Mem. Instr.: 0x03-0x04
  JMP end            // Dirección Mem. Instr.: 0x05
  func:
    PUSH BP          // Dirección Mem. Instr.: 0x06
    MOV BP,SP        // Dirección Mem. Instr.: 0x07
    MOV A,(BP + 3)   // Dirección Mem. Instr.: 0x08
    CMP A,0          // Dirección Mem. Instr.: 0x09
    JEQ base_end_0   // Dirección Mem. Instr.: 0x0A
    CMP A,1          // Dirección Mem. Instr.: 0x0B
    JEQ base_end_1   // Dirección Mem. Instr.: 0x0C
    SUB A,1          // Dirección Mem. Instr.: 0x0D
    PUSH A           // Dirección Mem. Instr.: 0x0E
    CALL func        // Dirección Mem. Instr.: 0x0F
    POP A            // Dirección Mem. Instr.: 0x10-0x11
    MOV A,(BP + 3)   // Dirección Mem. Instr.: 0x12
    SUB A,2          // Dirección Mem. Instr.: 0x13
    PUSH A           // Dirección Mem. Instr.: 0x14
    CALL func        // Dirección Mem. Instr.: 0x15
    POP A            // Dirección Mem. Instr.: 0x16-0x17
    JMP end_func     // Dirección Mem. Instr.: 0x18
    base_end_1:
      INC (result)   // Dirección Mem. Instr.: 0x19
    base_end_0:
      NOP            // Dirección Mem. Instr.: 0x1A
    end_func:
      POP BP         // Dirección Mem. Instr.: 0x1B-0x1C
      RET            // Dirección Mem. Instr.: 0x1D-0x1E
  end:
```

Solución: La subrutina `func` corresponde a la función de recurrencia de Fibonacci, *i.e.*:

$$F_N = \begin{cases} F_{N-1} + F_{N-2} & N > 1 \\ N & N \leq 1 \end{cases}$$

Esto se deduce de la siguiente forma:

- Todo `PUSH A` se realiza antes de un llamado a `func`, siendo este el parámetro `n`.
- Mediante `MOV A,(BP + 3)` se recupera el argumento respaldado. Por cada llamado se guardan en el *stack* los valores `A` (parámetro), `PC+1` (retorno) y `BP`. Como `SP` apunta siempre una dirección sobre el tope del *stack* y `BP` tiene el mismo valor por `MOV BP,SP`, entonces `BP + 3` apunta a la dirección en la que se guardó el parámetro.
- Se realizan dos llamados recursivos al interior de `func`, siendo uno de ellos realizado con el parámetro `n - 1` (proveniente de `SUB A,1`) y el otro con el parámetro `n - 2` (proveniente de `SUB A,2`).

- `base_end_0` y `base_end_1` corresponden a los casos base de la recursión y se llega a ellos si, y solo si `A` es 0 o 1 respectivamente. Interesa sobre todo `base_end_1`, dado que en dicho caso se incrementa el valor de la variable `result`.
- Al terminar un llamado de la subrutina en `end_func`, se reestablece el *stack* al ejecutar `POP BP`, `RET` y `POP A` luego de retornar. Esto asegura que siempre se utilice el valor correcto de `BP` para acceder al parámetro de cada llamado recursivo.

A partir de lo anterior, se deduce entonces que el valor de `result` es igual a:

$$\begin{aligned} F_3 &= F_2 + F_1 \\ &= F_1 + F_0 + F_1 \\ &= 1 + 0 + 1 \\ &= 2 \end{aligned}$$

Se otorgan **2 ptos.** por señalar correctamente el resultado de la ejecución correctamente y **1 pto.** por describir `func` a partir de Fibonacci. Si no se cumple ninguno de los dos criterios anteriores, se otorga **1 pto.** si se muestra un desarrollo parcial pero correcto de la ejecución de `func`.

Pregunta 4: Assembly con subrutinas (P3-I1-2024-1)

(a) Indique los valores de los registros `A` y `B` al finalizar la ejecución del código **(a)**.

(b) Indique los valores de los registros `A` y `B` al finalizar la ejecución del código **(b)**.

(c) El código **(c)** *debería* computar la multiplicación entre las variables `X` e `Y`, pero cuenta con errores que generan un resultado erróneo. Indique el valor de los registros `A`, `B` y la variable `res` al finalizar la ejecución del código. Luego, señale dónde se encuentra el o los errores y cómo se resuelven.

(a)

```
DATA:
    res     0

CODE:
    JMP _start

    func_1:
        SHR A, A
        JCR func_2
        CMP A, 0
        JNE func_1
        RET

    func_2:
        NOT B, A
        MOV (B), 14
        INC (res)
        JMP func_1

    _start:
        MOV A, 6
        CALL func_1
        MOV B, (res)
```

(b)

```
DATA:
    var   10

CODE:
    JMP _start

    func_1:
        MOV B, (var)
        NOT A, A
        ADD A, 1
        ADD B, A
        RET

    _start:
        MOV A, 3
        CALL func_1

    end:
        MOV A, (255)
```

(c)

```
DATA:
    res     0
    X       3
    Y       4
    iter    0

CODE:
    MOV A, (Y)
    PUSH A
    CALL mult
    MOV A, (res)
    JMP end

    mult:
        POP B
    sumar:
        MOV A, (res)
        ADD A, (X)
        MOV (res), A
        INC (iter)
        MOV A, (iter)
        CMP A, B
        JNE sumar
        RET

    end:
```

Consideraciones:

1. La primera instrucción del segmento de `CODE` se guardará en la dirección 0 de la memoria de instrucciones.
2. En las instrucciones de *shifting*, el bit descartado se almacena en el *condition code* de *carry*.

Solución:

(a) Los valores finales de los registros `A` y `B` son 0 y 255 respectivamente. La explicación es la siguiente:

- Al comenzar la ejecución de código, se inicializa el registro `A` con el literal 6 y luego se llama a la subrutina `func_1`.
- Lo que realiza esta subrutina es lo siguiente: (1) Aplica un *shift right* sobre el registro `A`, equivalente a actualizar su valor con el resultado de su división entera por 2; (2) se revisa si se activó la señal de *carry*, lo que ocurre si el bit descartado en el *shift right* fue igual a 1 (*i.e.* se dividió un número impar). Si es el caso, se salta al *label* `func_2`; en otro caso; (3) se revisa si el valor de `A` es igual a 0, en cuyo caso se retorna; si no, se vuelve a ejecutar desde un inicio la subrutina.
- En caso de que se gatille `func_2`, lo que realiza este fragmento es lo siguiente: (1) Almacena en el registro `B` la negación de `A`; (2) utiliza este valor como dirección de la memoria de datos y almacena en dicha ubicación el literal 14; (3) incrementa en una unidad la variable `res`; (4) vuelve a retomar la ejecución de la subrutina `func_1`.
- Para descifrar los valores finales, es necesario ejecutar todos los ciclos de la subrutina:
  1. `A` = `SHL(A)` = 3, $C = 0$. No se realiza el salto con `JCR` y se repite la ejecución de la subrutina.
  2. `A` = `SHL(A)` = 1, $C = 1$. Se se realiza el salto con `JCR` y se tiene que `B` = `NOT(00000001)` = 11111110. Por lo tanto, en dicha dirección se almacena el literal 14: `Mem[11111110]` = `Mem[254]` = 14. La variable `res` incrementa su valor a 1 y se procede a repetir la ejecución de la subrutina.
  3. `A` = `SHL(A)` = 0, $C = 1$. Se se realiza el salto con `JCR` y se tiene que `B` = `NOT(00000000)` = 11111111. Por lo tanto, en dicha dirección se almacena el literal 14: `Mem[11111111]` = `Mem[255]` = 14. La variable `res` incrementa su valor a 2 y se procede a repetir la ejecución de la subrutina.
  4. `A` = `SHL(A)` = 0, $C = 0$. No se realiza el salto con `JCR` y finaliza la ejecución, siendo entonces este el valor final del registro `A`. Al retornar, se carga en el contador `PC` el contenido del tope del `stack`, en este caso, `PC` = `Mem[255]` = 14.
- Considerando que `JMP _start` se almacena en la dirección 0 y que `RET` ocupa dos direcciones en la memoria de instrucciones, se tiene que la dirección de retorno original de la subrutina, correspondiente a la instrucción `MOV B, (res)`, es igual a 13. No obstante lo anterior, este valor se sobreescribe con la subrutina con el literal 14, lo que implica que al retornar el programa **no ejecute** esta instrucción y, por ende, `B` mantenga el valor 255 computado dentro de la subrutina.

Importante: Si bien la instrucción `MOV (B),Lit` no es parte de la ISA del computador básico, la microarquitectura permite su ejecución, por lo que durante la interrogación se informa que se puede asumir que existe y se puede utilizar en este fragmento de código.

Se otorgan **0.75 ptos.** por cada valor final señalado correctamente. En caso de que un valor esté erróneo pero exista desarrollo de la ejecución de código para su obtención, se otorgan **0.375 ptos.** como puntaje parcial.

(b) Los valores finales de los registros `A` y `B` son 9 y 7 respectivamente. La explicación es la siguiente:

- Al comenzar la ejecución del programa, se inicializa el registro `A` con el literal 3 y luego se llama a la subrutina `func_1`.
- Dentro de la subrutina, se almacena el valor de la variable `var` en `B`, que es igual a 10.
- Posteriormente, se actualiza el valor del registro `A` y se reemplaza por $\overline{\texttt{A}} + 1$, lo que es equivalente a computar su inverso aditivo, es decir, -3.
- Se le suma al registro `B` lo computado en `A`, por lo que $\texttt{B} = 10 + -3 = 7$. Este es el valor final del registro.
- Finalmente, al retornar, se almacena en `A` el valor de la dirección de memoria 255, correspondiente al primer valor almacenado en la memoria de *stack*: la dirección de retorno del llamado a `func_1`. Considerando que `JMP _start` se almacena en la dirección 0 y que `RET` ocupa dos direcciones en la memoria de instrucciones, se tiene entonces que la dirección de retorno del llamado, correspondiente a la ubicación de la instrucción `MOV A, (255)`, es igual a 9, siendo este el valor final de `A`. Es importante recordar que el incremento de `SP` en el retorno **no elimina el valor almacenado en el tope del *stack***.

Se otorgan **0.75 ptos.** por cada valor final señalado correctamente. En caso de que un valor esté erróneo pero exista desarrollo de la ejecución de código para su obtención, se otorgan **0.375 ptos.** como puntaje parcial.

(c) Los valores finales de los registros `A` y `B` son 3 y 3 respectivamente, mientras que el valor de la variable `res` es 9. La explicación es la siguiente:

- Al comenzar la ejecución del programa, se almacena en `A` el valor de la variable `Y` (correspondiente a 4) y se respalda en la memoria de *stack* a través de un `PUSH`.
- Se hace el llamado a la subrutina `mult`, almacenando en el tope del *stack* la dirección de retorno que apunta a la instrucción `MOV A,(res)`. No obstante, lo primero que se ejecuta en la subrutina es un `POP B`, lo que hace que se almacene la dirección de retorno en el registro `B` y **cambiando la posición del contador `SP`**, lo que tendrá consecuencias a ser descritas más adelante.
- Se empieza a computar la multiplicación a partir de la suma iterativa de la variable `X` sobre `res`, ejecutándose hasta que el total de iteraciones sea igual al valor de `B`. En este punto, se esperaría que la suma se repita `Y` veces para computar correctamente la multiplicación, pero el registro `B` ahora posee la dirección de memoria de la instrucción `MOV A, (res)`. Considerando que `MOV A, (Y)` se almacena en la dirección 0, se tiene que la dirección de retorno almacenada en `B` es 3. Por lo tanto, lo que se computará será la multiplicación entre `X` igual a 3 y 3, cuyo resultado es igual a 9 y no 12 como se esperaría. Este valor se almacena en la variable `res` y se termina la ejecución de la subrutina.
- Al retornar de la subrutina, se guarda en el contador `PC` el valor del tope del *stack*, correspondiente al valor del registro `A` respaldado en un comienzo. En este caso el

valor fue igual a 4, que corresponde a la dirección de la instrucción `JMP end`, por lo que no se ejecuta `MOV A, (res)` y el programa termina sin actualizar el valor de este registro. Por lo tanto, el valor del registro `A` es igual al asignado con la instrucción `MOV A,(iter)` en la última iteración, que fue igual a 3 (correspondiente a la cantidad de iteraciones ejecutadas). El valor de `B` no se vuelve a actualizar después de la ejecución de la instrucción `POP B`, por lo que su valor es igual a 3.

Para que la multiplicación se compute correctamente, basta con asegurar que: (1) a `B` se le asigne correctamente el valor de la variable `Y`; y (2) que en el tope del *stack* se encuentre almacenada la dirección de retorno de la subrutina. Esto se puede realizar de muchas maneras, pero a continuación se deja un ejemplo:

```
DATA:
    res     0
    X       3
    Y       4
    iter    0

CODE:
    MOV B, (Y)
    CALL mult
    MOV A, (res)
    JMP end

    mult:
        sumar:
        MOV A, (res)
        ADD A, (X)
        MOV (res), A
        INC (iter)
        MOV A, (iter)
        CMP A, B
        JNE sumar
        RET

    end:
```

Se otorgan **0.5 ptos.** por cada valor final señalado correctamente; **0.75 ptos.** por señalar correctamente el error en el programa; y **0.75 ptos.** por indicar una forma válida para su resolución. En caso de que un valor esté erróneo pero exista desarrollo de la ejecución de código para su obtención, se otorgan **0.25 ptos.** como puntaje parcial.

Pregunta 5: Assembly con subrutinas (P1-I1-2023-2)

Indique el valor final de los registros `A` y `B` luego de ejecutar el siguiente fragmento de código escrito en el Assembly del computador básico.

```
CODE:
    MOV A,7
    MOV B,1
    PUSH B
    PUSH A
    RET
    MOV A,0
    MOV B,0
    JMP end
    end:
```

Solución: Cuando se ejecuta el fragmento de código anterior, `RET` gatilla la carga del valor numérico del tope del *stack* en el registro `PC`. En este caso, el tope del `stack` posee el valor 7 y por ende el programa saltará a dicha dirección en la memoria de instrucciones. A continuación, se muestra el mismo fragmento mostrando las direcciones en las que se almacena cada instrucción.

```
CODE:
    MOV A,7     ;Dirección 0x00
    MOV B,1     ;Dirección 0x01
    PUSH B      ;Dirección 0x02
    PUSH A      ;Dirección 0x03
    RET         ;Dirección 0x04-0x05
    MOV A,0     ;Dirección 0x06
    MOV B,0     ;Dirección 0x07       <- RET gatilla un salto aquí.
    JMP end     ;Dirección 0x08
    end:
```

Por lo tanto, el programa terminará ejecutando las instrucciones `MOV B,0` y `JMP end`, saltándose `MOV A,0`. De esta forma, el valor final del registro `A` es igual a 7 y el del registro `B` es igual a 0. Se otorga **1 pto.** por cada registro donde se indique su valor final correcto

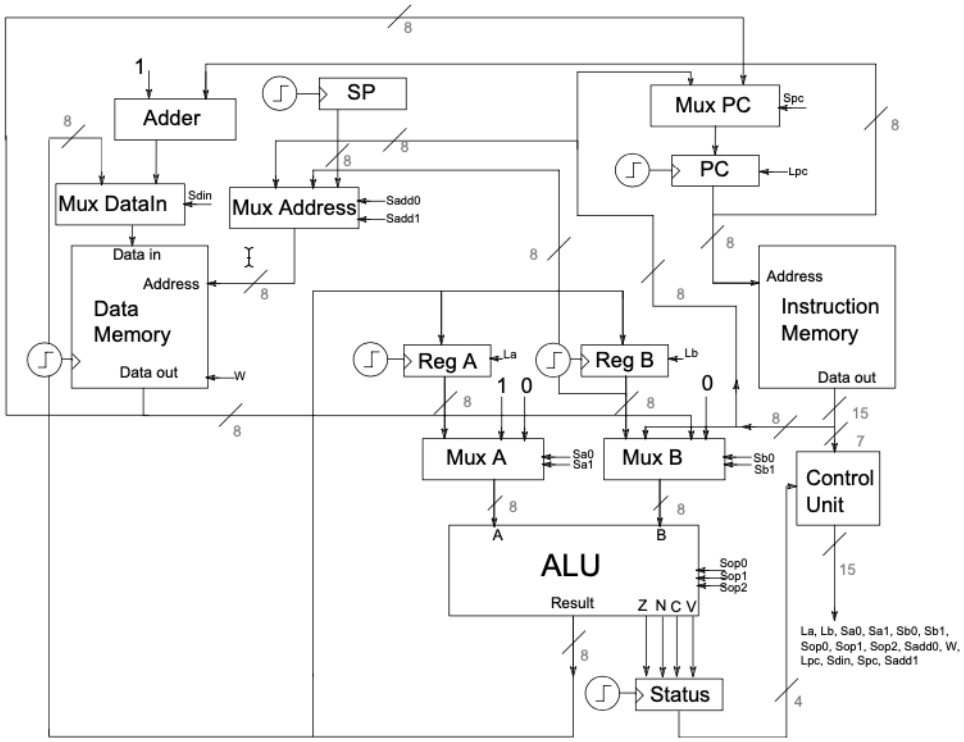
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: Subrutinas

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Fernanda Escobar (fer_jez@uc.cl)

Computador básico con subrutinas



Pregunta 1: Computador básico (P4-I1-2024-1)

Asuma que se encuentra trabajando como programador(a) en la oficina del famoso juego desarrollado completamente en Assembly: *Rollercoaster Tycoon*. Luego de unas horas, se da cuenta que la letra T del teclado de su computador ha dejado de funcionar, la que es de suma importancia al momento de escribir subrutinas por el uso de la instrucción `RET`. Adicionalmente, no cuenta con la opción de copiar y pegar texto, por lo que deberá desistir de su uso. Por el motivo anterior, deberá diseñar tres nuevas instrucciones que le permitan simular parte del comportamiento de la instrucción `RET`.

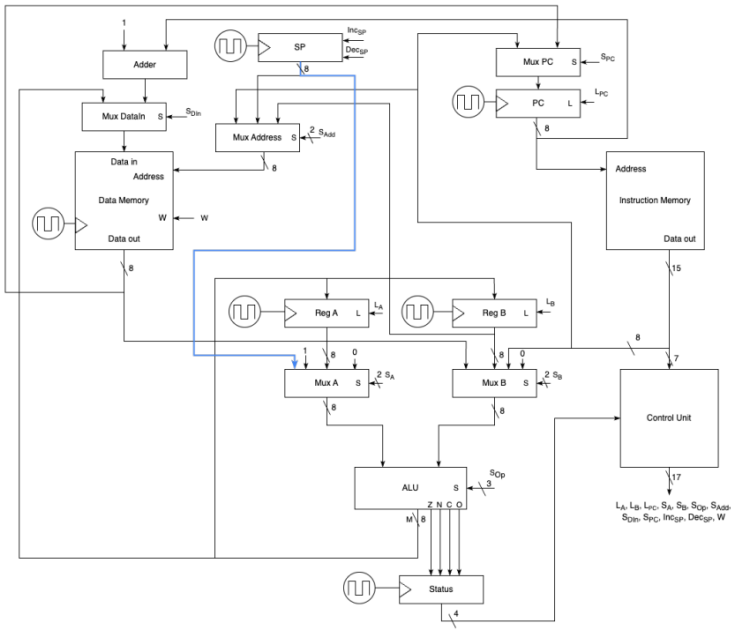
1. `MOV B, SP`: Almacena el valor del registro `SP` en el registro `B`.
2. `JMP B`: Salta a la instrucción con dirección igual al valor almacenado en el registro `B`.
3. `JMP (B)`: Salta a la instrucción con dirección igual al valor almacenado en la memoria de datos en la dirección `B`.

Deberá modificar la microarquitectura del computador básico para implementar las nuevas instrucciones y su funcionamiento. Para cada instrucción nueva, deberá incluir la combinación completa de señales que la ejecutan. Por cada señal de carga/escritura/incremento/decremento, deberá indicar si se activan (1) o no (0); en las señales de selección, deberá indicar el nombre de la entrada escogida ("-"si no afecta). Puede realizar todas las modificaciones en un solo diagrama.

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}		
MOV B, SP														
JMP B														
JMP (B)														

1. `MOV B, SP`. Guarda en `B` el valor de `SP`.

Solución: Esta instrucción no se puede habilitar sin modificaciones de hardware. Una opción consiste en conectar la salida del contador SP con la entrada restante del componente Mux A, como se muestra a continuación:



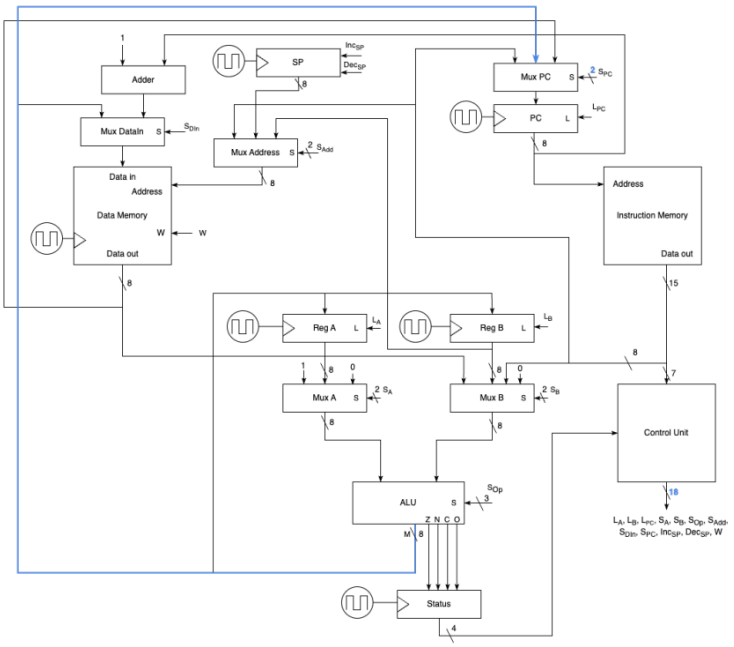
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV B, SP`	0	1	0	0	0	0	SP	ZERO	ADD	-	-	-

Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

2. `JMP B`. Salta a la instrucción con dirección igual al valor `B`.

Solución: Esta instrucción no se puede habilitar sin modificaciones de hardware. Una opción consiste en conectar la salida de la ALU con una de las entradas del componente Mux PC (lo que implica agregar un bit a su señal de selección), como se muestra a continuación:



Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
JMP B	0	0	1	0	0	0	ZERO	B	ADD	-	-	ALU

Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

3. `JMP (B)`. Salta a la instrucción con dirección igual al valor `Mem[B]`.

Solución: Esta instrucción se puede implementar sin modificaciones de hardware, como se muestra a continuación:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
JMP (B)	0	0	1	0	0	0	-	-	-	B	-	DOUT

Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

- Si no se modifica el diagrama: Se otorgan **1.5 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.
- Si se modifica el diagrama: Se otorgan **0.75 ptos.** por la correctitud de la modificación y **0.75 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

4. Asuma que se implementan las instrucciones anteriores y modifique el siguiente fragmento de código de forma que no utilice la instrucción `RET`, pero que se llegue al mismo resultado. Luego, comente sobre su resultado: ¿Cumple la misma función que el original? ¿Existe alguna consideración a tener en cuenta?

```
DATA:
    number_of_rides 0

CODE:
    JMP main

    // Incrementa la cantidad de veces a la que se sube a la montaña rusa
    increase_number_of_rides:
        MOV A, (number_of_rides)
        ADD A, 1
        MOV (number_of_rides), A
        RET

    main:
        PUSH A
        CALL increase_number_of_rides
        POP A
```

Solución: Para simular el comportamiento de `RET` con las instrucciones nuevas, existen dos alternativas:

a) Reemplazar `RET` por:

1) `POP B`
2) `JMP B`

En este caso, el código cumple la función original y no se requiere ningún arreglo adicional.

b) Reemplazar `RET` por:

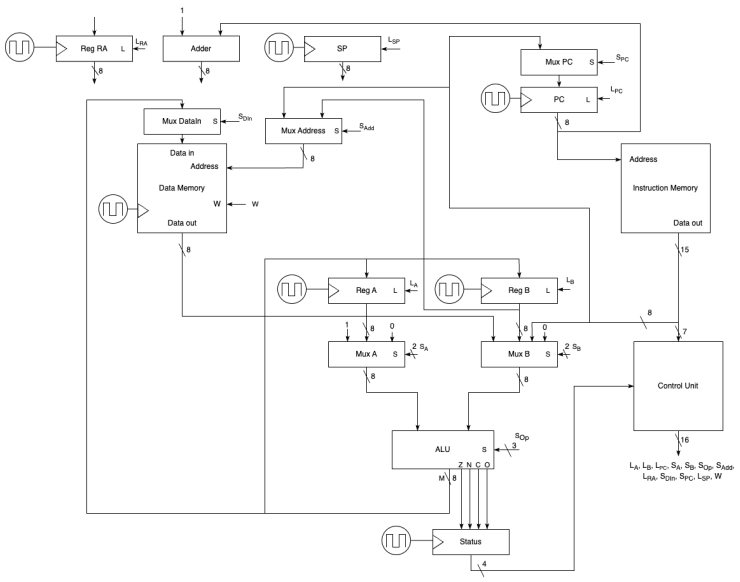
1) `MOV B, SP`
2) `INC B`
3) `JMP (B)`

En este caso, la ejecución no es exactamente igual a la original, dado que si bien se accede correctamente a la dirección de retorno, el valor de `SP` no es ajustado mediante un incremento. Esto genera que la ejecución de `POP A` almacene la dirección de retorno de la subrutina en `A` y no el valor respaldado al comienzo del programa.

Se otorgan **0.75 ptos.** por reemplazar correctamente el uso de `RET` con las instrucciones implementadas; y **0.75 ptos.** por señalar correctamente si su implementación cumple la misma función o no. También se aceptan como correctas alternativas donde se hagan modificaciones adicionales aparte del reemplazo de `RET` para almacenar correctamente el valor de `A`. En caso de existir como máximo un error de implementación en la implementación de código, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

Pregunta 2: Computador básico (P2-I2-2024-2)

En arquitecturas RISC, las direcciones de retorno de una subrutina se almacenan en un **registro de enlace** (`RA`), a diferencia del computador básico donde se guardan en el *stack*. Asimismo, estas arquitecturas no poseen `PUSH/POP`, sino que acceden al *stack* directamente con el *stack pointer* (`SP`). Deberá contestar las preguntas de esta sección a partir de este contexto y el siguiente diagrama:

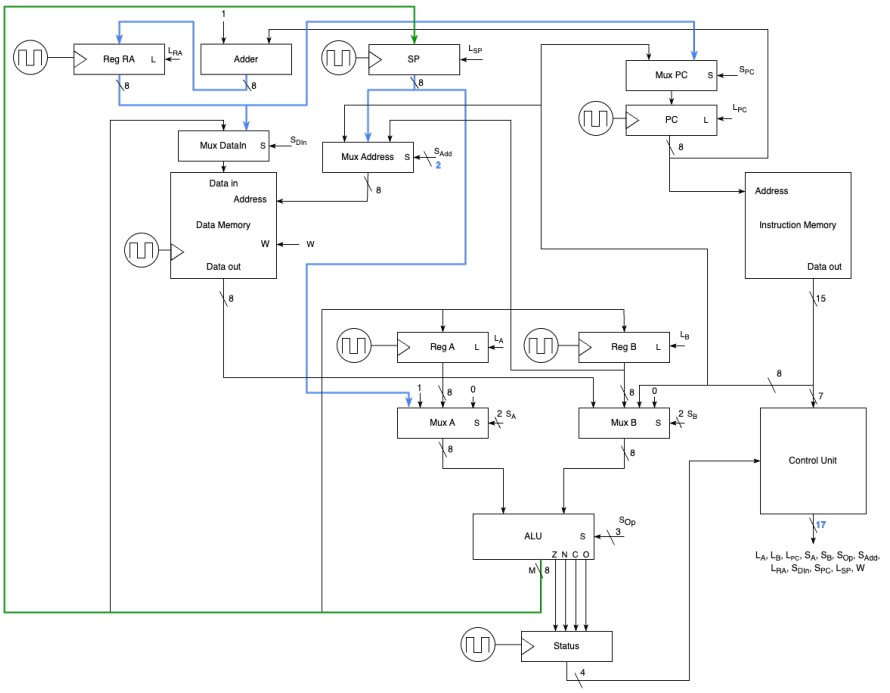


(a) **(5 ptos.)** A partir del diagrama de base adjunto, y asumiendo que se eliminan las instrucciones `CALL`, `RET`, `PUSH` y `POP`, realice las modificaciones de *hardware* necesarias e indique la combinación de señales completa para ejecutar las siguientes instrucciones en un ciclo:

- **(0.5 ptos.)** `MOV A,(SP)`. Guarda en `A` el valor `Mem[SP]`.
- **(0.5 ptos.)** `MOV (SP), A`. Guarda en `Mem[SP]` el valor `A`.
- **(1 pto.)** `ADD SP,Lit`. Guarda en `SP` el valor `SP + Lit`. `Lit` puede ser negativo.
- **(1 pto.)** `MOV (SP), RA`. Guarda en `Mem[SP]` el valor `RA`.
- **(1 pto.)** `JAL RA, label`. Guarda `PC+1` en `RA` y salta a la dirección asociada a `label`.
- **(1 pto.)** `JALR RA`. Salta a la dirección almacenada en `RA`.

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}		
`MOV A,(SP)`														
`MOV (SP), A`														
`ADD SP, Lit`														
`MOV (SP), RA`														
`JAL RA, label`														
`JALR RA`														

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- La conexión entre `SP` y `Mux Address` permite el uso de `SP` como dirección de memoria con la señal S_{Add}, la que ahora debe ser de 2 bits.
- La conexión entre `SP` y `Mux A` permite su uso como primer operando en la `ALU`.
- La conexión entre la salida de la `ALU` y `SP` permite cargar en este último los resultados de las operaciones con la señal L_{SP}.
- La conexión entre `RA` y `Mux DataIn` permite la escritura de `RA` en memoria.
- La conexión entre `RA` y `PC+1` permite escribir este valor en `RA` con la señal L_{RA}.
- La conexión entre `RA` y `Mux PC` permite realizar saltos hacia direcciones almacenadas en `RA`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA

Por cada instrucción, se otorga **la mitad** del puntaje por *hardware* y **la otra mitad** por la combinación de señales. En cada criterio, se descuenta la mitad del puntaje en caso de existir, como máximo, un error de implementación o valor en la combinación de señales. En otro caso, no se otorga puntaje.

(b) **(1 pto.)** Modifique el siguiente fragmento de código para que haga uso exclusivo de las instrucciones anteriores. Debe mantener su funcionamiento original y terminar sin errores.

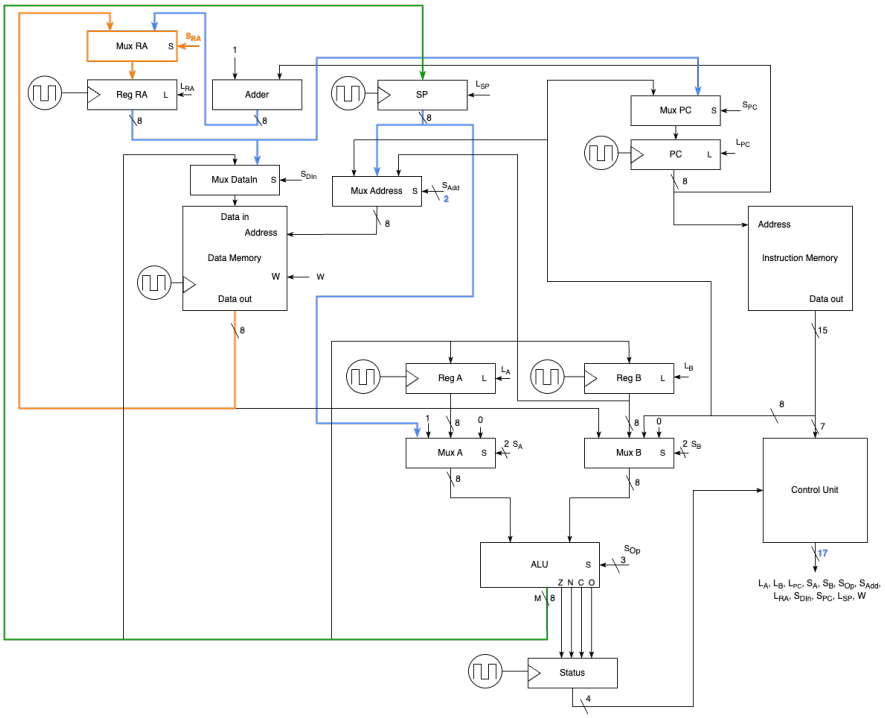
```
MOV A,1
PUSH A
CALL func
POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  CALL func
  end_func:
    RET
end:
```

Solución: Es importante señalar que, con las instrucciones entregadas, **no es posible** entregar un fragmento de código que mantenga el funcionamiento original **sin errores**. Esto se debe al hecho de que no existe una instrucción que permita recuperar una dirección de retorno respaldada en memoria (por ejemplo, `MOV RA, (SP)`. Por este motivo, se consideran correctas respuestas que solo caigan en ese caso. A continuación, una solución posible.

```
MOV A,1
MOV (SP),A        ; Reemplazo por PUSH A
ADD SP,-1         ; Reemplazo por PUSH A
JAL RA,func       ; Reemplazo por CALL func
ADD SP,1          ; Reemplazo por POP A
MOV A,(SP)        ; Reemplazo por POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  MOV (SP),RA     ; Respaldo de RA para evitar error en llamado recursivo
  ADD SP,-1       ; Respaldo de RA para evitar error en llamado recursivo
  JAL RA,func     ; Reemplazo por CALL func
  ADD SP,1        ; De todas formas, se restituye el valor de SP
  MOV RA,(SP)     ; Si existiera, podríamos recuperar el valor respaldado de RA
  end_func:
    JALR RA       ; Reemplazo por RET
end:
```

Se otorgan **0.25 ptos.** por reemplazar correctamente la instrucción `PUSH A`; **0.25 ptos.** por reemplazar correctamente la instrucción `CALL label`; **0.25 ptos.** por reemplazar correctamente la instrucción `POP A`; y **0.25 ptos.** por reemplazar correctamente la instrucción `RET`. Adicionalmente, si la respuesta alude a la falta de una instrucción para recuperar la dirección de retorno; o bien se usa de forma hipotética una instrucción que permita resolver el problema, **se otorgarán 0.5 ptos. de bonus al total**.

A modo de completitud, se incluye tanto el diagrama como la tabla de señales de la arquitectura que habilita la ejecución de la instrucción `MOV RA,(SP)`.



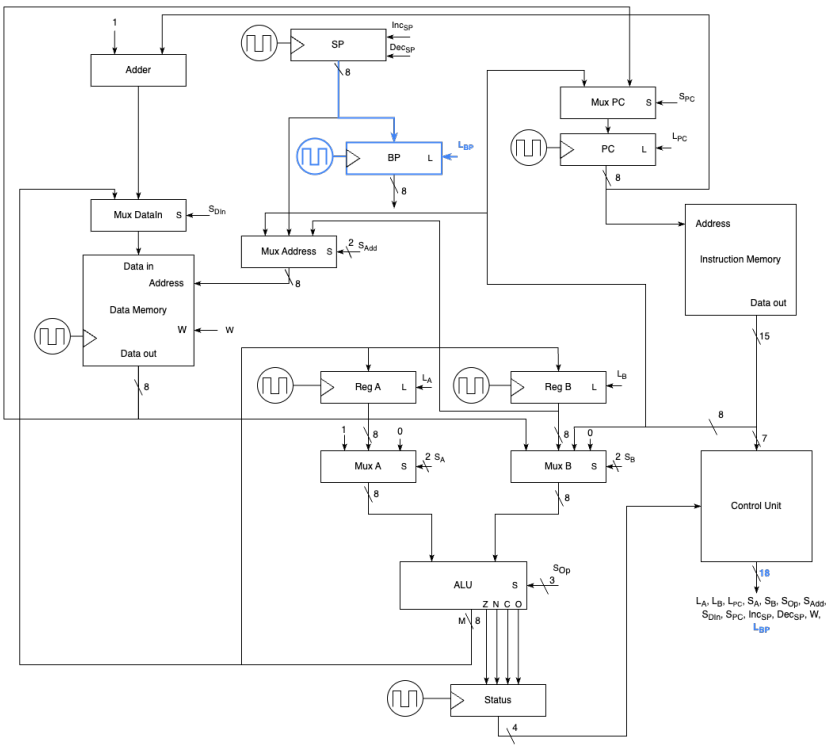
Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	S_{RA}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-	-
`MOV RA, (SP)`	0	0	0	0	1	0	-	-	-	SP	-	-	DOUT
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT	PC+1
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA	-

Pregunta 3: Computador básico (P4-I1-2023-2)

En varias arquitecturas se hace uso del registro *base pointer* (`BP`). Este funciona como punto de referencia para obtener los parámetros y dirección de retorno de cada llamada de una subrutina, lo que facilita el manejo de llamados anidados. Deberá modificar la microarquitectura del computador básico para implementar el registro `BP` de 8 bits y su funcionamiento. Se le listarán las instrucciones que deben ser implementadas. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida ("-" si no afecta). Puede realizar todas las modificaciones en un solo diagrama.

(a) (1 pto.) `MOV BP,SP`. Almacena en el registro `BP` el valor del registro `SP`.

Solución: Para esta modificación, agregamos el registro `BP` a nuestra arquitectura con una nueva señal de carga L_{BP} y con valor de entrada igual a la salida de `SP`. De momento, no conectamos la salida de `BP` con ninguna compuerta (se asume que va a tierra).



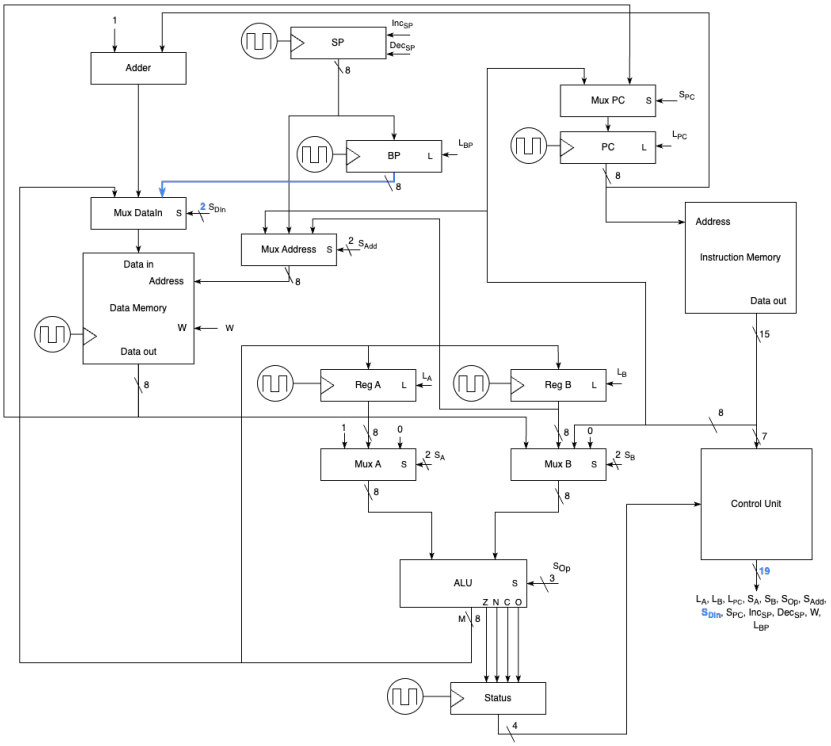
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}
`MOV BP,SP`	0	0	0	0	0	0	-	-	-	-	-	-	1

Se otorgan **0.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para la instrucción.

(b) (1 pto.) `PUSH BP`. Almacena en `Mem[SP]` el valor de `BP` y decrementa `SP` en una unidad.

Solución: Para esta modificación, conectamos la salida del registro `BP` con una de las entradas del componente `Mux DataIn`. Para ello, es necesario incrementar la cantidad de bits de la señal S_{DIn} a dos.



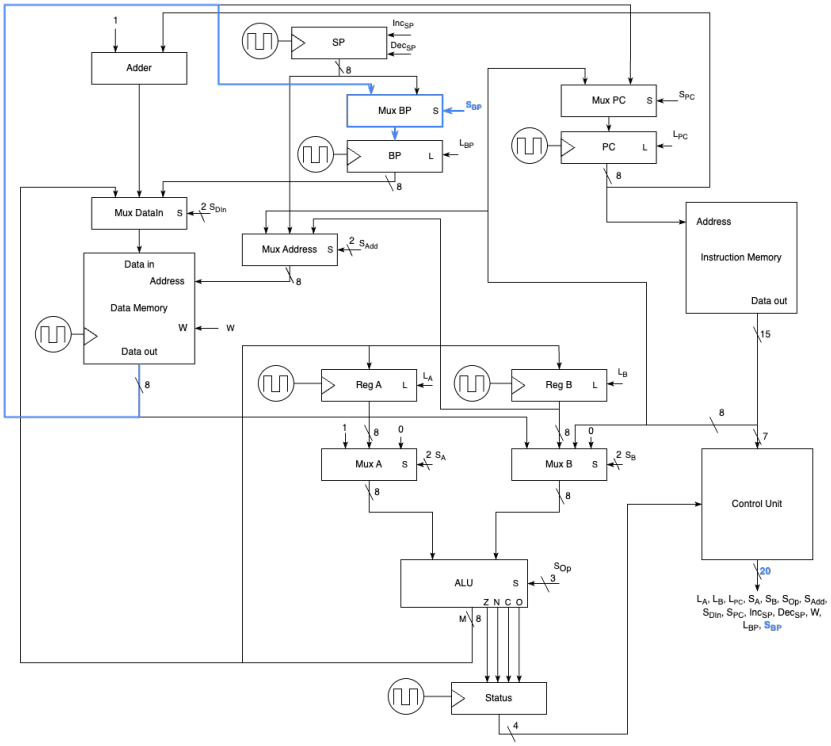
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}
PUSH BP	0	0	0	1	0	1	-	-	-	SP	BP	-	0

Se otorgan **0.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para la instrucción.

(c) (1 pto.) `POP BP`. Incrementa `SP` en una unidad y almacena en `BP` el valor `Mem[SP]`.

Solución: Para esta modificación, conectamos la salida de la memoria de datos con la entrada de `BP`. Dado que ya existe una entrada para `SP`, se agrega un nuevo componente `Mux BP` con señal de selección S_{BP} para escoger el valor de carga entre `SP` y `DOUT`.



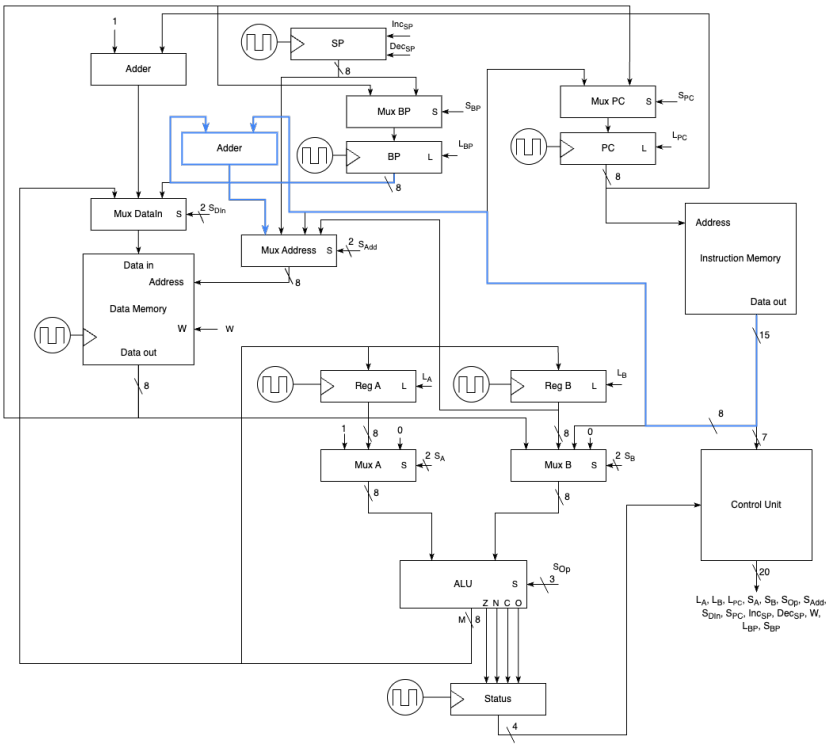
Para ejecutar la instrucción, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
POP BP	0	0	0	0	1	0	-	-	-	-	-	-	0	-
	0	0	0	0	0	0	-	-	-	SP	-	-	1	DOUT

Se otorgan **0.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para la instrucción.

(d) (2 ptos.) `MOV Reg,(BP + Lit)`. Almacena en `Reg` (`A` o `B`) el valor `Mem[BP + Lit]`.

Solución: Para esta modificación, agregamos un `Adder` que recibirá como entradas el valor del registro `BP` y el literal `LIT` de la memoria de instrucciones, para luego conectar su salida al componente `Mux Address`.

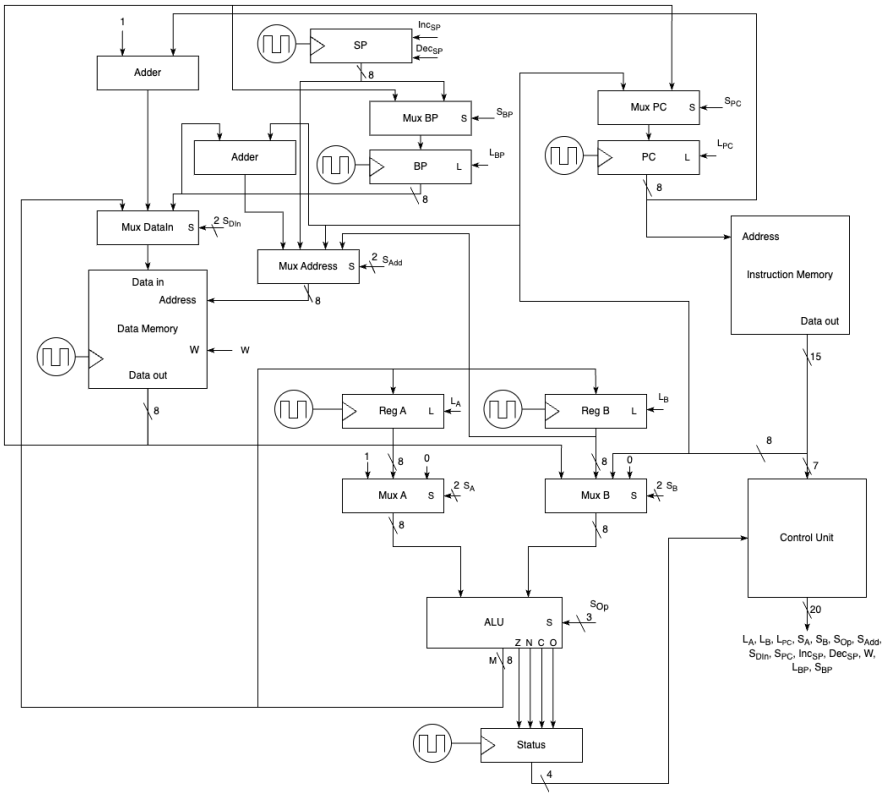


Para ejecutar las instrucciones, se utilizan la siguientes combinaciones de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
`MOV A,(BP + Lit)`	1	0	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-
`MOV B,(BP + Lit)`	0	1	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-

Se otorgan **1.5 ptos.** por la correctitud de la modificación y **0.5 ptos.** por entregar una combinación de señales correcta para una o ambas instrucciones. Se otorgan **0.75 ptos.** por la modificación si esta presenta como máximo un error de implementación.

En la siguiente página se muestra el diagrama completo de la arquitectura esperada y su tabla de instrucciones, asumiendo que se implementan todas las modificaciones juntas.



Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	L_{BP}	S_{BP}
MOV BP,SP	0	0	0	0	0	0	-	-	-	-	-	-	1	SP
PUSH BP	0	0	0	1	0	1	-	-	-	SP	BP	-	0	-
POP BP	0	0	0	0	1	0	-	-	-	-	-	-	0	-
	0	0	0	0	0	0	-	-	-	SP	-	-	1	DOUT
MOV A,(BP + Lit)	1	0	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-
MOV B,(BP + Lit)	0	1	0	0	0	0	0	DOUT	ADD	BP+LIT	-	-	0	-

(e) (3 ptos.) Asuma que se agregan las instrucciones anteriores a la ISA del computador básico y que se ejecuta el programa adjunto. Explique, a grandes rasgos, lo que realiza la subrutina `func` y señale el valor final de la variable `result`. Puede asumir que `BP` parte en 255.

```
DATA:
  n      3
  result 0
CODE:
  MOV A,(n)          // Dirección Mem. Instr.: 0x00
  PUSH A             // Dirección Mem. Instr.: 0x01
  CALL func          // Dirección Mem. Instr.: 0x02
  POP A              // Dirección Mem. Instr.: 0x03-0x04
  JMP end            // Dirección Mem. Instr.: 0x05
  func:
    PUSH BP          // Dirección Mem. Instr.: 0x06
    MOV BP,SP        // Dirección Mem. Instr.: 0x07
    MOV A,(BP + 3)   // Dirección Mem. Instr.: 0x08
    CMP A,0          // Dirección Mem. Instr.: 0x09
    JEQ base_end_0   // Dirección Mem. Instr.: 0x0A
    CMP A,1          // Dirección Mem. Instr.: 0x0B
    JEQ base_end_1   // Dirección Mem. Instr.: 0x0C
    SUB A,1          // Dirección Mem. Instr.: 0x0D
    PUSH A           // Dirección Mem. Instr.: 0x0E
    CALL func        // Dirección Mem. Instr.: 0x0F
    POP A            // Dirección Mem. Instr.: 0x10-0x11
    MOV A,(BP + 3)   // Dirección Mem. Instr.: 0x12
    SUB A,2          // Dirección Mem. Instr.: 0x13
    PUSH A           // Dirección Mem. Instr.: 0x14
    CALL func        // Dirección Mem. Instr.: 0x15
    POP A            // Dirección Mem. Instr.: 0x16-0x17
    JMP end_func     // Dirección Mem. Instr.: 0x18
    base_end_1:
      INC (result)   // Dirección Mem. Instr.: 0x19
    base_end_0:
      NOP            // Dirección Mem. Instr.: 0x1A
    end_func:
      POP BP         // Dirección Mem. Instr.: 0x1B-0x1C
      RET            // Dirección Mem. Instr.: 0x1D-0x1E
  end:
```

Solución: La subrutina `func` corresponde a la función de recurrencia de Fibonacci, *i.e.*:

$$F_N = \begin{cases} F_{N-1} + F_{N-2} & N > 1 \\ N & N \leq 1 \end{cases}$$

Esto se deduce de la siguiente forma:

- Todo `PUSH A` se realiza antes de un llamado a `func`, siendo este el parámetro `n`.
- Mediante `MOV A,(BP + 3)` se recupera el argumento respaldado. Por cada llamado se guardan en el *stack* los valores `A` (parámetro), `PC+1` (retorno) y `BP`. Como `SP` apunta siempre una dirección sobre el tope del *stack* y `BP` tiene el mismo valor por `MOV BP,SP`, entonces `BP + 3` apunta a la dirección en la que se guardó el parámetro.
- Se realizan dos llamados recursivos al interior de `func`, siendo uno de ellos realizado con el parámetro `n - 1` (proveniente de `SUB A,1`) y el otro con el parámetro `n - 2` (proveniente de `SUB A,2`).

- `base_end_0` y `base_end_1` corresponden a los casos base de la recursión y se llega a ellos si, y solo si `A` es 0 o 1 respectivamente. Interesa sobre todo `base_end_1`, dado que en dicho caso se incrementa el valor de la variable `result`.
- Al terminar un llamado de la subrutina en `end_func`, se reestablece el *stack* al ejecutar `POP BP`, `RET` y `POP A` luego de retornar. Esto asegura que siempre se utilice el valor correcto de `BP` para acceder al parámetro de cada llamado recursivo.

A partir de lo anterior, se deduce entonces que el valor de `result` es igual a:

$$\begin{aligned} F_3 &= F_2 + F_1 \\ &= F_1 + F_0 + F_1 \\ &= 1 + 0 + 1 \\ &= 2 \end{aligned}$$

Se otorgan **2 ptos.** por señalar correctamente el resultado de la ejecución correctamente y **1 pto.** por describir `func` a partir de Fibonacci. Si no se cumple ninguno de los dos criterios anteriores, se otorga **1 pto.** si se muestra un desarrollo parcial pero correcto de la ejecución de `func`.

Pregunta 4: Assembly con subrutinas (P3-I1-2024-1)

(a) Indique los valores de los registros `A` y `B` al finalizar la ejecución del código **(a)**.

(b) Indique los valores de los registros `A` y `B` al finalizar la ejecución del código **(b)**.

(c) El código **(c)** *debería* computar la multiplicación entre las variables `X` e `Y`, pero cuenta con errores que generan un resultado erróneo. Indique el valor de los registros `A`, `B` y la variable `res` al finalizar la ejecución del código. Luego, señale dónde se encuentra el o los errores y cómo se resuelven.

(a)

```
DATA:
    res     0

CODE:
    JMP _start

    func_1:
        SHR A, A
        JCR func_2
        CMP A, 0
        JNE func_1
        RET

    func_2:
        NOT B, A
        MOV (B), 14
        INC (res)
        JMP func_1

    _start:
        MOV A, 6
        CALL func_1
        MOV B, (res)
```

(b)

```
DATA:
    var   10

CODE:
    JMP _start

    func_1:
        MOV B, (var)
        NOT A, A
        ADD A, 1
        ADD B, A
        RET

    _start:
        MOV A, 3
        CALL func_1

    end:
        MOV A, (255)
```

(c)

```
DATA:
    res     0
    X       3
    Y       4
    iter    0

CODE:
    MOV A, (Y)
    PUSH A
    CALL mult
    MOV A, (res)
    JMP end

    mult:
        POP B
    sumar:
        MOV A, (res)
        ADD A, (X)
        MOV (res), A
        INC (iter)
        MOV A, (iter)
        CMP A, B
        JNE sumar
        RET

    end:
```

Consideraciones:

1. La primera instrucción del segmento de `CODE` se guardará en la dirección 0 de la memoria de instrucciones.
2. En las instrucciones de *shifting*, el bit descartado se almacena en el *condition code* de *carry*.

Solución:

(a) Los valores finales de los registros `A` y `B` son 0 y 255 respectivamente. La explicación es la siguiente:

- Al comenzar la ejecución de código, se inicializa el registro `A` con el literal 6 y luego se llama a la subrutina `func_1`.
- Lo que realiza esta subrutina es lo siguiente: (1) Aplica un *shift right* sobre el registro `A`, equivalente a actualizar su valor con el resultado de su división entera por 2; (2) se revisa si se activó la señal de *carry*, lo que ocurre si el bit descartado en el *shift right* fue igual a 1 (*i.e.* se dividió un número impar). Si es el caso, se salta al *label* `func_2`; en otro caso; (3) se revisa si el valor de `A` es igual a 0, en cuyo caso se retorna; si no, se vuelve a ejecutar desde un inicio la subrutina.
- En caso de que se gatille `func_2`, lo que realiza este fragmento es lo siguiente: (1) Almacena en el registro `B` la negación de `A`; (2) utiliza este valor como dirección de la memoria de datos y almacena en dicha ubicación el literal 14; (3) incrementa en una unidad la variable `res`; (4) vuelve a retomar la ejecución de la subrutina `func_1`.
- Para descifrar los valores finales, es necesario ejecutar todos los ciclos de la subrutina:
  1. `A` = `SHL(A)` = 3, $C = 0$. No se realiza el salto con `JCR` y se repite la ejecución de la subrutina.
  2. `A` = `SHL(A)` = 1, $C = 1$. Se se realiza el salto con `JCR` y se tiene que `B` = `NOT(00000001)` = 11111110. Por lo tanto, en dicha dirección se almacena el literal 14: `Mem[11111110]` = `Mem[254]` = 14. La variable `res` incrementa su valor a 1 y se procede a repetir la ejecución de la subrutina.
  3. `A` = `SHL(A)` = 0, $C = 1$. Se se realiza el salto con `JCR` y se tiene que `B` = `NOT(00000000)` = 11111111. Por lo tanto, en dicha dirección se almacena el literal 14: `Mem[11111111]` = `Mem[255]` = 14. La variable `res` incrementa su valor a 2 y se procede a repetir la ejecución de la subrutina.
  4. `A` = `SHL(A)` = 0, $C = 0$. No se realiza el salto con `JCR` y finaliza la ejecución, siendo entonces este el valor final del registro `A`. Al retornar, se carga en el contador `PC` el contenido del tope del `stack`, en este caso, `PC` = `Mem[255]` = 14.
- Considerando que `JMP _start` se almacena en la dirección 0 y que `RET` ocupa dos direcciones en la memoria de instrucciones, se tiene que la dirección de retorno original de la subrutina, correspondiente a la instrucción `MOV B, (res)`, es igual a 13. No obstante lo anterior, este valor se sobreescribe con la subrutina con el literal 14, lo que implica que al retornar el programa **no ejecute** esta instrucción y, por ende, `B` mantenga el valor 255 computado dentro de la subrutina.

Importante: Si bien la instrucción `MOV (B),Lit` no es parte de la ISA del computador básico, la microarquitectura permite su ejecución, por lo que durante la interrogación se informa que se puede asumir que existe y se puede utilizar en este fragmento de código.

Se otorgan **0.75 ptos.** por cada valor final señalado correctamente. En caso de que un valor esté erróneo pero exista desarrollo de la ejecución de código para su obtención, se otorgan **0.375 ptos.** como puntaje parcial.

(b) Los valores finales de los registros `A` y `B` son 9 y 7 respectivamente. La explicación es la siguiente:

- Al comenzar la ejecución del programa, se inicializa el registro `A` con el literal 3 y luego se llama a la subrutina `func_1`.
- Dentro de la subrutina, se almacena el valor de la variable `var` en `B`, que es igual a 10.
- Posteriormente, se actualiza el valor del registro `A` y se reemplaza por $\overline{\texttt{A}} + 1$, lo que es equivalente a computar su inverso aditivo, es decir, -3.
- Se le suma al registro `B` lo computado en `A`, por lo que $\texttt{B} = 10 + -3 = 7$. Este es el valor final del registro.
- Finalmente, al retornar, se almacena en `A` el valor de la dirección de memoria 255, correspondiente al primer valor almacenado en la memoria de *stack*: la dirección de retorno del llamado a `func_1`. Considerando que `JMP _start` se almacena en la dirección 0 y que `RET` ocupa dos direcciones en la memoria de instrucciones, se tiene entonces que la dirección de retorno del llamado, correspondiente a la ubicación de la instrucción `MOV A, (255)`, es igual a 9, siendo este el valor final de `A`. Es importante recordar que el incremento de `SP` en el retorno **no elimina el valor almacenado en el tope del *stack***.

Se otorgan **0.75 ptos.** por cada valor final señalado correctamente. En caso de que un valor esté erróneo pero exista desarrollo de la ejecución de código para su obtención, se otorgan **0.375 ptos.** como puntaje parcial.

(c) Los valores finales de los registros `A` y `B` son 3 y 3 respectivamente, mientras que el valor de la variable `res` es 9. La explicación es la siguiente:

- Al comenzar la ejecución del programa, se almacena en `A` el valor de la variable `Y` (correspondiente a 4) y se respalda en la memoria de *stack* a través de un `PUSH`.
- Se hace el llamado a la subrutina `mult`, almacenando en el tope del *stack* la dirección de retorno que apunta a la instrucción `MOV A,(res)`. No obstante, lo primero que se ejecuta en la subrutina es un `POP B`, lo que hace que se almacene la dirección de retorno en el registro `B` y **cambiando la posición del contador `SP`**, lo que tendrá consecuencias a ser descritas más adelante.
- Se empieza a computar la multiplicación a partir de la suma iterativa de la variable `X` sobre `res`, ejecutándose hasta que el total de iteraciones sea igual al valor de `B`. En este punto, se esperaría que la suma se repita `Y` veces para computar correctamente la multiplicación, pero el registro `B` ahora posee la dirección de memoria de la instrucción `MOV A, (res)`. Considerando que `MOV A, (Y)` se almacena en la dirección 0, se tiene que la dirección de retorno almacenada en `B` es 3. Por lo tanto, lo que se computará será la multiplicación entre `X` igual a 3 y 3, cuyo resultado es igual a 9 y no 12 como se esperaría. Este valor se almacena en la variable `res` y se termina la ejecución de la subrutina.
- Al retornar de la subrutina, se guarda en el contador `PC` el valor del tope del *stack*, correspondiente al valor del registro `A` respaldado en un comienzo. En este caso el

valor fue igual a 4, que corresponde a la dirección de la instrucción `JMP end`, por lo que no se ejecuta `MOV A, (res)` y el programa termina sin actualizar el valor de este registro. Por lo tanto, el valor del registro `A` es igual al asignado con la instrucción `MOV A,(iter)` en la última iteración, que fue igual a 3 (correspondiente a la cantidad de iteraciones ejecutadas). El valor de `B` no se vuelve a actualizar después de la ejecución de la instrucción `POP B`, por lo que su valor es igual a 3.

Para que la multiplicación se compute correctamente, basta con asegurar que: (1) a `B` se le asigne correctamente el valor de la variable `Y`; y (2) que en el tope del *stack* se encuentre almacenada la dirección de retorno de la subrutina. Esto se puede realizar de muchas maneras, pero a continuación se deja un ejemplo:

```
DATA:
    res     0
    X       3
    Y       4
    iter    0

CODE:
    MOV B, (Y)
    CALL mult
    MOV A, (res)
    JMP end

    mult:
        sumar:
        MOV A, (res)
        ADD A, (X)
        MOV (res), A
        INC (iter)
        MOV A, (iter)
        CMP A, B
        JNE sumar
        RET

    end:
```

Se otorgan **0.5 ptos.** por cada valor final señalado correctamente; **0.75 ptos.** por señalar correctamente el error en el programa; y **0.75 ptos.** por indicar una forma válida para su resolución. En caso de que un valor esté erróneo pero exista desarrollo de la ejecución de código para su obtención, se otorgan **0.25 ptos.** como puntaje parcial.

Pregunta 5: Assembly con subrutinas (P1-I1-2023-2)

Indique el valor final de los registros `A` y `B` luego de ejecutar el siguiente fragmento de código escrito en el Assembly del computador básico.

```
CODE:
    MOV A,7
    MOV B,1
    PUSH B
    PUSH A
    RET
    MOV A,0
    MOV B,0
    JMP end
    end:
```

Solución: Cuando se ejecuta el fragmento de código anterior, `RET` gatilla la carga del valor numérico del tope del *stack* en el registro `PC`. En este caso, el tope del `stack` posee el valor 7 y por ende el programa saltará a dicha dirección en la memoria de instrucciones. A continuación, se muestra el mismo fragmento mostrando las direcciones en las que se almacena cada instrucción.

```
CODE:
    MOV A,7     ;Dirección 0x00
    MOV B,1     ;Dirección 0x01
    PUSH B      ;Dirección 0x02
    PUSH A      ;Dirección 0x03
    RET         ;Dirección 0x04-0x05
    MOV A,0     ;Dirección 0x06
    MOV B,0     ;Dirección 0x07       <- RET gatilla un salto aquí.
    JMP end     ;Dirección 0x08
    end:
```

Por lo tanto, el programa terminará ejecutando las instrucciones `MOV B,0` y `JMP end`, saltándose `MOV A,0`. De esta forma, el valor final del registro `A` es igual a 7 y el del registro `B` es igual a 0. Se otorga **1 pto.** por cada registro donde se indique su valor final correcto

Pregunta 6: Ejecución de código

Para los siguientes fragmentos de código indique los valores de los registros `A` y `B` al finalizar la ejecución.

(1)

```
DATA:
    val1  6
    val2  3

CODE:
    MOV A,4
    MOV B,2
    PUSH A
    CALL func
    POP B
    JMP end

    func:
        PUSH B
        MOV B,A
        ADD A,B
        MOV B,A
        SUB B,2
        MOV A,(val2)
        POP A
        MOV (val1),B
        RET

    end:
```

(2)

```
DATA:
    val1  8
    val2  5

CODE:
    MOV A,3
    MOV B,6
    PUSH B
    CALL func
    POP A
    JMP end

    func:
        PUSH A
        MOV A,B
        SHL A,A
        MOV B,A
        ADD B,2
        MOV A,(val1)
        POP B
        MOV (val2),A
        RET

    end:
```

Solución:

6.1) Antes de llamar a `func`, se guarda `A` (valor 4) en el *stack* con `PUSH A`. Al entrar a la subrutina, se guarda `B` (valor 2) con `PUSH B`. Dentro de `func`, `A` se duplica con `ADD A,B` resultando en 8, luego `B` toma ese valor y se le resta 2 quedando en 6. Posteriormente `A` se sobreescribe con el valor de `val2` (3), y el `POP A` restaura el valor 2 desde el *stack* (el `B` original). Al retornar, `POP B` extrae el valor 4 (el `A` original guardado antes del `CALL`).

```
DATA:
    val1  6   // MEM[0] = 6
    val2  3   // MEM[1] = 3

CODE:
    MOV A,4       // A = 4
    MOV B,2       // B = 2
    PUSH A        // MEM[255] = 4, SP-- => SP = 254
    CALL func     // MEM[254] = PC+1 (dir. POP B), SP-- => SP = 253

    func:
        PUSH B        // MEM[253] = 2, SP-- => SP = 252
        MOV B,A       // B = 4
        ADD A,B       // A = 4 + 4 = 8
        MOV B,A       // B = 8
        SUB B,2       // B = 6
        MOV A,(val2)  // A = MEM[1] = 3
        POP A         // SP++ => SP = 253, A = MEM[253] = 2
        MOV (val1),B  // MEM[0] = 6
        RET           // SP++ => SP = 254, PC = MEM[254] = dir. POP B

    POP B         // SP++ => SP = 255, B = MEM[255] = 4
    JMP end

    end:          // A = 2, B = 4
```

6.2) Antes de llamar a `func`, se guarda `B` (valor 6) en el *stack* con `PUSH B`. Al entrar a la subrutina, se guarda `A` (valor 3) con `PUSH A`. Dentro de `func`, `A` toma el valor de `B` (6) y se aplica `SHL A,A` resultando en 12. Luego `B` toma ese valor y se le suma 2 quedando en 14. A continuación `A` se sobreescribe con el valor de `val1` (8), y el `POP B` restaura el valor 3 desde el *stack* (el `A` original). Al retornar, `POP A` extrae el valor 6 (el `B` original guardado antes del `CALL`).

```
DATA:
    val1  8   // MEM[0] = 8
    val2  5   // MEM[1] = 5

CODE:
    MOV A,3       // A = 3
    MOV B,6       // B = 6
    PUSH B        // MEM[255] = 6, SP-- => SP = 254
    CALL func     // MEM[254] = PC+1 (dir. POP A), SP-- => SP = 253

    func:
        PUSH A        // MEM[253] = 3, SP-- => SP = 252
        MOV A,B       // A = 6
        SHL A,A       // A = 12
        MOV B,A       // B = 12
        ADD B,2       // B = 14
        MOV A,(val1)  // A = MEM[0] = 8
        POP B         // SP++ => SP = 253, B = MEM[253] = 3
        MOV (val2),A  // MEM[1] = 8
        RET           // SP++ => SP = 254, PC = MEM[254] = dir. POP A

    POP A         // SP++ => SP = 255, A = MEM[255] = 6
    JMP end

    end:          // A = 6, B = 3
```

Pregunta 7: Corrección de código

Un grupo de estudiantes del ramo Arquitectura de Computadores implementó en *Assembly* un programa que recorre un arreglo de enteros con signo y reemplaza cada elemento por su valor absoluto, utilizando una subrutina para realizar la operación. Al revisar el código, el profesor del curso detectó que este contiene 2 errores que impiden su correcto funcionamiento. Identifique los errores, explique por qué son incorrectos y entregue el código corregido.

```
DATA:
    arr     3
            -2
            -5
            4
    len     4
    i       0

CODE:
    MOV B,arr
    loop:
        MOV A,(i)
        CMP A,(len)
        JEQ end
        PUSH B
        MOV A,(B)
        CALL abs_val
        MOV (B),A
        POP B
        INC B
        MOV A,(i)
        ADD A,1
        MOV (i),A
        JMP loop
    end:
        JMP end

    abs_val:
        CMP A,0
        JGE done
        NOT A,A
        ADD A,1
        done:
            RET
```

Solución:

El programa de los estudiantes recorre el arreglo `[3, -2, -5, 4]` y reemplaza cada elemento por su valor absoluto. La subrutina `abs_val` recibe el elemento en `A` y, si es negativo, aplica complemento a dos (`NOT A,A` seguido de `ADD A,1`) para obtener su equivalente positivo, escribiendo el resultado de vuelta en la misma posición del arreglo con `MOV (B),A`.

El primer error está en el loop principal: `PUSH B` aparece antes de `MOV A,(B)`, por lo que cuando se llama a `abs_val`, A aún contiene el índice `i` y no el elemento del arreglo. La corrección es mover `MOV A,(B)` antes del `PUSH B`. El segundo error está en la subrutina: los estudiantes olvidaron preservar `B` con `PUSH B` y `POP B`, lo que corrompe el puntero del arreglo en cada iteración. Agregando `PUSH B` al inicio de `abs_val` y `POP B` antes del `RET`, ambos errores quedan corregidos y el resultado final es `[3, 2, 5, 4]`.

```
DATA:
    arr     3
            -2
            -5
            4
    len     4
    i       0

CODE:
    MOV B,arr
    loop:
        MOV A,(i)
        CMP A,(len)
        JEQ end
        MOV A,(B)       // Error 1 corregido: hay que cargar el elemento
        PUSH B          // en A ANTES del PUSH B y el CALL, de lo contrario
        CALL abs_val    // A contiene el índice i, no el elemento del arreglo
        MOV (B),A
        POP B
        INC B
        MOV A,(i)
        ADD A,1
        MOV (i),A
        JMP loop
    end:
        JMP end

    abs_val:
        PUSH B          // Error 2 corregido: respaldamos el valor del registro B
        CMP A,0         // dentro de la subrutina antes de usarlo como auxiliar.
        JGE done        // Sin PUSH/POP B el registro queda corrupto al retornar
        NOT A,A
        ADD A,1
        done:
            POP B
            RET
```

Pregunta 8: Análisis de código

A continuación se presenta un código escrito en *Assembly*. A partir de él, realice la ejecución y describa el propósito funcional del algoritmo.

```
DATA:
    arr 3
        4
        7
        8
    len 4
CODE:
    MOV B,arr
    loop:
        MOV A,(len)
        CMP A,B
        JEQ end
        MOV A,(B)
        PUSH A
        SHR A,A
        JCR path_1
        path_2:
            POP A
            CALL op_2
            JMP op_3
        path_1:
            POP A
            CALL op_1
        op_3:
            INC B
            JMP loop
op_1:
    SHL A,A
    MOV (B),A
    RET
op_2:
    SHR A,A
    MOV (B),A
    RET

end:
```

Solución:

El programa recorre un arreglo de cuatro elementos `[3, 4, 7, 8]` y aplica una operación distinta según la paridad de cada elemento. Para cada elemento, se realiza un `SHR A,A` y se revisa el *flag* `C` (*carry*). Si el *bit* menos significativo del elemento era 1 antes del *shift*, es decir, si el número es impar, el *flag* `C` queda en 1 y se salta a `path_2`, donde se hace `POP A` para recuperar el valor original y se llama a `op_2`, que aplica `SHL A,A`, multiplicando el elemento por 2. Si el *bit* menos significativo era 0, es decir, el número es par, el *flag* `C` queda en 0 y se ejecuta `path_2`, donde se recupera el valor original y se llama a `op_2`, que aplica `SHR A,A`, dividiendo el elemento por 2.

En ambos casos el resultado se escribe de vuelta en la misma posición del arreglo con `MOV (B),A`, y al terminar la subrutina se avanza el puntero `B` para pasar al siguiente elemento. A continuación, se muestra la ejecución de la primera iteración del loop:

```
DATA:
    arr 3
        4
        7
        8
    len 4
    i 0
CODE:
    MOV B,arr           // B = 0
    loop:
        MOV A,(len)     // A = 4
        CMP A,B         // Z = 0
        JEQ end
        MOV A,(B)       // A = 3
        PUSH A          // Mem[255] = 3, SP-- => SP = 254
        SHR A,A         // A = 1, C = 1
        JCR path_1
        path_2:
            POP A
            CALL op_2
            JMP op_3
        path_1:
            POP A       // SP ++ => SP = 255, A = MEM[255] = 3
            CALL op_1
        op_3:
            INC B       // B = 1
            JMP loop
op_1:
    SHL A,A             // A = 6
    MOV (B),A           // (B) = 6
    RET
op_2:
    SHR A,A
    MOV (B),A
    RET

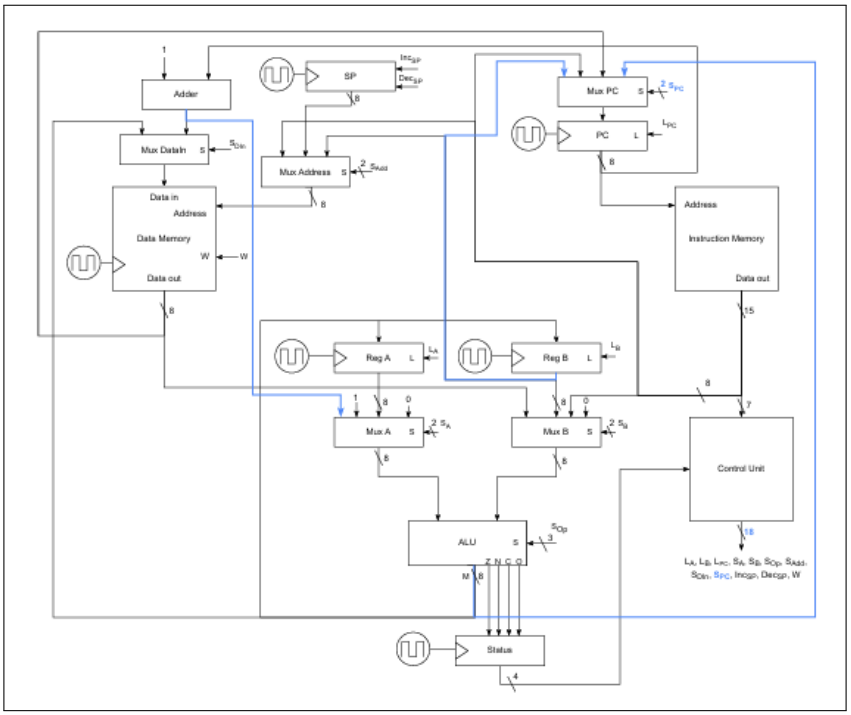
end:
```

Pregunta 9: Modificación al computador básico (I2 2025-2)

A continuación, se le describirán instrucciones que deberá implementar en el computador básico con subrutinas a partir del diagrama visto en clases. Para cada una de ellas, debe incluir la combinación completa de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (`1`) o no (`0`); en las señales de selección, debe indicar el nombre de la entrada escogida ("`-`" si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. Cada instrucción debe ejecutarse en un ciclo.

- `CALL A,B`. Almacena en el registro `A` la dirección de retorno `PC+1` y salta a la dirección de memoria almacenada en el registro `B`.
- `CALL B,Dir`. Almacena en el registro `B` la dirección de retorno `PC+1` y salta a la dirección de memoria equivalente al literal `Dir`.
- `RET A`. Salta a la dirección de retorno almacenada en el registro `A`.
- `RET B`. Salta a la dirección de retorno almacenada en el registro `B`.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se conecta la salida del `Adder` que computa `PC+1` con la entrada restante del componente `Mux A`, de forma que se pueda tener este valor como salida de la `ALU` para su almacenamiento en los registros `A` o `B`.
- Se conecta la salida de la `ALU` con una de las entradas del componente `MuxPC`, de forma que se pueda seleccionar el valor del registro `A` o del registro `B` como direcciones de memoria a saltar.
- Se conecta la salida del registro `B` con una de las entradas del componente `MuxPC`, de forma que se pueda seleccionar el valor de este como dirección de memoria a saltar, mientras que al mismo tiempo se compute `PC+1` desde la `ALU`.
- Finalmente, incrementa a 2 *bits* la señal S_{PC} para poder soportar las dos nuevas entradas del multiplexor `MuxPC`.

Instrucción	L_A	L_B	L_{PC}	Inc_{SP}	Dec_{SP}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
CALL A,B	1	0	1	0	0	0	PC+1	ZERO	ADD	-	-	B
CALL B,Dir	0	1	1	0	0	0	PC+1	ZERO	ADD	-	-	LIT
RET A	0	0	1	0	0	0	A	ZERO	ADD	-	-	ALU
RET B	0	0	1	0	0	0	ZERO	B	ADD	-	-	ALU

Pregunta 10: Modificación al computador básico (2025-1)

a) Has sido contratado/a como programador/a en una bolsa de valores internacional. Al revisar la infraestructura tecnológica, nota que todos los procesadores se basan en el computador básico del curso, lo que limita el rendimiento necesario para reaccionar ante cambios del mercado. Su misión es diseñar una versión mejorada, llamada *FastBasic Computer*, que permita ejecutar programas con mayor velocidad y eficiencia. Para ello, deberá incorporar instrucciones nuevas que disminuyan los ciclos de ejecución. Deberá modificar la microarquitectura del computador básico para implementar las instrucciones nuevas y su funcionamiento. Para cada instrucción nueva, deberá incluir la combinación completa de señales que la ejecutan. Por cada señal de carga/escritura/incremento/decremento, deberá indicar si se activan (1) o no (0); en las señales de selección, deberá indicar el nombre de la entrada escogida ("-" si no afecta). Puede realizar todas las modificaciones en un solo diagrama.

- Implemente la instrucción `CALLFAST label`: Almacena `PC+1` en un nuevo registro llamado *Return Address Register* y no modifica el *stack*. Además, salta a la dirección `label`.
- Implemente la instrucción `RETFAST`: Carga en el contador `PC` el valor almacenado en el registro *Return Address Register* y no modifica el `stack`.
- Implemente la instrucción `JEQFAST A, Lit, label`: Ejecuta la operación `A - Lit` en la `ALU` y salta a la dirección asociada a `label` si, y solo si, la bandera `Z` computada en el mismo ciclo es igual a 1.

b) Para evaluar la velocidad de *FastBasic Computer*, se realiza una comparativa entre un programa escrito en el *Assembly* del computador básico (a) y otro equivalente en la nueva arquitectura (b). Indique los valores de los registros `A` y `B` al finalizar la ejecución del código en cada caso, junto con la cantidad de ciclos de ejecución. Asuma que no se agregan instrucciones para inicializar la memoria `RAM`.

(1)

```
DATA:
    n 8
    is_even 0
CODE:
    MOV A,(n)
    CALL check_is_even
    JMP end
    check_is_even:
        MOV B,1
        AND A,B
        CMP A,0
        JEQ set_is_even
    end_check_is_even:
        RET
    set_is_even:
        MOV (is_even),B
        JMP end_check_is_even
    end:
        MOV A,(is_even)
```

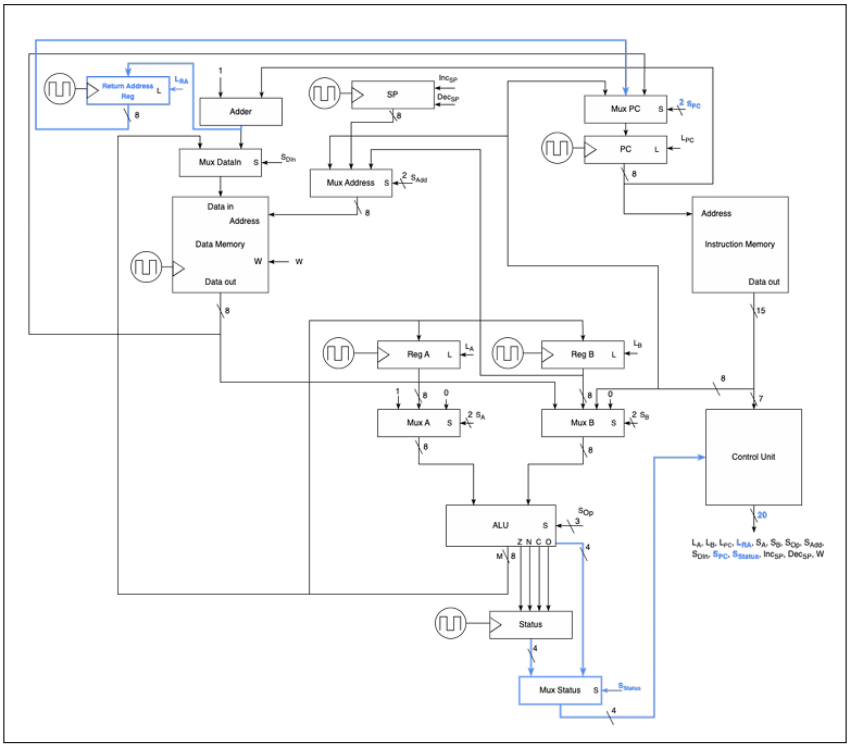
(2)

```
DATA:
    n 8
    is_even 0
CODE:
    MOV A,(n)
    CALLFAST check_is_even
    JMP end
    check_is_even:
        MOV B,1
        AND A,B
        CMP A,0
        JEQFAST set_is_even
    end_check_is_even:
        RETFAST
    set_is_even:
        MOV (is_even),B
        JMP end_check_is_even
    end:
        MOV A,(is_even)
```

c) Las instrucciones `CALLFAST label` y `RETFAST` presentan un problema al ejecutar llamados anidados, situación que no ocurre con `CALL` y `RET`. En particular, se observa que no se realiza de forma correcta el retorno. Explique porqué ocurre esto.

Solución:

a) A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se añade el registro *Return Address Register* con su señal de carga $\texttt{L}_{\texttt{RA}}$.
- La conexión entre la salida del `Adder` que computa `PC + 1` y la entrada de datos de *Return Address Register* permite cargar el valor en el registro con la señal $\texttt{L}_{\texttt{RA}}$.
- La conexión entre la salida de *Return Address Register* y una de las entradas del multiplexor `Mux PC` permite seleccionar el valor del registro como dato a cargar en el contador `PC`. Es importante destacar que, al tener tres valores a seleccionar en el multiplexor, es necesario incrementar a 2 *bits* la señal de selección $\texttt{S}_{\texttt{PC}}$.
- Se añade el multiplexor `Mux Status` y su señal de selección $\texttt{S}_{\texttt{Status}}$ para seleccionar las *flags* a ser evaluadas por la Unidad de Control. Si provienen del registro `Status`, entonces se utilizan las *flags* computadas en el ciclo anterior; en otro caso, se utilizan las computadas en el ciclo actual.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	L_{RA}	Inc_{SP}	Dec_{SP}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	S_{Status}
`CALLFAST label`	0	0	1	1	0	0	0	-	-	-	-	-	LIT	-
`RETFAST`	0	0	1	0	0	0	0	-	-	-	-	-	RA	-
`JEQFAST A, Lit, label`	0	0	Z == 1	0	0	0	0	A	LIT	SUB	-	-	LIT	ALU

Algunas consideraciones importantes:

- En la columna S_{PC}, R_A representa la selección de la salida de *Return Address Register*.
- En la columna S_{Status}, ALU representa la selección de las *flags* proveniente de ella y no del registro Status.
- En la columna L_{PC}, no importa la notación, pero sí debe quedar claro que su valor no es por defecto 1, sino que depende de la *flag* Z.
- En la instrucción `JEQFAST A, Lit, label` existe una limitante importante: se debe usar el mismo literal tanto para la comparación como para el `label` dado que en la memoria de instrucciones solo se define uno. A modo de simplificación, se considerará correcto "asumir" que pueden ser distintos sin modificaciones adicionales de *hardware*, pero en la práctica su implementación requiere de la definición de dos literales por instrucción, lo que implicaría más cambios en la microarquitectura.

b) Los programas (a) y (b) determinan si la variable `n` es par (`is_even` = 1) o no (`is_even` = 0) a partir de la revisión de su *bit* menos significativo (`AND n, 1`). En ambos casos, el valor final de `A` será igual a 1, ya que es correspondiente al resultado de `is_even` para `n` igual a 8; mientras que el valor de `B` también será 1, dado que se le asigna este literal y no se modifica de nuevo. En términos de ejecución, el programa (a) toma un total de 12 ciclos, ya que se debe considerar que se realiza el salto condicional `JEQ_set_is_even` y que `RET` ejecuta en dos ciclos; (b), en cambio, toma 10 ciclos al no requerir el uso de `CMP` y al tener un retorno de un solo ciclo con `RETFAST`.

c) Si se tienen llamados anidados, existirá más de una dirección de retorno (una por llamado). Al usar `CALL` y `RET`, las direcciones se respaldan en el *stack*, que simplemente incrementa su tamaño. Al usar `CALLFAST` y `RETFAST`, en cambio, se tiene un único registro que se sobrescribe con cada llamado, manteniendo así solo la última dirección de retorno.

Instrucción	Operandos	*Opcode*	Condición	L_{PC}	L_A	L_B	S_A0,1	S_B0,1	S_{Op}0,1,2	S_{Add}0,1	S_{Din}	S_{PC}	W	Inc_{SP}	Dec_{SP}
MOV	A,B	0000000		0	1	0	ZERO	B	ADD	-	-	-	0	0	0
	B,A	0000001		0	0	1	A	ZERO	ADD	-	-	-	0	0	0
	A,Lit	0000010		0	1	0	ZERO	LIT	ADD	-	-	-	0	0	0
	B,Lit	0000011		0	0	1	ZERO	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0000100		0	1	0	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	B,(Dir)	0000101		0	0	1	ZERO	DOUT	ADD	LIT	-	-	0	0	0
	(Dir),A	0000110		0	0	0	A	ZERO	ADD	LIT	ALU	-	1	0	0
	(Dir),B	0000111		0	0	0	ZERO	B	ADD	LIT	ALU	-	1	0	0
	A,(B)	0001000		0	1	0	ZERO	DOUT	ADD	B	-	-	0	0	0
	B,(B)	0001001		0	0	1	ZERO	DOUT	ADD	B	-	-	0	0	0
	(B),A	0001010		0	1	0	A	ZERO	ADD	B	ALU	-	1	0	0
ADD	A,B	0001011		0	1	0	A	B	ADD	-	-	-	0	0	0
	B,A	0001100		0	0	1	A	B	ADD	-	-	-	0	0	0
	A,Lit	0001101		0	1	0	A	LIT	ADD	-	-	-	0	0	0
	A,(Dir)	0001110		0	1	0	A	DOUT	ADD	LIT	-	-	0	0	0
	A,(B)	0001111		0	1	0	A	DOUT	ADD	B	-	-	0	0	0
	(Dir)	0010000		0	0	0	A	B	ADD	LIT	ALU	-	1	0	0
SUB	A,B	0010001		0	1	0	A	B	SUB	-	-	-	0	0	0
	B,A	0010010		0	0	1	A	B	SUB	-	-	-	0	0	0
	A,Lit	0010011		0	1	0	A	LIT	SUB	-	-	-	0	0	0
	A,(Dir)	0010100		0	1	0	A	DOUT	SUB	LIT	-	-	0	0	0
	A,(B)	0010101		0	1	0	A	DOUT	SUB	B	-	-	0	0	0
	(Dir)	0010110		0	0	0	A	B	SUB	LIT	ALU	-	1	0	0
AND	A,B	0010111		0	1	0	A	B	AND	-	-	-	0	0	0
	B,A	0011000		0	0	1	A	B	AND	-	-	-	0	0	0
	A,Lit	0011001		0	1	0	A	LIT	AND	-	-	-	0	0	0
	A,(Dir)	0011010		0	1	0	A	DOUT	AND	LIT	-	-	0	0	0
	A,(B)	0011011		0	1	0	A	DOUT	AND	B	-	-	0	0	0
	(Dir)	0011100		0	0	0	A	B	AND	LIT	ALU	-	1	0	0
OR	A,B	0011101		0	1	0	A	B	OR	-	-	-	0	0	0
	B,A	0011110		0	0	1	A	B	OR	-	-	-	0	0	0
	A,Lit	0011111		0	1	0	A	LIT	OR	-	-	-	0	0	0
	A,(Dir)	0100000		0	1	0	A	DOUT	OR	LIT	-	-	0	0	0
	A,(B)	0100001		0	1	0	A	DOUT	OR	B	-	-	0	0	0
	(Dir)	0100010		0	0	0	A	B	OR	LIT	ALU	-	1	0	0
NOT	A,A	0100011		0	1	0	A	-	NOT	-	-	-	0	0	0
	B,A	0100100		0	0	1	A	-	NOT	-	-	-	0	0	0
	(Dir)	0100101		0	0	0	A	B	NOT	LIT	ALU	-	1	0	0
XOR	A,B	0100110		0	1	0	A	B	XOR	-	-	-	0	0	0
	B,A	0100111		0	0	1	A	B	XOR	-	-	-	0	0	0
	A,Lit	0101000		0	1	0	A	LIT	XOR	-	-	-	0	0	0
	A,(Dir)	0101001		0	1	0	A	DOUT	XOR	LIT	-	-	0	0	0
	A,(B)	0101010		0	1	0	A	DOUT	XOR	B	-	-	0	0	0
	(Dir)	0101011		0	0	0	A	B	XOR	LIT	ALU	-	1	0	0
SHL	A,A	0101100		0	1	0	A	-	SHL	-	-	-	0	0	0
	B,A	0101101		0	0	1	A	-	SHL	-	-	-	0	0	0
	(Dir)	0101110		0	0	0	A	B	SHL	LIT	ALU	-	1	0	0
SHR	A,A	0101111		0	1	0	A	-	SHR	-	-	-	0	0	0
	B,A	0110000		0	0	1	A	-	SHR	-	-	-	0	0	0
	(Dir)	0110001		0	0	0	A	B	SHR	LIT	ALU	-	1	0	0
INC	B	0110010		0	0	1	ONE	B	ADD	-	-	-	0	0	0
CMP	A,B	0110101		0	0	0	A	B	SUB	-	-	-	0	0	0
	A,Lit	0110110		0	0	0	A	LIT	SUB	-	-	-	0	0	0
JMP	Dir	0111001		1	0	0	-	-	-	-	-	LIT	0	0	0
JEQ	Dir	0111010	Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JNE	Dir	0111011	Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JGT	Dir	0111100	N=0 y Z=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLT	Dir	0111101	N=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JGE	Dir	0111110	N=0	1	0	0	-	-	-	-	-	LIT	0	0	0
JLE	Dir	0111111	N=1 o Z=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JCR	Dir	1000000	C=1	1	0	0	-	-	-	-	-	LIT	0	0	0
JOV	Dir	1000001	V=1	1	0	0	-	-	-	-	-	LIT	0	0	0
CALL	Dir	1000010		1	0	0	-	-	-	SP	PC	LIT	1	0	1
RET		1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000100		1	0	0	-	-	-	SP	-	DOUT	0	0	0
PUSH	A	1000101		0	0	0	A	ZERO	ADD	SP	ALU	-	1	0	1
	B	1000110		0	0	0	ZERO	B	ADD	SP	ALU	-	1	0	1
POP	A	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1000111		0	1	0	ZERO	DOUT	ADD	SP	-	-	0	0	0
POP	B	1000011		0	0	0	-	-	-	-	-	-	0	1	0
		1001000		0	0	1	ZERO	DOUT	ADD	SP	-	-	0	0	0
NOP		1001001		0	0	0	-	-	-	-	-	-	0	0	0

Tabla 1: ISA del computador básico.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2024)

Interrogación 2

Pauta de evaluación

Pregunta 1: Representación de Números Reales (6 ptos.)

(a) (1.5 ptos.) Suponga que posee dos representaciones para números reales que puede utilizar para almacenar el número 101,10011b. La primera corresponde a una representación de punto fijo con la siguiente estructura: 1 bit de signo; 3 bits de parte entera; y 4 bits de parte fraccional. La segunda, en contraste, corresponde a una representación de punto flotante **sin bit implícito** con la siguiente distribución: 1 bit de signo de significante; 4 bits de significante normalizado; 1 bit de signo de exponente; y 2 bits de exponente. ¿Cuál de estas representaciones recomendaría para almacenar el número? Justifique su respuesta en base a la precisión de la representación.

Solución: Para este número, es recomendable hacer uso de la representación de **punto fijo**. A través de ella, el número se almacena como: $0_{\text{Signo}}101_{\text{Entero}}1001_{\text{Signo}}$, donde se deduce que se pierde el bit menos significativo de la parte fraccional. En cambio, en el formato de punto flotante, su representación normalizada equivale a $1,0110011 \times 10^{10}$, por lo que se almacena la siguiente secuencia: $0_{\text{Signo sig.}}1011_{\text{Significante}}0_{\text{Signo exp.}}10_{\text{Exponente}}$. Se observa que en esta última representación se pierden los 4 bits menos significativos, por lo que su resultado es menos preciso. Se otorgan **0.75 ptos.** por indicar la representación que implica menor pérdida de precisión; y **0.75 ptos.** por justificar indicando la cantidad de bits perdidos en cada caso.

(b) (1.5 ptos.) Suponga que se quiere utilizar el computador básico visto en el curso para realizar operaciones sobre **números reales** bajo la representación de **punto fijo**. Para ello, los tres bits más significativos de los registros `A`, `B` se reservan para la parte entera; mientras que los cinco bits restantes se utilizan para la parte fraccional. Considerando que **no se realizan modificaciones de *hardware***, ¿existirá algún problema al momento de realizar operaciones con la ALU? Justifique su respuesta solo aludiendo a las operaciones aritméticas (`ADD`, `SUB`, `SHL`, `SHR`) ejecutadas en la arquitectura.

Solución: Si se opera con registros que almacenan números representados por punto fijo con **la misma cantidad de bits fraccionales** (en este caso, 5), entonces la suma y resta se pueden expresar como: $A \pm B = (A' \times 10^{-5} \pm B' \times 10^{-5}) = (A' \pm B') \times 10^{-5}$. Es decir, podemos computar de forma consistente la suma y la resta de los registros, aplicando solo el punto decimal al momento de representar el resultado. Por otra parte, las operaciones de *shifting* seguirán funcionando como la multiplicación o división por 2, solo que manteniendo una cantidad constante de puntos decimales. Por lo tanto, **no existe problema en operar con esta representación a nivel de *hardware***. Se otorgan **0.75 ptos.** por señalar correctamente que no existen problemas en términos aritméticos; y **0.75 ptos.** por justificación. En este último criterio, se aceptan ejemplos para mostrar que los resultados no varían en esta representación.

(c) (3 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar si la representación en formato float del resultado presenta cambios en el valor obtenido.

- (1.5 ptos.) $A + B$. A = 0x4A800000, B = 0x41700000.

Solución:

- A = 01001010100000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10010101b → 149 - 127 = 22.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $(1,0 \times 10^{10110})$b = 4194304.
- B = 01000001011100000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000010b → 130 - 127 = 3.
  - **Significante:** 11100000000000000000000b → 1,111b.
  - **Valor:** $(1,111 \times 10^{11})$b = 1111b = 15.

De lo anterior, se deduce que la operación a realizar es 4194304+15. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se igualan los exponentes y B se expresa como $0,00000000000000000111 \times 10^{10110}$. Luego, sumando los significantes, se tiene $A + B = 1,00000000000000000111 \times 10^{10110}$. Finalmente, si se almacena como `float`, se tiene el valor 01001010100000000000000000011110b = 0x4A80001E = 4194319. **No existe pérdida de precisión**.
Se otorgan **0.375 ptos.** por la desagregación e interpretación correcta de cada número; y **0.75 ptos.** por señalar correctamente el resultado de la representación en formato `float`.

- (1.5 ptos.) $A \times B$. A = 0x7F800000, B = 0x40000000.

Solución:

- A = 01111111100000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 11111111b → 255 - 127 = 128.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** + ínf.
- B = 01000000000000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000000b → 128 - 127 = 1.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $1,0 \times 10^1 = 10\text{b} = 2$.

De lo anterior, se deduce que la operación a realizar es igual a $+\infty \times 2$, lo que en formato float se traduce como $+\infty$ = 0x7F800000. Se otorgan **0.375 ptos.** por la desagregación e interpretación correcta de cada número; y **0.75 ptos.** por señalar correctamente el resultado de la representación en formato `float`.

Pregunta 2: Comunicación con dispositivos I/O (6 ptos.)

(a) (1 pto.) Explique el significado y uso del identificador que envía el controlador de interrupciones luego de recibir la señal INTA por parte de la CPU.

Solución: Al recibir la señal INTA, la CPU espera obtener el identificador asociado al **dispositivo cuya interrupción será atendida**. Este será enviado por el controlador de interrupciones. Luego, la CPU hace uso de este valor como índice del vector de interrupciones para obtener el puntero de la ISR del dispositivo atendido, permitiendo así la ejecución de su subrutina. Se otorgan **0.5 ptos.** por indicar la asociación entre el identificador y los dispositivos I/O; y **0.5 ptos.** por indicar su uso en la ejecución de la ISR deseada.

(b) (1 pto.) Indique cómo se lleva a cabo la transferencia de datos entre dispositivos I/O y la memoria RAM en arquitecturas que **carecen** de un controlador de DMA.

Solución: En este caso se lleva a cabo *Programmed* I/O; *i.e.* la transferencia manual de los datos hacia los registros de la CPU, los que luego son transferidos a destino (ya sea la RAM o dispositivos I/O de almacenamiento). Se otorga **1 pto.** si se describe correctamente del rol de la CPU en este tipo de interacción. No se otorga puntaje parcial.

(c) (4 ptos.) Para evitar que las plantas de interior de su hogar se sequen, usted implementa un sistema de regado automatizado conectado a un mini-computador con ISA RISC-V. A cada una de sus plantas le añade un sensor de humedad y un regador, cuyos registros de 32 bits son mapeados en memoria desde la dirección 0xABADBABE:

Offset	Nombre	Descripción
0x00	`humidity_sensor_cactus`	Sensor humedad cactus
0x04	`humidity_sensor_monstera`	Sensor humedad monstera
0x08	`irrigator_status_cactus`	Registro estado regador cactus
0x0C	`irrigator_status_monstera`	Registro estado regador monstera
0x10	`irrigator_command_cactus`	Registro comando regador cactus
0x14	`irrigator_command_monstera`	Registro comando regador monstera

Nombre	Descripción	Valor
`humidity_sensor_*`	Porcentaje humedad	0-100
`irrigator_status_*`	Apagado	0
`irrigator_status_*`	Encendido	1
`irrigator_status_*`	Riego suave	2
`irrigator_status_*`	Riego medio	4
`irrigator_status_*`	Riego alto	8
`irrigator_command_*`	Encender	1
`irrigator_command_*`	Apagar	255
`irrigator_command_*`	Detener riego	128
`irrigator_command_*`	Regar (por defecto suave)	2
`irrigator_command_*`	Incrementar riego	3
`irrigator_command_*`	Decrementar riego	4

Luego de varias pruebas, usted desarrolla el siguiente programa para llevar a cabo el riego:

```
irrigation_system:
  li t0, 0xABADBABE            # Carga en el registro t0 el valor del literal 0xABADBABE
  li s0, 15
  li s1, 20
  li s2, 0
  li s3, 1
  step_one:
    lw t1, 8(t0)
    beq t1, s2, step_two
    bne t1, s3, step_three
    beq zero, zero, step_four                # Salto incondicional a step_four
  step_two:
    li t2, 1
    sw t2, 16(t0)
    beq zero, zero, step_four                # Salto incondicional a step_four
  step_three:
    li t2, 128
    sw t2, 16(t0)
  step_four:
    li t2, 2
    sw t2, 16(t0)
    li t2, 3
    sw t2, 16(t0)
    while_one:
        lw t3, 0(t0)
        blt t3, s0, while_one
    li t2, 4
    sw t2, 16(t0)
    while_two:
        lw t3, 0(t0)
        blt t3, s1, while_two
    li t2, 128
    sw t2, 16(t0)
    li t2, 255
    sw t2, 16(t0)
  end:                         # Término del programa.
```

En base a este programa, conteste las siguientes preguntas:

1. (1 pto.) ¿Qué modo de comunicación existe entre el computador y el sistema de riego?

Solución: El modo de comunicación existente consiste en *polling*, dado que en el *label* `step_four` se evidencia la consulta constante sobre el valor del sensor de humedad para gestionar el riego de la planta. No obstante lo anterior, también se aceptan las siguientes respuestas:

- Comunicación en base a interrupciones, si se justifica que el código anterior consiste en la ISR ejecutada por el computador para llevar a cabo el riego de una planta.
- *Memory mapped* I/O. Si bien esto alude a la forma en la que se accede a los datos de los registros asociados a los dispositivos, dada la ambigüedad del enunciado se acepta también como respuesta **siempre** que se señale como argumento una de dos opciones: (1) la única forma de acceso a dispositivos a RISC-V; (2) el acceso a los registros de los dispositivos con instrucciones de lectura y escritura de memoria.

En cualquiera de los tres casos, se otorga **1 pto.** siempre y cuando **exista una justificación donde se apliquen correctamente los conceptos de I/O**. No se otorgará puntaje parcial.

2. (3 ptos.) Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por línea de código.

Solución:

- `step_one`: Se revisa el estado del regador del cactus. Si está apagado, se ejecuta el código del *label* `step_two`. Si está regando, independiente de la intensidad, se ejecuta el código del *label* `step_three`. En cualquier otro caso (*i.e.* está encendido), se ejecuta el código del *label* `step_four`. Se otorgan **0.5 ptos.** si se describe correctamente; **0.25 ptos.** si existe como máximo un error; y no se otorga puntaje si existen dos o más errores.

- `step_two`: Se enciende el regador del cactus y se procede a ejecutar el código del *label* `step_four`. Se otorgan **0.5 ptos.** si se describe correctamente; **0.25 ptos.** si existe como máximo un error; y no se otorga puntaje si existen dos o más errores.

- `step_three`: Se detiene el riego del regador del cactus y se procede a ejecutar el código del *label* `step_four`. Se otorgan **0.5 ptos.** si se describe correctamente; **0.25 ptos.** si existe como máximo un error; y no se otorga puntaje si existen dos o más errores. En este caso, no es necesario explicitar la ejecución de `step_four`.

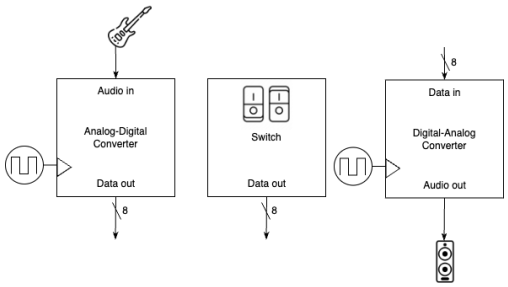
- `step_four`: Se activa el riego del regador del cactus y se incrementa para que realice riego medio. Luego, en `while_one` se consulta el valor del sensor de humedad del cactus constantemente hasta que sea mayor o igual a un 15 %. Posteriormente, se decrementa el nivel de riego para que tenga una intensidad suave. Finalmente, en `while_two` se consulta el valor del sensor de humedad del cactus hasta que sea mayor o igual a un 20 %, momento en el que se detiene el regador y se apaga. Se otorgan **1.5 ptos.** si se describe correctamente; y se descuentan **0.5 ptos.** por cada detalle faltante en la descripción.

Importante: Se considera como error en cada *step* si se toma en cuenta la monstera: en el código nunca se accede a los dispositivos conectados a ella.

Pregunta 3: Computador básico e I/O (6 ptos.)

Dado su interés por la música y la tecnología, ha empezado a explorar el funcionamiento de los pedales de efecto haciendo uso del computador básico para modificar las señales de sonido emitidas por una guitarra. Para esto, habilita la conexión con los siguientes dispositivos *port-mapped*:

Dispositivo	Tipo	Puerto
Conversor análogo-digital	Entrada	0
Switch de efecto	Entrada	1
Conversor digital-análogo	Salida	2



El conversor análogo-digital envía la digitalización de la señal generada por la guitarra; mientras que el conversor digital-análogo recibe la señal digital con efectos aplicados por el computador, los que son finalmente transmitidos a un amplificador. Adicionalmente, añade un *switch* con señal de salida de 8 bits, pero solo con dos valores posibles: 0 si está apagado y 1 si está prendido. El efecto del pedal se aplica solo si el *switch* está encendido. Para lograr este comportamiento, debe modificar la microarquitectura del computador básico implementando las siguientes instrucciones:

Instrucción	Operandos	Operación
`IN`	`A,Lit`	`A` = Dispositivo asociado a puerto `Lit`
	`B,Lit`	`B` = Dispositivo asociado a puerto `Lit`
`OUT`	`Lit,A`	Dispositivo asociado a puerto `Lit` = `A`
	`Lit,B`	Dispositivo asociado a puerto `Lit` = `B`

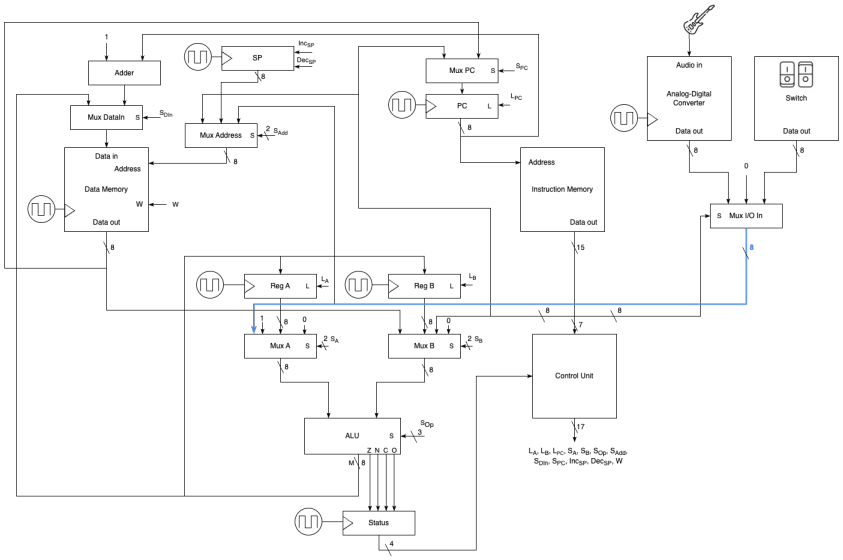
Para cada instrucción nueva, deberá incluir la combinación **completa** de señales que la ejecutan. Por cada señal de carga/escritura/incremento/decremento, deberá indicar si se activan (1) o no (0); en las señales de selección, deberá indicar el **nombre** de la entrada escogida ("-" si no afecta). Puede realizar todas las modificaciones en un solo diagrama. Para el conversor digital-análogo, asuma que si no se le envían datos mediante `OUT`, debe recibir un 0 por defecto.

(a) (2.5 ptos.) `IN Reg, Lit`. Guarda en `Reg` (`A`/`B`) el valor de salida del puerto `Lit`.

Solución: A continuación, se describe una posible implementación:

- Se agrega el multiplexor "`Mux I/O In`" con una señal de selección de 8 bits para escoger uno de varios dispositivos I/O. Si bien solo se conectarán dos, es necesario considerar que el puerto de 8 bits requiere funcionar con valores exactos. Para los puertos no definidos, se selecciona una señal constante igual a cero. Para la selección, se utiliza el literal proveniente de la memoria de instrucciones.
- Se conecta la salida del multiplexor anterior con la entrada libre del multiplexor "`Mux A`", lo que permite almacenar el valor de los dispositivos en cualquiera de los registros.

El siguiente diagrama muestra la implementación anterior:



Para ejecutar las instrucciones, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`IN A, Lit`	1	0	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-
`IN B, Lit`	0	1	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-

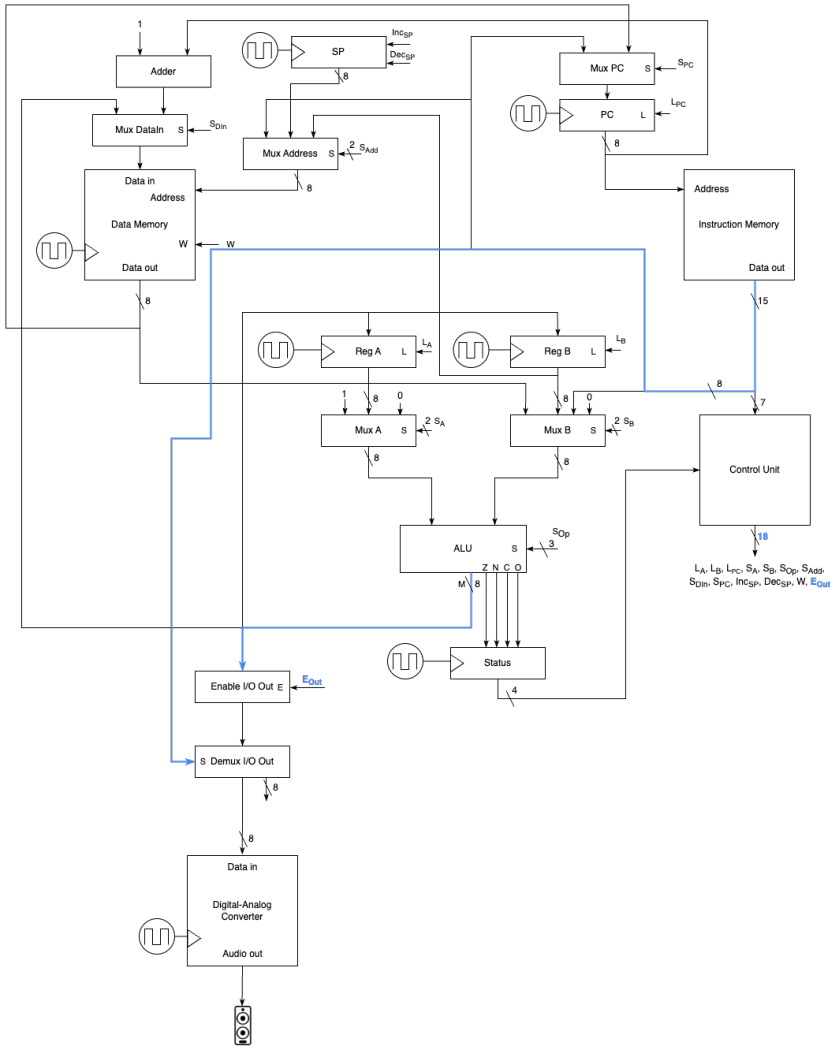
Se otorgan **1.25 ptos.** por la correctitud de la modificación y **1.25 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

(b) (2.5 ptos.) `OUT Lit, Reg`. Entrega como entrada al puerto `Lit` el valor de `Reg` (`A`/`B`).

Solución: A continuación, se describe una posible implementación:

- Se agrega el demultiplexor "`Demux I/O Out`" con una señal de selección de 8 bits para enviar el valor de entrada a uno de varios dispositivos I/O. Si bien solo habrá un dispositivo disponible para el envío, es necesario considerar que el puerto de 8 bits requiere funcionar con valores exactos. Para los puertos no definidos, simplemente se envía la señal a cables no conectados a otros dispositivos.
- La salida de la ALU se conecta con un *enabler* "`Enable I/O Out`", con señal habilitadora E_{Out} solo activa para las instrucciones de tipo `OUT`. Finalmente, su salida se conecta con la entrada del demultiplexor anterior para el envío correcto del valor de los registros `A` o `B`.

El siguiente diagrama muestra la implementación anterior:

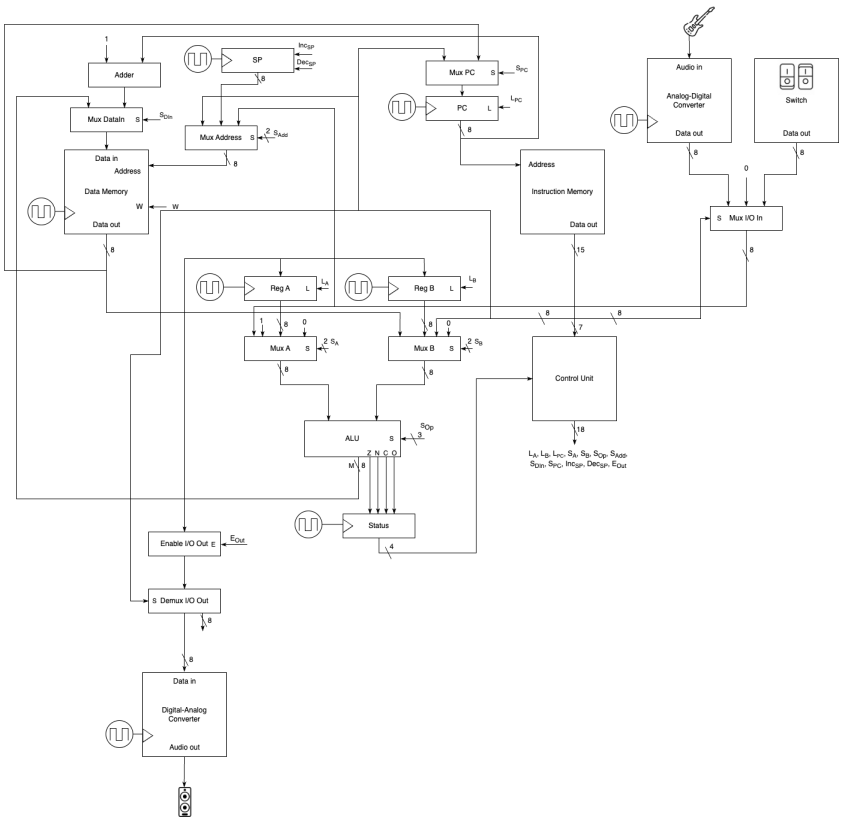


Para ejecutar las instrucciones, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	E_{Out}
`OUT Lit, A`	0	0	0	0	0	0	A	ZERO	ADD	-	-	-	1
`OUT Lit, B`	0	0	0	0	0	0	ZERO	B	ADD	-	-	-	1

Se otorgan **1.25 ptos.** por la correctitud de la modificación y **1.25 ptos.** por entregar una combinación de señales correcta para la instrucción. En caso de existir como máximo un error de implementación en uno de los criterios antes descritos, se otorga la mitad del puntaje. Si existe más de un error, no se otorga puntaje.

A modo de completitud, se comparte a continuación el diagrama completo con todas las instrucciones solicitadas.



Para ejecutar las instrucciones, se utiliza la siguiente combinación de señales:

Instrucción	L_A	L_B	L_{PC}	W	Inc_{SP}	Dec_{SP}	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	E_{Out}
`IN A, Lit`	1	0	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-	0
`IN B, Lit`	0	1	0	0	0	0	`I/O`	`ZERO`	`ADD`	-	-	-	0
`OUT Lit, A`	0	0	0	0	0	0	`A`	`ZERO`	`ADD`	-	-	-	1
`OUT Lit, B`	0	0	0	0	0	0	`ZERO`	`B`	`ADD`	-	-	-	1

(c) (1 pto.) Asuma que se implementan las instrucciones anteriores. Elabore un fragmento de código Assembly que aplique el efecto del pedal a través del llamado a una subrutina `apply_effect`. Esta asumirá que la señal de sonido se encontrará almacenada en `B` y modificará su valor directamente. El programa debe, entonces:

1. Obtener el valor de la señal del conversor análogo-digital y el estado del *switch*.
2. Aplicar el efecto a partir del llamado a `apply_effect` **solo** si el *switch* está activado.
3. Enviar al conversor digital-análogo la señal resultante.

Debe hacer uso de las instrucciones `IN` y `OUT` para cumplir lo anterior. Puede agregar instrucciones adicionales ya existentes en la ISA del computador básico para completar el programa. **Solo debe llamar a `apply_effect`**, no es necesario que defina su código.

Solución: A continuación, se muestra un fragmento de código que cumple lo solicitado.

```
CODE:
  ; Paso 1: Obtención de la señal del conversor ADC y el switch.
  IN A,1
  IN B,0
  ; Paso 2: Aplicación del efecto solo si el switch está activo.
  CMP A,1
  JNE end
  CALL apply_effect
  ; Paso 3: Envío de la señal resultante a través de B, independiente
  ; de la aplicación del efecto.
  end:
    OUT 2,B
```

Se otorgan **0.25 ptos.** por obtener correctamente el valor de las señales de los dispositivos de entrada; **0.5 ptos.** por aplicar correctamente el efecto según el valor del *switch*; y **0.25 ptos.** por enviar correctamente el resultado al conversor digital-análogo.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2024)

Interrogación 2

Pauta de evaluación

Pregunta 1: Representación de Números Reales (6 ptos.)

(a) (1.5 ptos.) Suponga que posee dos representaciones para números reales que puede utilizar para almacenar el número 0,1101101b. La primera corresponde a una representación de punto fijo con la siguiente estructura: 1 bit de signo; 3 bits de parte entera; y 4 bits de parte fraccional. La segunda, en contraste, corresponde a una representación de punto flotante con la siguiente distribución: 1 bit de signo de significante; 5 bits de significante; 1 bit de signo de exponente; y 1 bit de exponente. ¿Cuál de estas representaciones presenta una menor pérdida de precisión para almacenar el número? Justifique su respuesta.

Nota: Durante la evaluación, se aclara que en punto flotante se usa la representación con bit explícito, no implícito.

Solución: Para este número, es recomendable hacer uso de la representación de **punto flotante**. Su representación normalizada equivale a $1,101101 \times 10^{-1}$, por lo que **con bit explícito** se almacena la siguiente secuencia: $0_{\text{Signo sig.}}11011_{\text{Significante}}1_{\text{Signo exp.}}1_{\text{Exp.}}$. De este resultado, se deduce que se pierden los dos bits menos significativos, equivalente a una pérdida de precisión de 2^{-7}. En cambio, en el formato de punto fijo, se almacena la siguiente secuencia: $0_{\text{Signo}}000_{\text{Entero}}1101_{\text{Fracción}}$. Se observa que en esta última representación se pierden los 3 bits menos significativos, por lo que su resultado es menos preciso al tener una pérdida de precisión de $2^{-5} + 2^{-7}$. Se otorgan **0.75 ptos.** por indicar la representación que implica menor pérdida de precisión; y **0.75 ptos.** por justificar indicando la cantidad de valor o bits perdidos en cada caso.

(b) (1.5 ptos.) Indique el valor que representa el float 0x7FC00010. Justifique su respuesta en base al estándar IEEE754.

Solución: Como el exponente desfasado del número es 255 (*i.e.* todos sus bits iguales a 1), que se reserva para ciertas representaciones, su valor puede ser $\pm\infty$ o `NaN` (*Not a Number*). Luego, su significante posee al menos dos bits distintos de cero, por lo que su valor equivale a `NaN`. Se otorgan **0.75 ptos.** por señalar correctamente el valor que representa el float; y **0.75 ptos.** por justificación en función de su exponente y significante.

(c) (3 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar tanto el valor teórico (resultado de la operación) como el valor real (almacenado como float del estándar IEEE754) de la operación. Si son iguales, explicite que lo son. Los valores deben presentarse en base binaria con notación científica normalizada.

- (1.5 ptos.) $A + B$. $A =$ 0x40400000, $B =$ 0x3F800000.

Solución:

- $A =$0100000001000000 00000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 10000000b $\rightarrow$ 128 - 127 = 1.
  - **Significante:** 10000000000000000000000b $\rightarrow$ 1,1b.
  - **Valor:** $(1,1 \times 10^1)$b = 11b = 3.
- $B =$ 00111111100000000000000000000000.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 01111111b $\rightarrow$ 127 - 127 = 0.
  - **Significante:** 00000000000000000000000b $\rightarrow$ 1,0b.
  - **Valor:** $(1,0 \times 10^0)$b = 1b = 1.

De lo anterior, se deduce que la operación a realizar es 3 + 1. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se igualan los exponentes y B se expresa como $0,1 \times 10^1$. Luego, sumando los significantes, se tiene $A + B = (10,0 \times 10^1)$b = $(1,0 \times 10^{10})$. Finalmente, si se almacena como `float`, se tiene el valor 01000000100000000000000000000000b = 0x40800000 = 4. **No existe pérdida de precisión**.

Se otorgan **0.375 ptos.** por la interpretación correcta de cada número; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

- (1.5 ptos.) $A \times B$. A = 0x41820000, B = 0x3F200000.

Solución:

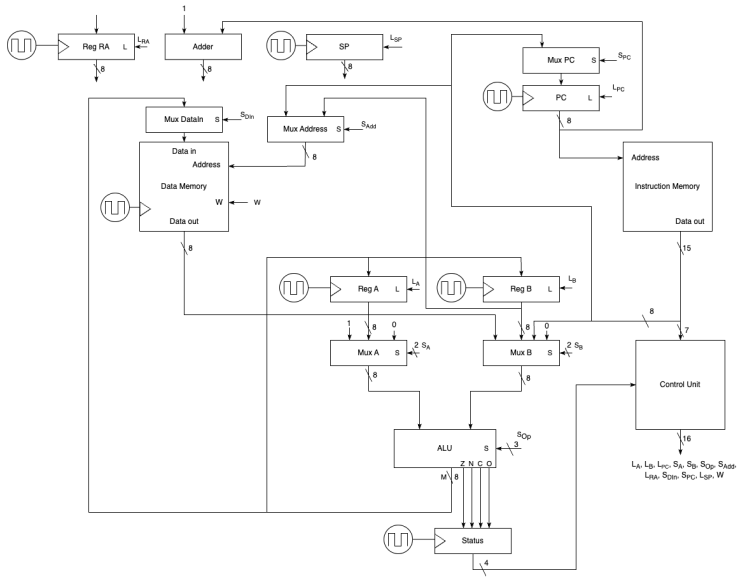
- A = 01000001100000100000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000011b → 131 - 127 = 4.
  - **Significante:** 00000100000000000000000b → 1,000001b.
  - **Valor:** $(1,000001 \times 10^{100})$b = 10000,01b = 16,25.
- B = 00111111001000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 01111110b → 126 - 127 = -1.
  - **Significante:** 01000000000000000000000b → 1,01b.
  - **Valor:** $1,01 \times 10^{-1} = 0,101$b = 0,625.

De lo anterior, se deduce que la operación a realizar es igual a $16,25 \times 0,625$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $-1 + 4 = 3 = 11$b. Luego, se multiplican los significantes: $1,01 \times 1,000001 = (101 \times 10^{-2}) \times (1000001 \times 10^{-6}) = (101 \times 1000001) \times 10^{-8} = 101000101 \times 10^{-8} = 1,01000101$. Finalmente, se tiene como resultado $1,01000101 \times 10^{11}$b = 1010,00101 = 10,15625, que en formato float se almacena como 01000001001000101000000000000000b = 0x41228000. **No existe pérdida de precisión**.

Se otorgan **0.375 ptos.** por la interpretación correcta de cada número; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 2: Computador básico con subrutinas (6 ptos.)

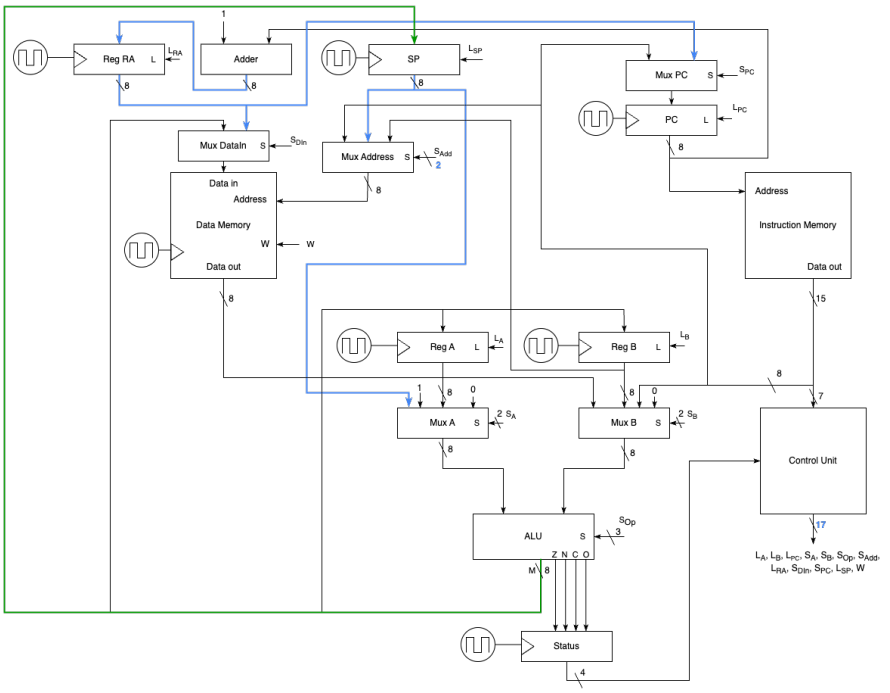
En arquitecturas RISC, las direcciones de retorno de una subrutina se almacenan en un **registro de enlace** (`RA`), a diferencia del computador básico donde se guardan en el *stack*. Asimismo, estas arquitecturas no poseen `PUSH/POP`, sino que acceden al *stack* directamente con el *stack pointer* (`SP`). Deberá contestar las preguntas de esta sección a partir de este contexto y el siguiente diagrama:



(a) **(5 ptos.)** A partir del diagrama de base adjunto, y asumiendo que se eliminan las instrucciones `CALL`, `RET`, `PUSH` y `POP`, realice las modificaciones de *hardware* necesarias e indique la combinación de señales completa para ejecutar las siguientes instrucciones en un ciclo:

- **(0.5 ptos.)** `MOV A,(SP)`. Guarda en `A` el valor `Mem[SP]`.
- **(0.5 ptos.)** `MOV (SP), A`. Guarda en `Mem[SP]` el valor `A`.
- **(1 pto.)** `ADD SP,Lit`. Guarda en `SP` el valor `SP + Lit`. `Lit` puede ser negativo.
- **(1 pto.)** `MOV (SP), RA`. Guarda en `Mem[SP]` el valor `RA`.
- **(1 pto.)** `JAL RA, label`. Guarda `PC+1` en `RA` y salta a la dirección asociada a `label`.
- **(1 pto.)** `JALR RA`. Salta a la dirección almacenada en `RA`.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- La conexión entre `SP` y `Mux Address` permite el uso de `SP` como dirección de memoria con la señal S_{Add}, la que ahora debe ser de 2 bits.
- La conexión entre `SP` y `Mux A` permite su uso como primer operando en la `ALU`.
- La conexión entre la salida de la `ALU` y `SP` permite cargar en este último los resultados de las operaciones con la señal L_{SP}.
- La conexión entre `RA` y `Mux DataIn` permite la escritura de `RA` en memoria.
- La conexión entre `RA` y `PC+1` permite escribir este valor en `RA` con la señal L_{RA}.
- La conexión entre `RA` y `Mux PC` permite realizar saltos hacia direcciones almacenadas en `RA`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA

Por cada instrucción, se otorga **la mitad** del puntaje por *hardware* y **la otra mitad** por la combinación de señales. En cada criterio, se descuenta la mitad del puntaje en caso de existir, como máximo, un error de implementación o valor en la combinación de señales. En otro caso, no se otorga puntaje.

(b) **(1 pto.)** Modifique el siguiente fragmento de código para que haga uso exclusivo de las instrucciones anteriores. Debe mantener su funcionamiento original y terminar sin errores.

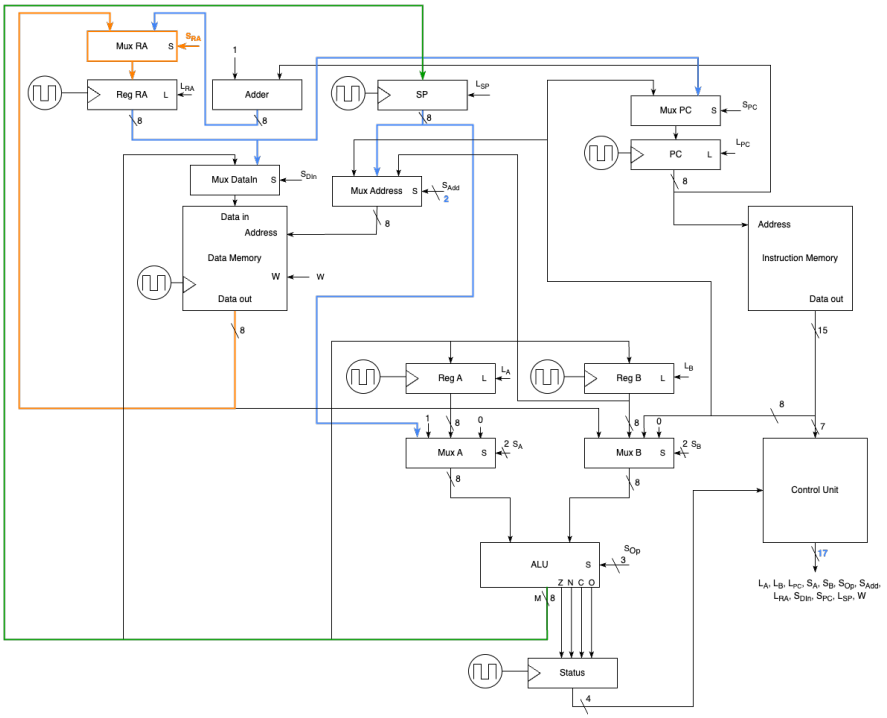
```
MOV A,1
PUSH A
CALL func
POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  CALL func
  end_func:
    RET
end:
```

Solución: Es importante señalar que, con las instrucciones entregadas, **no es posible** entregar un fragmento de código que mantenga el funcionamiento original **sin errores**. Esto se debe al hecho de que no existe una instrucción que permita recuperar una dirección de retorno respaldada en memoria (por ejemplo, `MOV RA, (SP)`. Por este motivo, se consideran correctas respuestas que solo caigan en ese caso. A continuación, una solución posible.

```
MOV A,1
MOV (SP),A        ; Reemplazo por PUSH A
ADD SP,-1         ; Reemplazo por PUSH A
JAL RA,func       ; Reemplazo por CALL func
ADD SP,1          ; Reemplazo por POP A
MOV A,(SP)        ; Reemplazo por POP A
JMP end
func:
  SHL A,A
  CMP A,8
  JEQ end_func
  MOV (SP),RA     ; Respaldo de RA para evitar error en llamado recursivo
  ADD SP,-1       ; Respaldo de RA para evitar error en llamado recursivo
  JAL RA,func     ; Reemplazo por CALL func
  ADD SP,1        ; De todas formas, se restituye el valor de SP
  MOV RA,(SP)     ; Si existiera, podríamos recuperar el valor respaldado de RA
  end_func:
    JALR RA       ; Reemplazo por RET
end:
```

Se otorgan **0.25 ptos.** por reemplazar correctamente la instrucción `PUSH A`; **0.25 ptos.** por reemplazar correctamente la instrucción `CALL label`; **0.25 ptos.** por reemplazar correctamente la instrucción `POP A`; y **0.25 ptos.** por reemplazar correctamente la instrucción `RET`. Adicionalmente, si la respuesta alude a la falta de una instrucción para recuperar la dirección de retorno; o bien se usa de forma hipotética una instrucción que permita resolver el problema, **se otorgarán 0.5 ptos. de bonus al total**.

A modo de completitud, se incluye tanto el diagrama como la tabla de señales de la arquitectura que habilita la ejecución de la instrucción `MOV RA,(SP)`.



Instrucción	L_A	L_B	L_{PC}	L_{SP}	L_{RA}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	S_{RA}
`MOV A, (SP)`	1	0	0	0	0	0	ZERO	DOUT	ADD	SP	-	-	-
`MOV (SP), A`	0	0	0	0	0	1	A	ZERO	ADD	SP	ALU	-	-
`ADD SP, Lit`	0	0	0	1	0	0	SP	LIT	ADD	-	-	-	-
`MOV (SP), RA`	0	0	0	0	0	1	-	-	-	SP	RA	-	-
`MOV RA, (SP)`	0	0	0	0	1	0	-	-	-	SP	-	-	DOUT
`JAL RA, label`	0	0	1	0	1	0	-	-	-	-	-	LIT	PC+1
`JALR RA`	0	0	1	0	0	0	-	-	-	-	-	RA	-

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

(a) (1 pto.) Si un computador ya tiene dispositivos *memory mapped* a través de un *address decoder*, ¿se necesita *hardware* adicional para habilitar la comunicación vía *polling*? Justifique.

> **Solución: No**, no es necesario ya que en la comunicación vía *polling* es la CPU la que se encarga de interactuar con los dispositivos a través de la revisión de sus estados, lo que ya puede realizar con el acceso directo a las direcciones de memoria asociadas a ellos. Se otorgan **0.5 ptos.** por responder correctamente; y **0.5 ptos.** por justificación.

(b) (1 pto.) ¿Cuál es la función del vector de interrupciones? ¿Cuál es su contenido?

> **Solución:** La función del vector de interrupciones es permitir la ejecución correcta de la ISR (*Interruption Service Routine*) a partir del identificador del dispositivo a ser atendido, lo que es gestionado por el controlador de interrupciones. Este vector contiene los **punteros** de las ISR, por lo que si se desea atender al dispositivo `i`, se ejecuta el llamado de la subrutina a partir de su dirección igual a `vector_interrupciones[i]`. Se otorgan **0.5 ptos.** por señalar correctamente la función del vector de interrupciones; y **0.5 ptos.** por indicar de forma correcta su contenido.

(c) (4 ptos.) Suponga que decide, con el propósito de ahorrar tiempo, automatizar la preparación de café filtrado a partir de un molinillo y hervidor eléctricos conectados a un computador con ISA RISC-V. Para la comunicación con los electrodomésticos, sus registros de 32 bits son mapeados en memoria desde la dirección 0x600DCAFE:

Offset	**Nombre**	**Descripción**
0x00	`coffee_grinder_status`	Registro estado molinillo
0x04	`coffee_grinder_command`	Registro comando molinillo
0x08	`coffee_grinder_grind_size`	Config. grosor molienda
0x0C	`coffee_grinder_weight`	Config. gramos a moler
0x10	`kettle_status`	Registro estado hervidor
0x14	`kettle_command`	Registro comando hervidor
0x18	`kettle_temperature`	Config. temperatura hervidor (ºC)

Nombre	**Descripción**	**Valor**
`coffee_grinder_status`	Apagado	0
`coffee_grinder_status`	Encendido	1
`coffee_grinder_status`	Moliendo	2
`kettle_status`	Apagado	0
`kettle_status`	Encendido	1
`kettle_status`	Hirviendo	2
`coffee_grinder_command`	Encender	1
`coffee_grinder_command`	Apagar	255
`coffee_grinder_command`	Moler	128
`kettle_command`	Encender	1
`kettle_command`	Apagar	255
`kettle_command`	Hervir	128
`coffee_grinder_grind_size`	Fino	1
`coffee_grinder_grind_size`	Medio	2
`coffee_grinder_grind_size`	Grueso	3

Pensando en su preparación favorita, desarrolla el siguiente programa:

```
coffee_system:
# Carga 0 + 0x600DCAFE en t0
addi t0, zero, 0x600DCAFE
addi t2, zero, 0
addi t3, zero, 0
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 128
addi s4, zero, 255
addi s5, zero, 30
addi s6, zero, 92

step_one:
    lw t1, 0(t0) # Carga Mem[t0 + 0] en t1
    # Salta a continue_step_one si t1 != s2
    bne t1, s2, continue_step_one
    jal ra, step_five # Llamado a step_five
    beq zero, zero, step_three
    continue_step_one:
        beq t1, s1, step_two
        sw s1, 4(t0) # Carga s1 en Mem[t0 + 4]
step_two:
    sw s2, 8(t0)
    sw s5, 12(t0)
    sw s3, 4(t0)
    jal ra, step_five
# Continúa en step_three -->
```

```
step_three:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_three si t1 != s2
    bne t1, s2, continue_step_three
    jal ra, step_six
    beq zero, zero, end # Salto incondicional
    continue_step_three:
        beq t1, s1, step_four
        sw s1, 20(t0)
step_four:
    sw s6, 24(t0)
    sw s3, 20(t0)
    jal ra, step_six
    beq zero, zero, end
step_five:
    while_step_five:
        lw t1, 0(t0)
        beq t1, s2, while_step_five
    sw s4, 4(t0)
    jalr zero, 0(ra) # Retorno
step_six:
    while_step_six:
        lw t1, 16(t0)
        beq t1, s2, while_step_six
    sw s4, 20(t0)
    jalr zero, 0(ra)
end: # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(0.7 ptos.)**: Se revisa el estado del molinillo. Si está moliendo, se ejecuta `step_five` y se salta a `step_three`. Si está encendida, se salta a `step_two`. En otro caso, se enciende antes de continuar con `step_two`.
- `step_two` **(0.7 ptos.)**: Se configura el molinillo con una molienda de grosor medio y un total de 30 gramos. Luego, empieza a moler y se ejecuta `step_five` antes de seguir a `step_three`.
- `step_three` **(0.7 ptos.)**: Se revisa el estado del hervidor. Si está hirviendo, se ejecuta `step_six` y se termina el programa. Si está encendida, se salta a `step_four`. En otro caso, se enciende antes de continuar con `step_four`.
- `step_four` **(0.7 ptos.)**: Se configura el hervidor con una temperatura de 92ºC. Luego, empieza a hervir y se ejecuta `step_six` antes de terminar el programa.
- `step_five` **(0.6 ptos.)**: Se revisa el estado del molinillo hasta que deje de moler.
- `step_six` **(0.6 ptos.)**: Se revisa el estado del hervidor hasta que deje de hervir.

Para cada `step`, se otorga la mitad del puntaje si existe como máximo un error o detalle faltante. Si existen dos o más errores o detalles faltantes, no se otorga puntaje.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2025)

Interrogación 2

Pauta de evaluación

Pregunta 1: Representación de Números Reales (6 ptos.)

(a) (2 ptos.) Compare la representación de punto fijo con la representación de punto flotante en términos de rango y precisión. ¿Cuál es más conveniente en cada caso y por qué?

Solución: La representación de punto fijo es más conveniente si lo que se busca es **precisión**, dado que sus valores se distribuyen de manera uniforme en su rango gracias a la porción de bits de parte fraccional. No obstante, tiene un rango muy acotado que depende de los bits restantes para representar la parte entera del número. La representación de punto flotante, en cambio, es más conveniente si lo que se busca es un **rango mayor**, ya que el uso de bits para representar el exponente permite "flotar" el punto y representar valores mucho más grandes o pequeños. Hay que considerar, de todas formas, que mientras más grande o pequeño sea el valor a representar, su precisión disminuye significativamente al depender de los bits restantes para el significante. Se otorga **1 pto.** por describir correctamente en qué caso se prefiere una representación de punto fijo, y **1 pto.** por describir correctamente en qué caso se prefiere una representación de punto flotante.

(b) (1 pto.) Suponga que imprime en la consola de su computador un valor que se encuentra almacenado en formato float del estándar IEEE754. Al imprimir, en la consola ve que aparece el valor "`NaN`". ¿Es posible, a partir de este resultado, obtener la secuencia de 32 bits exacta que representa este valor? Justifique.

Solución: No es posible. La representación del valor `NaN` del formato float tiene las siguientes características:

- Su exponente es igual a 11111111.
- **Al menos uno de sus bits de significante es distinto de cero.**

El último punto es relevante. Dado que cualquiera de los bits del significante es distinto de cero, al leer en consola el valor "`NaN`" no tenemos cómo saber cuál de los 23 bits es igual a 1. Se otorgan **0.5 ptos.** por responder correctamente, y **0.5 ptos.** por justificación correcta. Si solo responde que no es posible sin ninguna justificación, no se otorga puntaje.

(c) (3 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (1.5 ptos.) $A + B$. $A =$ 0x40400000, $B =$ 0x80000000.

Solución:

- $A = 0$ 10000000 10000000000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 10000000b $\rightarrow$ 128 - 127 = 1.
  - **Significante:** 10000000000000000000000b $\rightarrow$ 1,1b.
  - **Valor:** $(1,1 \times 10^{1})$b = 11b = 3.
- $B = 1$ 00000000 00000000000000000000000b.
  - **Signo:** 1b $\rightarrow$ negativo.
  - **Exponente:** 00000000b $\rightarrow$ **reservado para el cero**.
  - **Significante:** 00000000000000000000000b $\rightarrow$ 1,0b.
  - **Valor:** -0 (valor reservado).

De lo anterior, se deduce que la operación a realizar es $3 + (-0)$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a 3** y, por ende, no hay pérdida de precisión al almacenar el valor de vuelta como float.

Se otorgan **0.375 ptos.** por la interpretación correcta de A; **0.375 ptos.** por la interpretación correcta de B; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Además, en este caso se acepta no realizar la operación si se deduce que la suma es por 0, manteniendo el valor final de A.

- (1.5 ptos.) $A \times B$. A = 0x40000000, B = 0xC0A00000.

Solución:

- A = 0 10000000 00000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000000b → 128 - 127 = 1.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $(1,0 \times 10^1)$b = 10b = 2.
- B = 1 10000001 01000000000000000000000b.
  - **Signo:** 1b → negativo.
  - **Exponente:** 10000001b → 129 - 127 = 2.
  - **Significante:** 01000000000000000000000b → 1,01b.
  - **Valor:** $-(1,01 \times 10^2)$b = $-$101b = -5.

De lo anterior, se deduce que la operación a realizar es 2×-5. En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $1 + 2 = 3 = 11$b. Luego, se multiplican los significantes: $1,0 \times -1,01 = -1,01$. Finalmente, se tiene como resultado $-1,01 \times 10^{11}$b = $-$1010b = $-$10, que en formato float se almacena como 11000001001000000000000000000000b = 0xC1200000. **No existe pérdida de precisión**.

Se otorgan **0.375 ptos.** por la interpretación correcta de A; **0.375 ptos.** por la interpretación correcta de B; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 2: Computador básico con subrutinas (6 ptos.)

En los ítems **(a)** y **(b)**, se le describirán instrucciones que deberá implementar en el computador básico con subrutinas a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida ("-" si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. **Cada instrucción debe ejecutarse en un ciclo**.

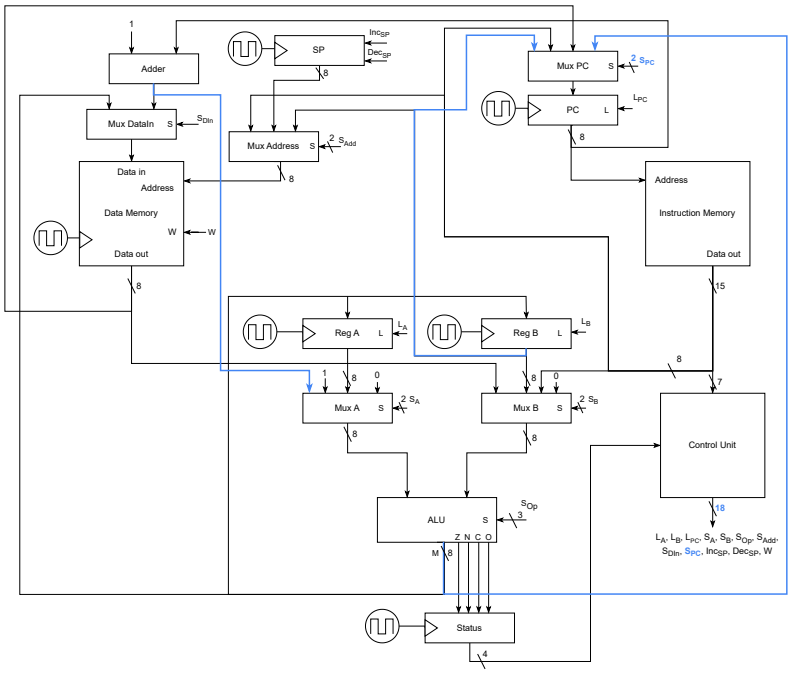
(a) (0.75 ptos.) `CALL A,B`. Almacena en el registro `A` la dirección de retorno `PC+1` y salta a la dirección de memoria almacenada en el registro `B`.

(b) (0.75 ptos.) `CALL B,Dir`. Almacena en el registro `B` la dirección de retorno `PC+1` y salta a la dirección de memoria equivalente al literal `Dir`.

(c) (0.75 ptos.) `RET A`. Salta a la dirección de retorno almacenada en el registro `A`.

(d) (0.75 ptos.) `RET B`. Salta a la dirección de retorno almacenada en el registro `B`.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se conecta la salida del *Adder* que computa `PC+1` con la entrada restante del componente `Mux A`, de forma que se pueda tener este valor como salida de la ALU para su almacenamiento en los registros `A` o `B`.
- Se conecta la salida de la ALU con una de las entradas del componente `Mux PC`, de forma que se pueda seleccionar el valor del registro `A` o del registro `B` como direcciones de memoria a saltar.
- Se conecta la salida del registro `B` con una de las entradas del componente `Mux PC`, de forma que se pueda seleccionar el valor de este como dirección de memoria a saltar mientras que al mismo tiempo se compute `PC+1` desde la ALU.
- Finalmente, incrementa a 2 bits la señal S_{PC} para poder soportar las dos nuevas entradas del multiplexor `Mux PC`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	Inc_{SP}	Dec_{SP}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
CALL A,B	1	0	1	0	0	0	PC+1	ZERO	ADD	-	-	B
CALL B,Dir	0	1	1	0	0	0	PC+1	ZERO	ADD	-	-	LIT
RET A	0	0	1	0	0	0	A	ZERO	ADD	-	-	ALU
RET B	0	0	1	0	0	0	ZERO	B	ADD	-	-	ALU

Por cada instrucción, se otorgan:

- **0.375 ptos.** por *hardware* correctamente modificado; **0.25 ptos.** si existe **un solo error** de implementación; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso.
- **0.375 ptos.** por entregar una combinación de señales correcta; **0.25 ptos.** si existe **un solo error**; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso. Si una instrucción se define en más de un ciclo, tampoco se otorga puntaje en este criterio. Si la instrucción se implementa correctamente sin modificación de *hardware*, se otorga el puntaje completo.

(e) (3 ptos.) A continuación, se adjuntan dos fragmentos de código **(1)** y **(2)** escritos en el Assembly del computador básico, con la diferencia de que **(2)** incorpora las instrucciones implementadas en los ítems anteriores. Para cada fragmento, indique los valores de los registros `A` y `B` al finalizar la ejecución del código. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, **no se otorgará puntaje**.

(1)

```
DATA:
  var1 12
  var2 4
CODE:
  MOV A,5     ; Dir. memoria 0x00
  MOV B,3     ; Dir. memoria 0x01
  RET         ; Dir. memoria 0x02-0x03
  case_1:
    ADD A,B   ; Dir. memoria 0x04
    PUSH A    ; Dir. memoria 0x05
    MOV A,B   ; Dir. memoria 0x06
    SHL A,A   ; Dir. memoria 0x07
    MOV B,A   ; Dir. memoria 0x08
    POP A     ; Dir. memoria 0x09-0x0A
    JMP end   ; Dir. memoria 0x0B
  case_2:
    SUB A,B   ; Dir. memoria 0x0C
    PUSH A    ; Dir. memoria 0x0D
    MOV A,B   ; Dir. memoria 0x0E
    SHR A,A   ; Dir. memoria 0x0F
    MOV B,A   ; Dir. memoria 0x10
    POP A     ; Dir. memoria 0x11-0x12
  end:
```

(2)

```
DATA:
  n    2
  res  0
CODE:
  MOV A,(n)       ; Dir. memoria 0x00
  CALL B,fun      ; Dir. memoria 0x01
  JMP end         ; Dir. memoria 0x02
  fun:
    CMP A,0       ; Dir. memoria 0x03
    JLE fun_end   ; Dir. memoria 0x04
    PUSH B        ; Dir. memoria 0x05
    MOV B,A       ; Dir. memoria 0x06
    ADD A,(res)   ; Dir. memoria 0x07
    MOV (res),A   ; Dir. memoria 0x08
    MOV A,B       ; Dir. memoria 0x09
    SUB A,1       ; Dir. memoria 0x0A
    CALL B,fun    ; Dir. memoria 0x0B
    POP B         ; Dir. memoria 0x0C-0x0D
    fun_end:
      RET B       ; Dir. memoria 0x0E

  end:
    MOV A,(res)   ; Dir. memoria 0x0F
```

Solución: El fragmento de código **(1)** accede, por *overflow* del registro `SP`, a la dirección de memoria 0 (equivalente a `var1`) para cargar su valor en el registro `PC`, por lo que la ejecución de `RET` genera el salto al *label* `case_2`. El fragmento de código **(2)**, por otra parte, computa de forma recursiva la suma de la variable `n` con todos sus antecesores hasta llegar a cero, almacenando el resultado final en `res`. A continuación, se muestra el resultado de la ejecución para ambos fragmentos:

(1)

```
DATA:
  var1 12     ; MEM[0] = 12
  var2 4      ; MEM[1] = 4
CODE:
  MOV A,5     ; A = 5
  MOV B,3     ; B = 3
  RET         ; SP++ => 255 + 1 = 0, PC = MEM[0] = 12
  case_1:
    ADD A,B   ; NO SE EJECUTA
    PUSH A    ; NO SE EJECUTA
    MOV A,B   ; NO SE EJECUTA
    SHL A,A   ; NO SE EJECUTA
    MOV B,A   ; NO SE EJECUTA
    POP A     ; NO SE EJECUTA
    JMP end   ; NO SE EJECUTA
  case_2:
    SUB A,B   ; A = 2
    PUSH A    ; MEM[SP] = A => MEM[0] = 2, SP-- => 0 - 1 = 255
    MOV A,B   ; A = 3
    SHR A,A   ; A = 1
    MOV B,A   ; B = 1
    POP A     ; SP++ => 255 + 1 = 0, A = MEM[0] = 2
  end: ; A = 2, B = 1
```

(2)

```
DATA:
  n    2          ; MEM[0] = 2
  res  0          ; MEM[1] = 0
CODE:
  MOV A,(n)       ; A = n = 2
  CALL B,fun      ; B = 2, fun(n)
  JMP end
  fun:
    CMP A,0
    JLE fun_end   ; Termina cuando A = 0
    PUSH B        ; Respalda B en stack (retorno)
    MOV B,A       ; B = A = n
    ADD A,(res)   ; A += res
    MOV (res),A   ; res = res + n
    MOV A,B       ; A = n
    SUB A,1       ; n--
    CALL B,fun    ; fun(n-1)
    POP B         ; Recupera B (retorno)
    fun_end:
      RET B       ; Retorna de la forma esperada
  end:
    ; A = res = fun(2) = 2+fun(1) = 2+1+fun(0) = 2+1+0 = 3
    MOV A,(res) ; B = Primer retorno computado = 2
```

Para cada fragmento de código, se otorgan **0.75 ptos.** por registro correctamente computado siempre que su cómputo esté respaldado por desarrollo o justificación. Si no se llega a ningún resultado final correcto por **error de arrastre**, se otorgan **0.5 ptos.** en el fragmento.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

(a) (1 pto.) Señale una ventaja y una desventaja de mapear los dispositivos I/O de un computador en memoria.

Solución: Una ventaja es que no implica modificaciones en la ISA, lo que permite comunicarse directamente con los dispositivos a través de instrucciones de lectura y escritura de memoria. Una desventaja, en cambio, es el uso significativo de direcciones de memoria a reservar, que en memorias más pequeñas puede implicar problemas de ejecución en los programas (*memory barrier*). Se otorgan **0.5 ptos.** por describir correctamente una ventaja y **0.5 ptos.** por describir correctamente una desventaja.

(b) (1 pto.) Explique el uso e importancia de las ISRs ejecutadas por la CPU en el contexto de comunicación vía interrupciones.

Solución: Las ISRs o *Interrupt Service Routines* son subrutinas asociadas al manejo que requieren los dispositivos por parte de la CPU una vez que son atendidas por la CPU. Son importantes ya que, a través de su ejecución, se lleva a cabo la comunicación entre la CPU y los dispositivos. Se otorgan **0.5 ptos.** por describir correctamente en qué consiste una ISR y **0.5 ptos.** por destacar su importancia.

(c) (4 ptos.) Suponga que decide automatizar la disponibilidad de un baño mediante sensores y un emisor Bluetooth conectados a un computador con arquitectura RISC-V. Para la comunicación con los sensores y el emisor, sus registros de 32 bits están mapeados en memoria desde la dirección `0x600DD0D0`:

Offset	**Nombre**	**Descripción**
0x00	`door_sensor_status`	Registro estado sensor de puerta
0x04	`light_sensor_status`	Registro estado sensor de luz
0x08	`notifier_command`	Registro comando emisor
0x0C	`notifier_status`	Registro estado emisor

Nombre	**Descripción**	**Valor**
`door_sensor_status`	Puerta cerrada	0
`door_sensor_status`	Puerta abierta	1
`light_sensor_status`	Luz apagada	0
`light_sensor_status`	Luz encendida	1
`notifier_status`	Inactivo	0
`notifier_status`	Activo	1
`notifier_status`	Enviando	2
`notifier_command`	Encender emisor	1
`notifier_command`	Notificar `OPEN`	2
`notifier_command`	Notificar `CLOSED`	3
`notifier_command`	Apagar emisor	4

El sistema debe enviar una notificación Bluetooth según la disponibilidad del baño: `OPEN` si es que está disponible, y `CLOSED` si es que está ocupado según los estados de los sensores de puerta (`door_sensor_status`) y luz (`light_sensor_status`). Para ello, se desarrolla el siguiente programa:

```
notifier_automation:
    # Carga zero + 0x600DD0D0 en t0
    addi t0, zero, 0x600DD0D0
    addi s0, zero, 0
    addi s1, zero, 1
    addi s2, zero, 2
    addi s3, zero, 3
    addi s4, zero, 4

step_one:
    lw t3, 12(t0) # Carga Mem[t0 + 12] en t3
    # Salta a step_two si t3 != s0
    bne t3, s0, step_two
    sw s1, 8(t0) # Carga s1 en Mem[t0 + 8]

step_two:
    lw t1, 0(t0)
    lw t2, 4(t0)
    bne t1, s1, step_three
    bne t2, s0, step_three
    # Salto incondicional a step_four
    jal zero, step_four
```

```
# => Continuación

step_three:
    sw s3, 8(t0)

    while_step_three:
        lw t3, 12(t0)
        # Salta a while_step_three si t3 == s2
        beq t3, s2, while_step_three

    sw s4, 8(t0)
    jal zero, step_one

step_four:
    sw s2, 8(t0)

    while_step_four:
        lw t3, 12(t0)
        beq t3, s2, while_step_four

    sw s4, 8(t0)
    jal zero, step_one
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(1 pto.)**: Se revisa el estado del emisor. Si el emisor está inactivo, se enciende y continúa a `step_two`. En otro caso, salta directamente a `step_two`.
- `step_two` **(1 pto.)**: Se revisa el estado del sensor de puerta y el sensor de luz del baño. Si la puerta está cerrada o la luz está encendida, salta a `step_three`. En otro caso, salta directamente a `step_four`.
- `step_three` **(1 pto.)**: A través del emisor, se envía la notificación `CLOSED`. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al `step_one`.
- `step_four` **(1 pto.)**: A través del emisor, se envía la notificación `OPEN`. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al `step_one`.

Para cada paso, se descuenta un 25 % del puntaje si existe como máximo un error o detalle faltante; un 50 % si existen como máximo dos errores; y un 100 % en otro caso.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Interrogación 2 - Parte 1 - Sección 1

Pauta de evaluación

Pregunta 1: Computador básico con subrutinas (6 ptos.)

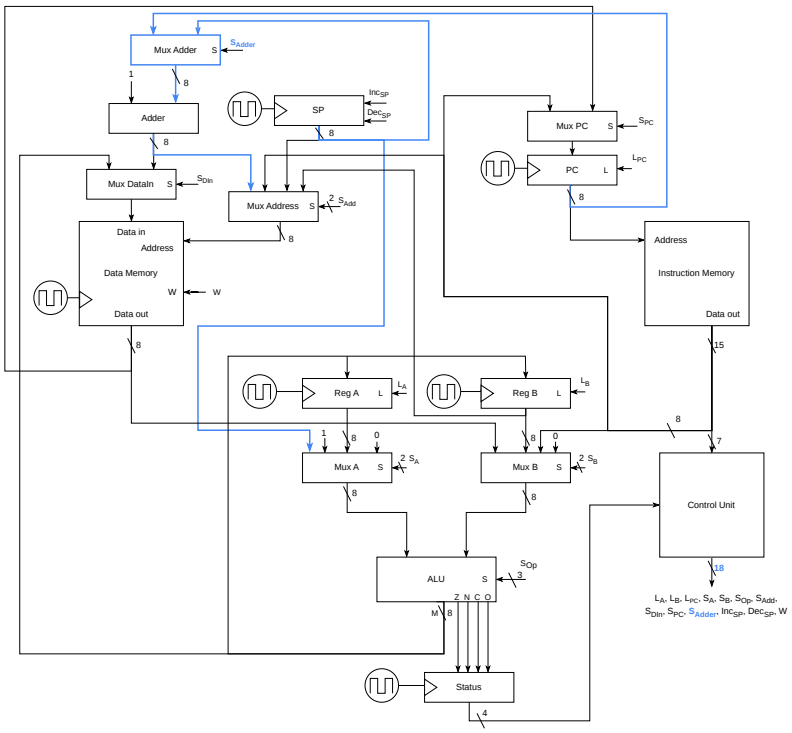
En los ítems **(a)**, **(b)** y **(c)**, se le describirán instrucciones que deberá implementar en el computador básico con subrutinas a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida ("-" si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. **Cada instrucción debe ejecutarse en un ciclo**.

(a) (1 pto.) `POPRD A`. Almacena el valor del tope del *stack* en el registro `A` sin modificar el *stack pointer*.

(b) (1 pto.) `MOV A, SP`. Almacena en el registro `A` el valor de `SP`.

(c) (1 pto.) `POP1 A`. Extrae el valor del tope del *stack* y lo almacena en el registro `A` **en un solo ciclo**. En otras palabras, es la versión óptima de `POP` al realizar la misma operación en un ciclo y no en dos.

> **Solución:** A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se modifica la entrada del *Adder* que computa `PC+1` para añadir un multiplexor `Mux Adder` con señal de selección S_{Adder}. Este selecciona entre el valor de `PC` y el de `SP` para realizar la suma, lo que da la posibilidad de computar `PC+1` o `SP+1`.
- Se conecta la salida del *Adder* con la entrada restante del `Mux Address`, permitiendo seleccionar `PC+1` o `SP+1` (el que nos interesa) como dirección de la memoria de datos.
- Se conecta la salida de `SP` con una entrada de `Mux A`, de forma que se pueda tener este valor como salida de la ALU para su almacenamiento en los registros `A` o `B`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	Inc_{SP}	Dec_{SP}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	S_{Adder}
`POPRD A`	1	0	0	0	0	0	ZERO	DOUT	ADD	ADDER	-	-	SP
`MOV A, SP`	1	0	0	0	0	0	SP	ZERO	ADD	-	-	-	-
`POP1 A`	1	0	0	1	0	0	ZERO	DOUT	ADD	ADDER	-	-	SP

Por cada instrucción, se otorgan:

- **0.5 ptos.** por *hardware* correctamente modificado; **0.3 ptos.** si existe **un solo error** de implementación; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso.
- **0.5 ptos.** por entregar una combinación de señales correcta; **0.3 ptos.** si existe **un solo error**; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso. Si una instrucción se define en más de un ciclo, tampoco se otorga puntaje en este criterio. Si la instrucción se implementa correctamente sin modificación de *hardware*, se otorga el puntaje completo.

(d) (3 ptos.) A continuación, se adjuntan dos fragmentos de código **(1)** y **(2)** escritos en el Assembly del computador básico, con la diferencia de que **(2)** incorpora las instrucciones implementadas en los ítems anteriores. Para cada fragmento, indique los valores de los registros `A` y `B` al finalizar la ejecución del código. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, **no se otorgará puntaje**.

(1)

```
DATA:
CODE:
  MOV A,7     ; Dir. memoria 0x00
  MOV B,1     ; Dir. memoria 0x01
  PUSH B      ; Dir. memoria 0x02
  PUSH A      ; Dir. memoria 0x03
  RET         ; Dir. memoria 0x04-0x05
  MOV A,0     ; Dir. memoria 0x06
  MOV B,0     ; Dir. memoria 0x07
  JMP end     ; Dir. memoria 0x08
end:
```

(2)

```
DATA:
  n    5
  res  0
CODE:
  MOV A,(n)             ; Dir. memoria 0x00
  CALL fun              ; Dir. memoria 0x01
  JMP end               ; Dir. memoria 0x02
  fun:
    CMP A,0             ; Dir. memoria 0x03
    JLE fun_end         ; Dir. memoria 0x04
    PUSH A              ; Dir. memoria 0x05
    SUB A,1             ; Dir. memoria 0x06
    JMP fun             ; Dir. memoria 0x07
    fun_end:
      loop:
        MOV A,SP        ; Dir. memoria 0x08
        CMP A,254       ; Dir. memoria 0x09
        JEQ end_loop    ; Dir. memoria 0x0A
        read:
          POPRD A       ; Dir. memoria 0x0B
          MOV B,A       ; Dir. memoria 0x0C
          AND A,1       ; Dir. memoria 0x0D
          JGT continue  ; Dir. memoria 0x0E
          MOV A,(res)   ; Dir. memoria 0x0F
          ADD (res)     ; Dir. memoria 0x10
        continue:
          POP1 A        ; Dir. memoria 0x11
          JMP loop      ; Dir. memoria 0x12
      end_loop:
        RET             ; Dir. memoria 0x13-0x14
  end:
    MOV A,(res)         ; Dir. memoria 0x15
```

Solución: El fragmento de código **(1)** gatilla la carga del valor numérico del tope del *stack* en el registro `PC` al ejecutar `RET`. Al leer este valor, el programa saltará a la dirección 0x07, saltándose la ejecución de `MOV A,0` (instrucción en la dirección 0x06). De esta forma, el valor final del registro `A` es igual a 7 y el del registro `B` es igual a 0.

El fragmento de código **(2)**, por otro lado, hace lo siguiente: (i) llama a la subrutina `fun` y *pushea* los valores `n`, `n-1`, `n-2`, ..., 1; (ii) revisa el valor del tope del *stack* –con `POPRD A`– y, **si es par** –`AND A,1` < 1–, lo suma a la variable `res`; (iii) independiente de su paridad, se descarta con `POP1` y repite el procedimiento hasta descartar todos los números *pusheados*; (iv) una vez que los revisa todos –`SP` $= 254$, siendo el tope del *stack* la dirección de retorno de `fun`–, retorna y almacena el valor de `res` en `A`. Por ende, `A` es igual a 6 ($4 + 2$, los únicos pares en el rango $[1, 5]$), mientras que `B` es igual a 5, dado que la última vez que se actualiza su valor es con la instrucción `MOV B,A` bajo el *label* `read`, en la iteración donde se lee el último valor *pusheado* al *stack* –correspondiente a `n` $= 5$–.

A continuación, se muestra el resultado de la ejecución para ambos fragmentos:

(1)

```
DATA:
CODE:
  MOV A,7     ; A = 7
  MOV B,1     ; B = 1
  PUSH B      ; MEM[255] = 1, SP = 254
  PUSH A      ; MEM[254] = 7, SP = 253
  RET         ; SP = 254
              ; PC = MEM[254] = 7
  MOV A,0     ; No ejecuta
  MOV B,0     ; RET carga aquí -> B = 0
  JMP end     ; Termina programa
end:          ; A = 7, B = 0
```

(2)

```
DATA:
  n    5
  res  0
CODE:
  MOV A,(n)             ; A = 5
  CALL fun              ; MEM[255] = 0x02
  JMP end               ; Termina el programa
  fun:
    CMP A,0             ; if A <= 0
    JLE fun_end         ;   -> Salta a fun_end
    PUSH A              ; MEM[254] = 5, MEM[253] = 4, ..., MEM[250] = 1 y SP = 249
    SUB A,1             ; A = 4, 3, 2, 1, 0
    JMP fun             ; Repetir hasta que A == 0
    fun_end:
      loop:
        MOV A,SP        ; A = 249, 250, ..., 254
        CMP A,254       ; if A == 254
        JEQ end_loop    ;   -> Salta a end_loop
        read:
          POPRD A       ; A = MEM[250] = 1, MEM[251] = 2, ..., MEM[254] = 5
          MOV B,A       ; B = 1, 2, ..., 5 (valor final de B)
          AND A,1       ; 1 AND 1 = 1, 2 AND 1 = 0, 3 AND 1 = 1, 4 AND 1 = 0, 5 AND 1 = 1
          JGT continue  ; Si A es impar, continúa sin actualizar res
          MOV A,(res)   ; e.o.c. (A par), A = res
          ADD (res)     ; res = A + B -> res = res + B -> res += 2, 4 (suma solo pares)
        continue:
          POP1 A        ; A = MEM[250] y SP = 250, ..., A = MEM[254] y SP = 254
          JMP loop      ; Repetir hasta que SP == 254
      end_loop:
        RET             ; PC = MEM[255] = 0x02 (retorna a JMP end)
  end:
    MOV A,(res)         ; A = res = 2 + 4 = 6; B = 5 (última actualización de MOV B,A)
```

Para cada fragmento, se otorgan **0.75 ptos.** por registro correctamente computado siempre que su cómputo esté respaldado por desarrollo o justificación. Si no se llega a ningún resultado final correcto por **error de arrastre**, se otorgan **0.5 ptos.** en el fragmento.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Interrogación 2 - Parte 1 - Sección 2

Pauta de evaluación

Pregunta 1: Computador básico con subrutinas (6 ptos.)

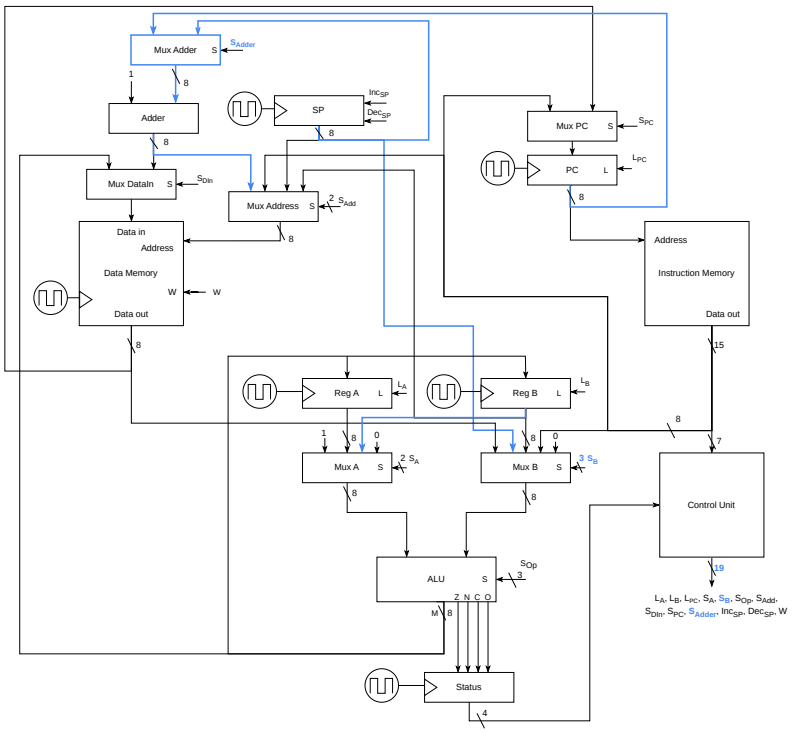
En los ítems **(a)**, **(b)** y **(c)**, se le describirán instrucciones que deberá implementar en el computador básico con subrutinas a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida ("-" si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. **Cada instrucción debe ejecutarse en un ciclo**.

(a) (1 pto.) `PUSHRW A`. Sobreescribe el tope del *stack* con el valor del registro `A` sin modificar el *stack pointer*.

(b) (1 pto.) `CMP B, SP`. Computa la operación `B - SP` sin actualizar los registros.

(c) (1 pto.) `RET1`. Salta a la dirección de retorno almacenada en el tope del *stack* **en un solo ciclo**. En otras palabras, es la versión óptima de `RET` al realizar la misma operación en un ciclo y no en dos.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se modifica la entrada del *Adder* que computa `PC+1` para añadir un multiplexor `Mux Adder` con señal de selección S_{Adder}. Este selecciona entre el valor de `PC` y el de `SP` para realizar la suma, lo que da la posibilidad de computar `PC+1` o `SP+1`.
- Se conecta la salida del *Adder* con la entrada restante del `Mux Address`, permitiendo seleccionar `PC+1` o `SP+1` (el que nos interesa) como dirección de la memoria de datos.
- Se conecta la salida de `B` con una entrada de `Mux A` y la salida de `SP` con una entrada de `Mux B`, de forma que puedan seleccionarse como primer y segundo operando de la ALU respectivamente. Esto requiere incrementar a 3 bits la señal S_B.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	Inc_{SP}	Dec_{SP}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}	S_{Adder}
PUSHRW A	0	0	0	0	0	1	A	ZERO	ADD	ADDER	ALU	-	SP
CMP B, SP	0	0	0	0	0	0	B	SP	SUB	-	-	-	-
RET1	0	0	1	1	0	0	-	-	-	ADDER	-	DOUT	SP

Por cada instrucción, se otorgan:

- **0.5 ptos.** por *hardware* correctamente modificado; **0.3 ptos.** si existe **un solo error** de implementación; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso.
- **0.5 ptos.** por entregar una combinación de señales correcta; **0.3 ptos.** si existe **un solo error**; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso. Si una instrucción se define en más de un ciclo, tampoco se otorga puntaje en este criterio. Si la instrucción se implementa correctamente sin modificación de *hardware*, se otorga el puntaje completo.

(d) (3 ptos.) A continuación, se adjuntan dos fragmentos de código **(1)** y **(2)** escritos en el Assembly del computador básico, con la diferencia de que **(2)** incorpora las instrucciones implementadas en los ítems anteriores. Para cada fragmento, indique los valores de los registros `A` y `B` al finalizar la ejecución del código. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, **no se otorgará puntaje**.

(1)

```
DATA:
CODE:
  MOV A,2          ; Dir. memoria 0x00
  MOV B,5          ; Dir. memoria 0x01
  PUSH A           ; Dir. memoria 0x02
  PUSH B           ; Dir. memoria 0x03
  CALL mystery     ; Dir. memoria 0x04
  POP B            ; Dir. memoria 0x05-0x06
  POP A            ; Dir. memoria 0x07-0x08
  JMP end          ; Dir. memoria 0x09
  mystery:
    MOV A,0        ; Dir. memoria 0x0A
    NOT A,A        ; Dir. memoria 0x0B
    SUB A,2        ; Dir. memoria 0x0C
    MOV B,A        ; Dir. memoria 0x0D
    MOV A,(B)      ; Dir. memoria 0x0E
    ADD A,4        ; Dir. memoria 0x0F
    MOV (B),A      ; Dir. memoria 0x10
    RET            ; Dir. memoria 0x11-0x12
  end:
```

(2)

```
DATA:
  n    5
  res  0
CODE:
  MOV A,(n)          ; Dir. memoria 0x00
  CALL fun           ; Dir. memoria 0x01
  JMP end            ; Dir. memoria 0x02
  fun:
    CMP A,0          ; Dir. memoria 0x03
    JLE fun_end      ; Dir. memoria 0x04
    PUSH A           ; Dir. memoria 0x05
    MOV B,A          ; Dir. memoria 0x06
    AND A,1          ; Dir. memoria 0x07
    JLE rewrite      ; Dir. memoria 0x08
    continue:
      MOV A,B        ; Dir. memoria 0x09
      SUB A,1        ; Dir. memoria 0x0A
      JMP fun        ; Dir. memoria 0x0B
    rewrite:
      MOV A,0        ; Dir. memoria 0x0C
      PUSHRW A       ; Dir. memoria 0x0D
      JMP continue   ; Dir. memoria 0x0E
    fun_end:
      MOV B,254      ; Dir. memoria 0x0F
      loop:
        CMP B,SP     ; Dir. memoria 0x10
        JEQ end_loop ; Dir. memoria 0x11
        POP A        ; Dir. memoria 0x12-0x13
        ADD A,(res)  ; Dir. memoria 0x14
        MOV (res),A  ; Dir. memoria 0x15
        JMP loop     ; Dir. memoria 0x16
      end_loop:
        RET1         ; Dir. memoria 0x17
  end:
```

Solución: El fragmento de código **(1)**, a través de la subrutina `mystery`, **sobreescribe la dirección de retorno de su llamado**. Si bien este valor era 0x05 (`POP B`), este se lee con direccionamiento indirecto –`MOV A,(B)`–, se incrementa en 4 unidades –`ADD A,4`–, y luego se sobreescribe de vuelta –`MOV (B), A`–, generando que `RET` salte directamente a 0x09 (`JMP end`). Esto implica que el valor final del registro `A` sea igual a 9 (computado en 0x0F), y que el de `B` sea igual a 253 o -3, según interpretación (computado en 0x0D).

El fragmento de código **(2)**, por otro lado, hace lo siguiente: (i) llama a la subrutina `fun` y *pushea* los valores `n`, `n-1`, `n-2`, ..., 1; (ii) al *pushear*, revisa si el valor escrito es par –`AND A,1` < 1– y, en dicho caso, lo **sobreescribe por cero** –`PUSHRW A` en 0x0D–; (iii) al terminar de *pushear* y sobreescribir los pares, descarta el contenido del *stack* y acumula la suma de los elementos en `res` a través de `A`; (iv) una vez que los revisa todos –`SP` $= 254$, siendo el tope del *stack* la dirección de retorno de `fun`–, retorna y termina el programa. Por ende, `A` es igual a 9 ($5 + 3 + 1$, los impares del rango $[1, 5]$), mientras que `B` es igual a 254, ya que la última vez que actualiza su valor es con `MOV B,254` en 0x0F.

A continuación, se muestra el resultado de la ejecución para ambos fragmentos:

(1)

```
DATA:
CODE:
  MOV A,2          ; A = 2
  MOV B,5          ; B = 5
  PUSH A           ; MEM[255] = 2, SP = 254
  PUSH B           ; MEM[254] = 5, SP = 253
  CALL mystery     ; MEM[253] = 0x05, SP = 252
  POP B            ; No se ejecuta
  POP A            ; No se ejecuta
  JMP end          ; RET salta aquí -> Termina programa
  mystery:
    MOV A,0        ; A = 0
    NOT A,A        ; A = 11111111b = 255
    SUB A,2        ; A = 253
    MOV B,A        ; B = 253
    MOV A,(B)      ; A = MEM[253] = 5
    ADD A,4        ; A = 5 + 4 = 9
    MOV (B),A      ; MEM[253] = 9
    RET            ; SP = 253
                   ; PC = MEM[253] = 9 = 0x09
  end:             ; A = 9, B = 253
```

(2)

```
DATA:
  n    5
  res  0
CODE:
  MOV A,(n)           ; A = 5
  CALL fun            ; MEM[255] = 0x02
  JMP end             ; Termina el programa
  fun:
    CMP A,0           ; if A <= 0
    JLE fun_end       ;   -> Salta a fun_end
    PUSH A            ; MEM[254] = 5, MEM[253] = 4, ..., MEM[250] = 1 y SP = 249
    MOV B,A           ; B = 5, 4, 3, 2, 1
    AND A,1           ; 5 AND 1 = 1, 4 AND 1 = 0, 3 AND 1 = 1, 2 AND 1 = 0, 1 AND 1 = 1
    JLE rewrite       ; Si A es par, salta a rewrite
    continue:         ; e.o.c. es impar y continua sin reescribir
      MOV A,B         ; A = 5, 4, 3, 2, 1
      SUB A,1         ; A = 4, 3, 2, 1, 0
      JMP fun         ; Repetir hasta que A == 0
    rewrite:
      MOV A,0         ; A = 0
      PUSHRW A        ; MEM[253] = 0, MEM[251] = 0
      JMP continue    ; Salta a continue
    fun_end:
      MOV B,254       ; B = 254
      loop:
        CMP B,SP      ; if SP == B (se revisaron los elementos pusheados)
        JEQ end_loop  ;   -> Salta a end_loop
        POP A         ; A = MEM[250] y SP = 250, ..., A = MEM[254] y SP = 254
        ADD A,(res)   ; A += res (acumula suma en res)
        MOV (res),A   ; res = A (actualiza res)
        JMP loop      ; Repetir hasta que SP == 254
      end_loop:
        RET1          ; PC = MEM[255] = 0x02 (retorna a JMP end en un ciclo)
  end:                ; A = res = 5 + 3 + 1 = 9; B = 254 (última actualización con MOV B,254)
```

Para cada fragmento, se otorgan **0.75 ptos.** por registro correctamente computado siempre que su cómputo esté respaldado por desarrollo o justificación. Si no se llega a ningún resultado final correcto por **error de arrastre**, se otorgan **0.5 ptos.** en el fragmento.